



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

学院: 计算机科学与技术学院

时间: 2026春季学期

提纲

CONTENTS

9.1 查找的基本概念

9.2 线性表的查找

9.3 树表的查找

9.4 哈希表的查找

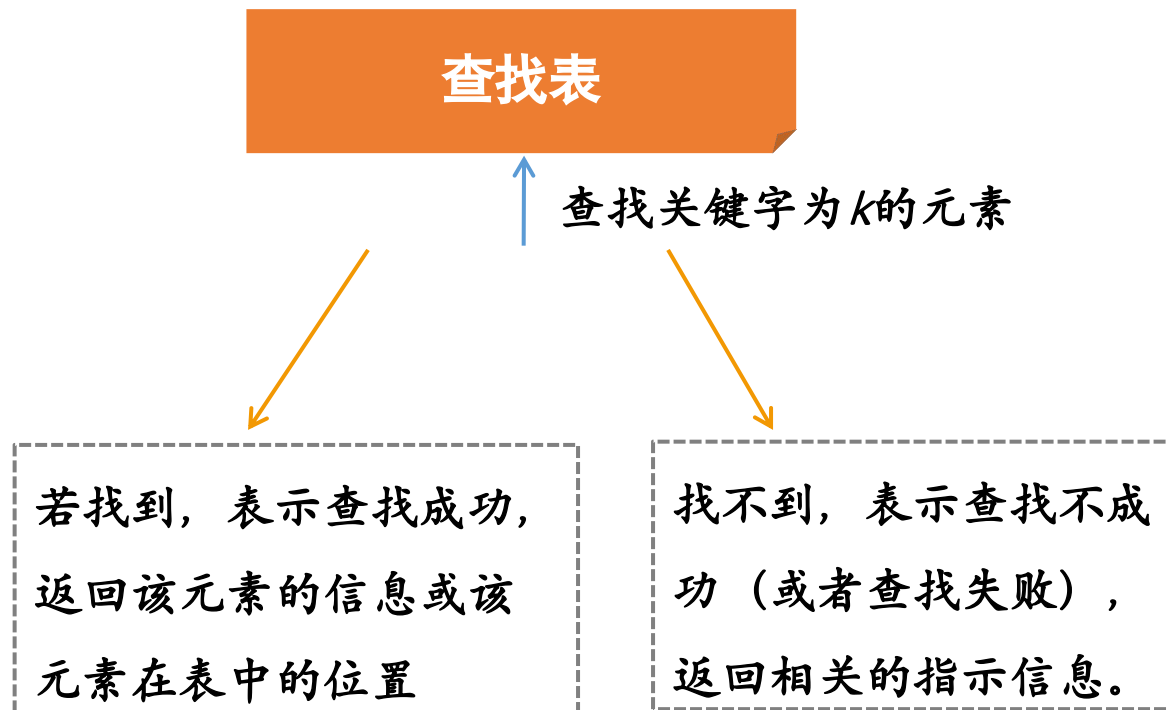


9.1 查找的基本概念

● 1、查找的定义

查找表

是由一组元素组成的表或文件，而每个元素由若干个数据项组成，并假设每个元素都有一个能唯一标识该元素的关键字。



2、内查找和外查找

采用数组或者链表存储的数据均在内存中



若整个查找过程都在内存进行，则称之为内查找；

反之，若查找过程中需要访问外存，则称之为外查找。

3、查找的数据组织

采用何种存储结构？

• 顺序表

• 链表

• 其他

若在查找的同时对表做修改操作（如插入和删除），则相应的表称之为动态查找表；
否则称之为静态查找表。

4、影响查找的因素

采用何种查找方法？

- 使用哪种数据结构来表示“表”，即表中元素是按何种方式组织的？
- 表中关键字的次序。是对无序集合查找还是对有序集合查找？

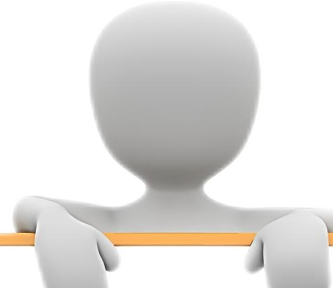
5. 查找方法的性能指标

● 5. 查找方法的性能指标

- 查找运算时间主要花费在关键字比较上，通常把查找过程中执行的关键字平均比较个数（称为平均查找长度）作为衡量一个查找算法效率优劣的标准。
- 平均查找长度ASL（Average Search Length）定义为：

$$ASL = \sum_{i=1}^n p_i c_i$$

- n 是查找表中元素的个数。
- p_i 是查找第 i 个元素的概率，一般地，认为每个元素的查找概率相等，即 $p_i = 1/n$ ($1 \leq i \leq n$)。
- c_i 是找到第 i 个元素所需进行的比较次数。



平均查找长度ASL分为

- 成功情况下的平均查找长度
—— $ASL_{成功}$
- 不成功情况（失败）下的平均查找长度
—— $ASL_{不成功}$

查找表T：含有n个元素

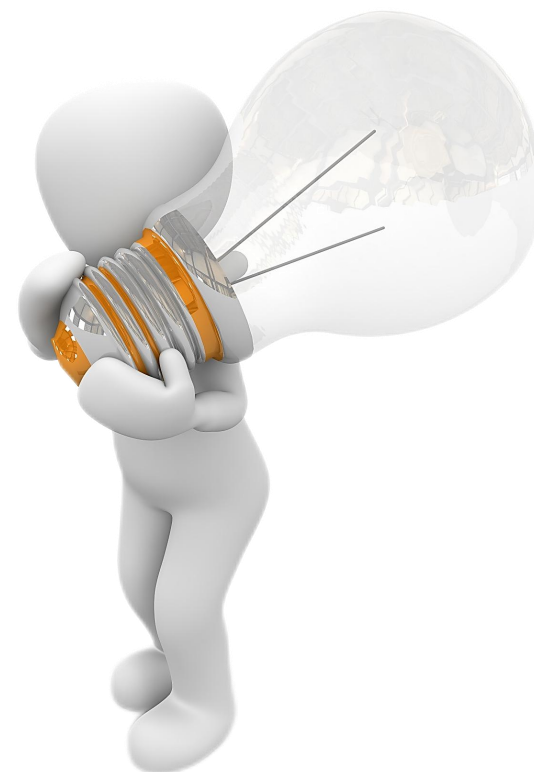
$$ASL = \sum_{i=1}^n p_i c_i$$

- 成功情况下（概率相等）的平均查找长度 $ASL_{成功}$ 是指**找到T中任一元素平均需要的关键字比较次数**。
- 例如， $n=9$ ：

关键字	5	1	4	8	7	9	2	4	3
找到时的比较次数	1	2	3	4	5	6	7	8	9

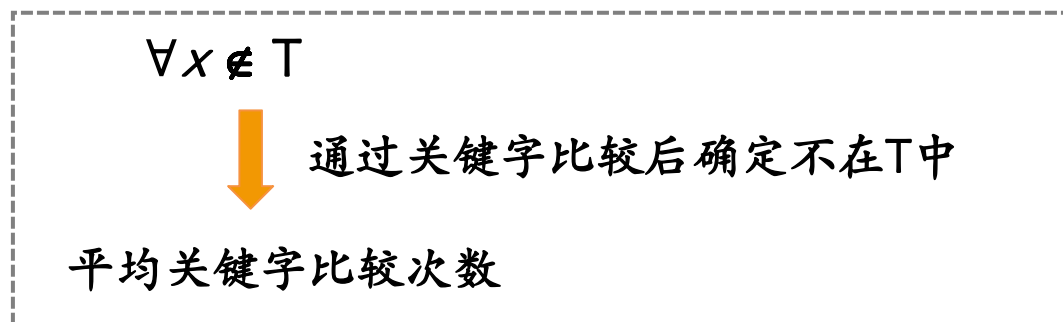


$$ASL_{成功} = \frac{1+2+3+4+5+6+7+8+9}{9} = 5$$



查找表T: 含有n个元素

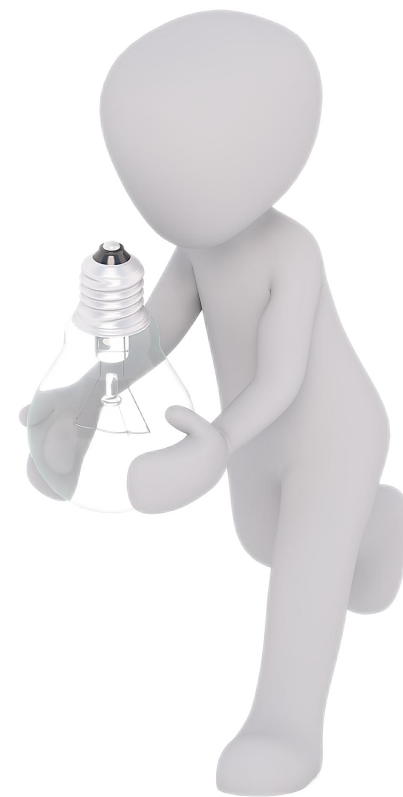
- 不成功情况下的平均查找长度 $ASL_{\text{不成功}}$ 是指**查找失败（在T中未查找
到）平均需要的关键字比较次数。**



- 例如, $n=9$:

关键字	5	1	4	8	7	9	2	4	3
找到时的比较次数	1	2	3	4	5	6	7	8	9

- 按顺序查找, $ASL_{\text{不成功}}=n=9$

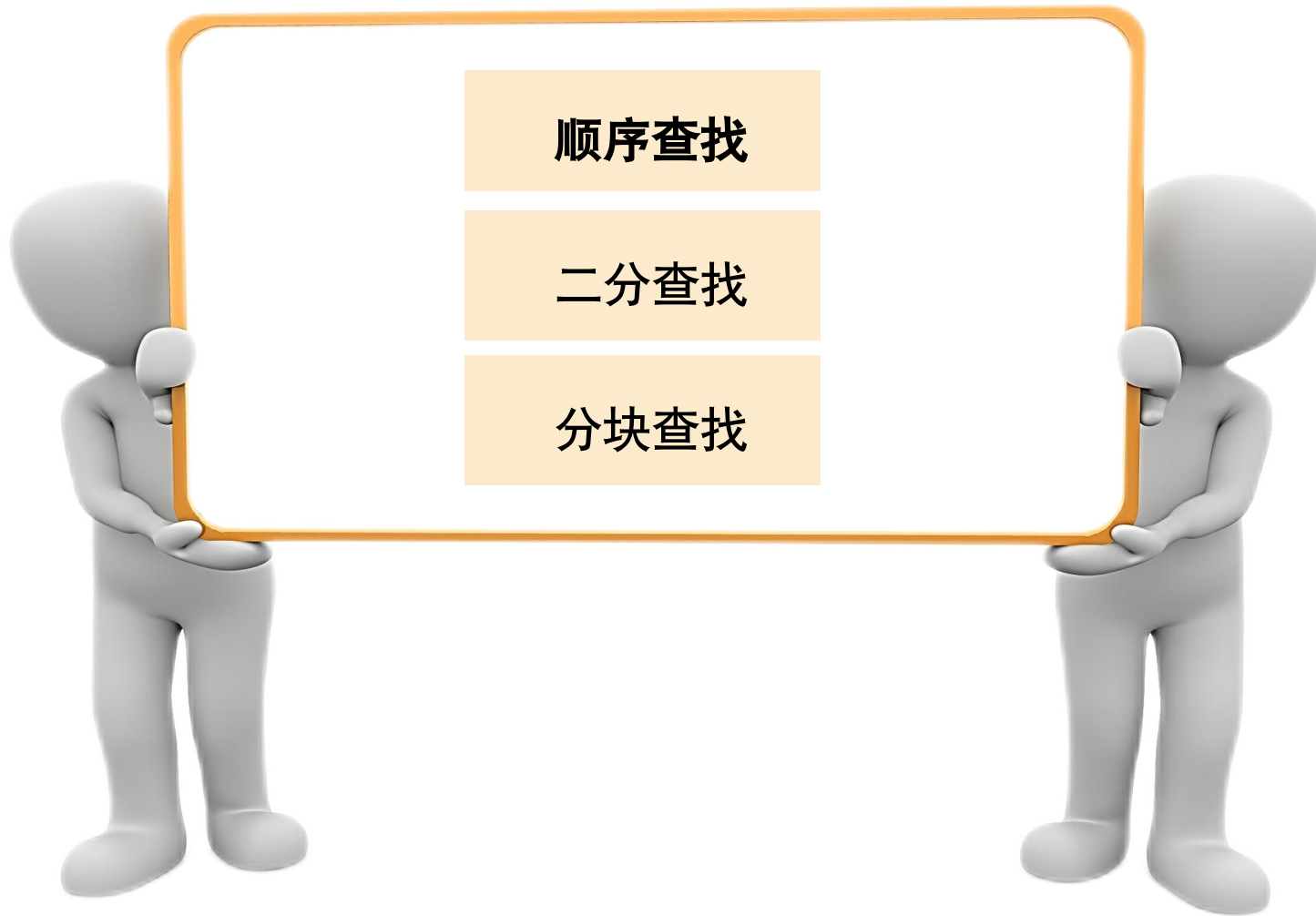


1、线性表查找的主要方法

顺序查找

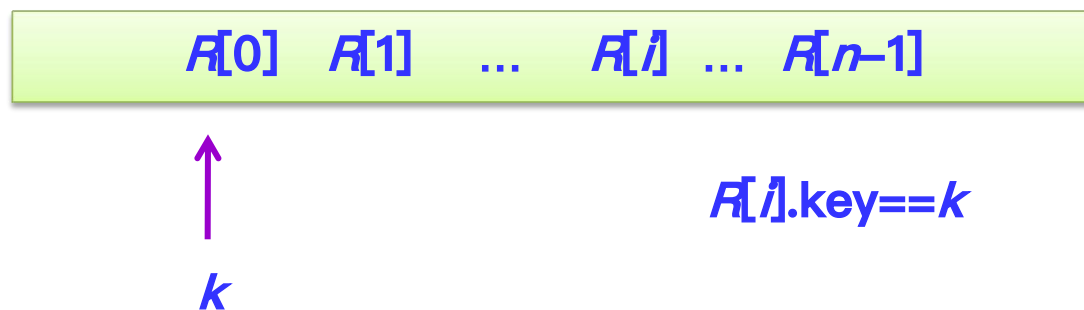
二分查找

分块查找



9.2.1 顺序查找

思路：从表的一端开始，顺序扫描线性表，依次将扫描到的关键字和给定值 k 相比较：



- 若当前扫描到的关键字与 k 相等，则查找成功；
- 若扫描结束后，仍未找到关键字等于 k 的记录，则查找失败。

9.2.1 顺序查找

顺序查找的算法如下（在顺序表 $R[0..n-1]$ 中查找关键字为 k 的元素，成功时返回找到的元素的逻辑序号，失败时返回0）：

```
int SeqSearch(RecType R[],int n,KeyType k)
{ int i=0;
  while (i<n && R[i].key!=k)    //从表头往后找
    i++;
  if (i>=n)                      //未找到返回0
    return 0;
  else
    return i+1;                  //找到返回逻辑序号i+1
}
```



顺序查找

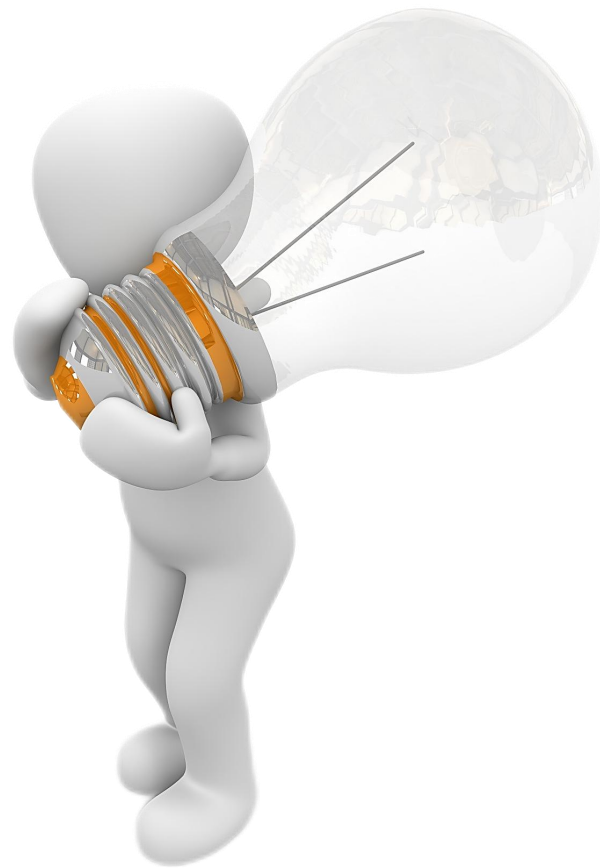
成功情况下的平均查找长度——ASL成功

- 有n个元素，等概率时 $p_i=1/n$ 。
- 查找到表中第i个元素 $R[i-1]$ 时，需比较i次。
- 因此成功时的顺序查找的平均查找长度为：

$$ASL_{\text{成功}} = \sum_{i=1}^n p_i c_i = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \times \frac{n(n+1)}{2} = \frac{n+1}{2}$$



查找成功时的平均比较次数约为表长的一半。



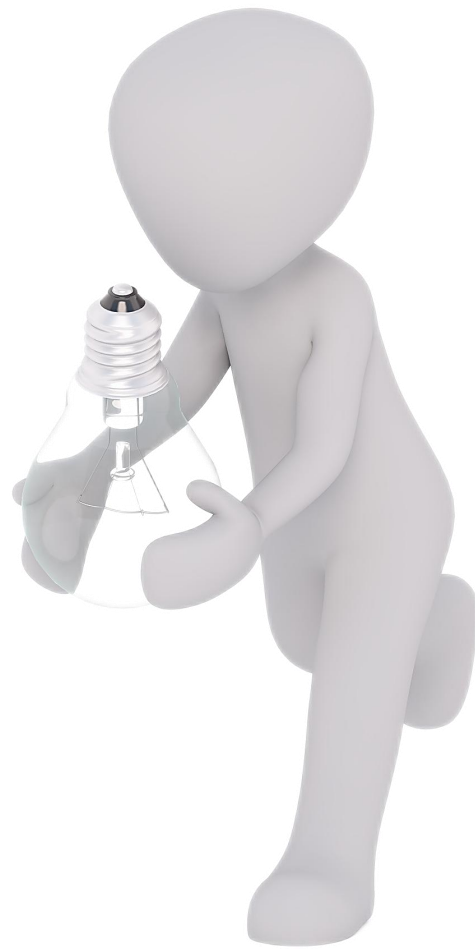
顺序查找

不成功情况下的平均查找长度——ASL不成功

- 查找不成功时需要和表中所有元素都比较一次。
- $ASL_{\text{不成功}} = n$ 。



顺序查找的时间复杂度为 $O(n)$ 。



示例

对含有 n 个元素的线性表采用顺序查找方法，假设查找成功和不成功的可能性相同，对每个元素的查找概率也相同，此时顺序查找的平均查找长度为（ ）。

A. $(n+1)/2$ B. $(n+1)/4$ C. $(n-1)/2$ D. $(3n+1)/4$

- 查找成功和不成功的概率均为 $1/2$
- 查找成功时，每个元素的查找概率为 $(1/2)/n$ ，查找 k_i ($1 \leq i \leq n$) 比较次数为 i ，有：

$$ASL_{\text{成功}} = \sum_{i=1}^n i \times \frac{1}{2n} = \frac{n+1}{4}$$

- 查找不成功时，每个关键字都需要比较 n 次，有：

$$ASL = ASL_{\text{成功}} + ASL_{\text{不成功}} = \frac{n+1}{4} + \frac{n}{2} = \frac{3n+1}{4}$$

答：D。

对长度为 3 的顺序表进行查找，若查找第一个元素的概率为 $1/2$ ，查找第二个元素的概率为 $1/3$ ，查找第三个元素的概率为 $1/6$ ，则查找任一元素的平均查找长度为（）

A. $5/3$

B. 2

C. $7/3$

D. $4/3$

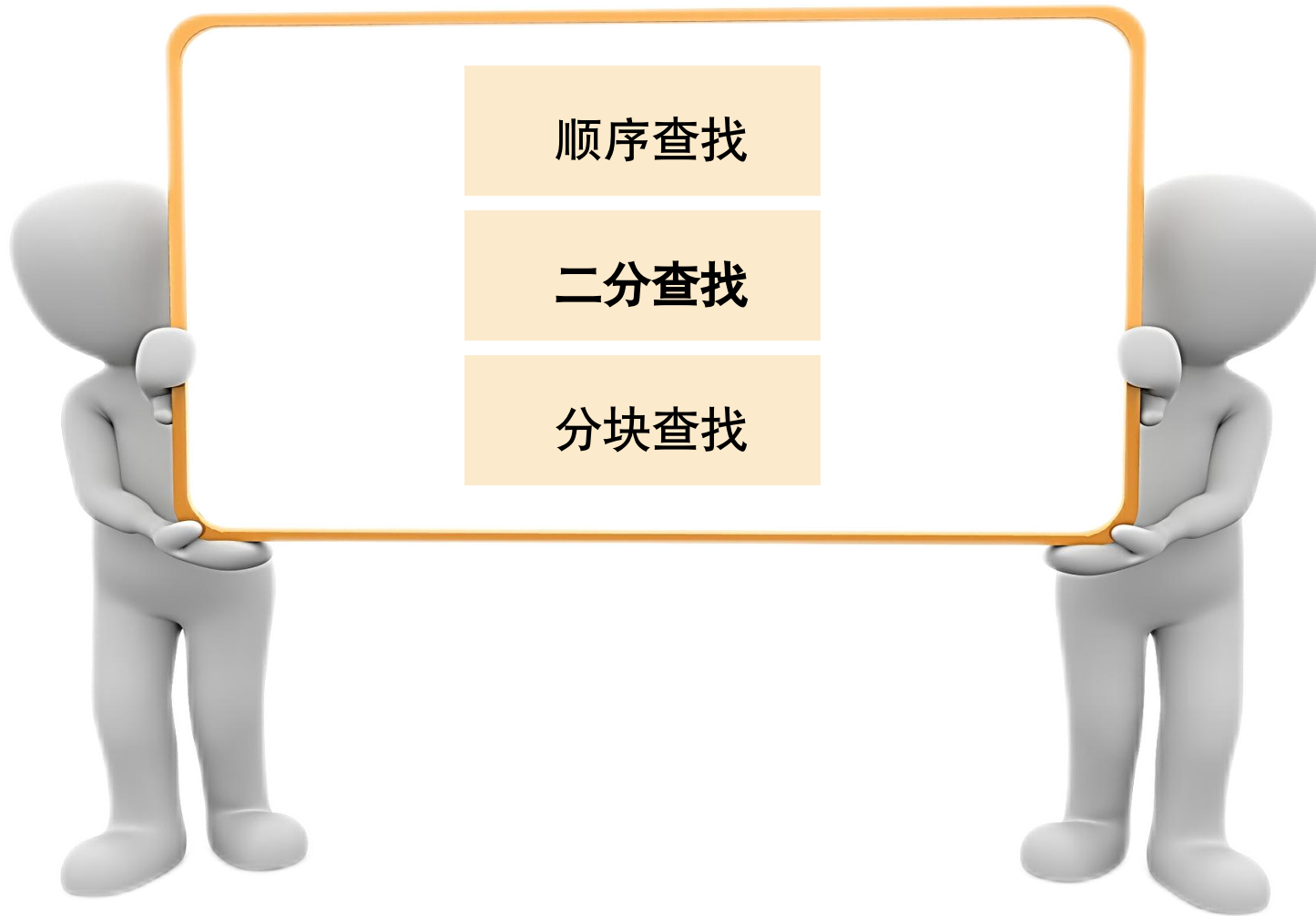
答案：A

1、线性表查找的主要方法

顺序查找

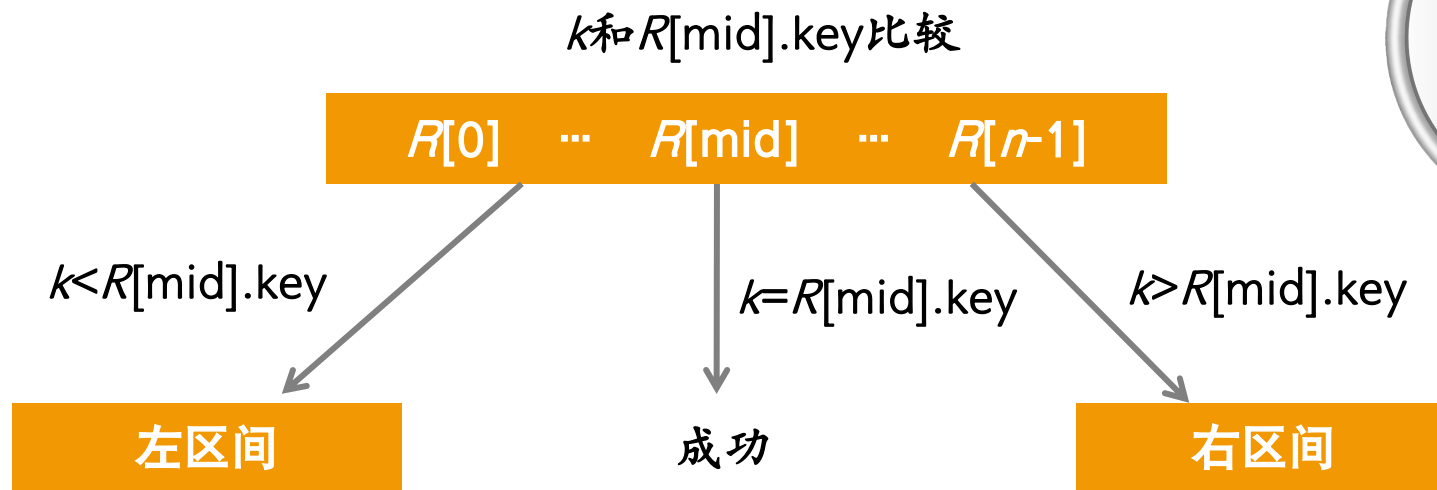
二分查找

分块查找



9.2.2 折半查找

- 折半查找也称为二分查找，要求线性表中的元素必须按关键字值有序（递增或递减）排列。
- 思路：



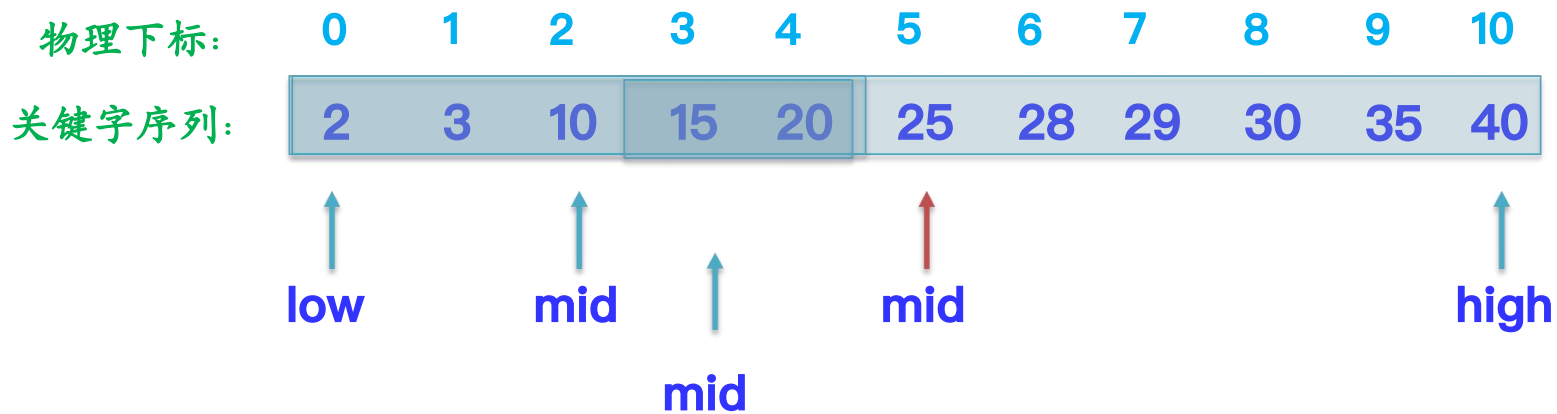
折半查找演示

例如，在关键字有序序列：

(2,3,10,15,20,25,28,29,30,35,40)

中采用折半查找法查找关键字为15的元素。

找关键字为15的记录



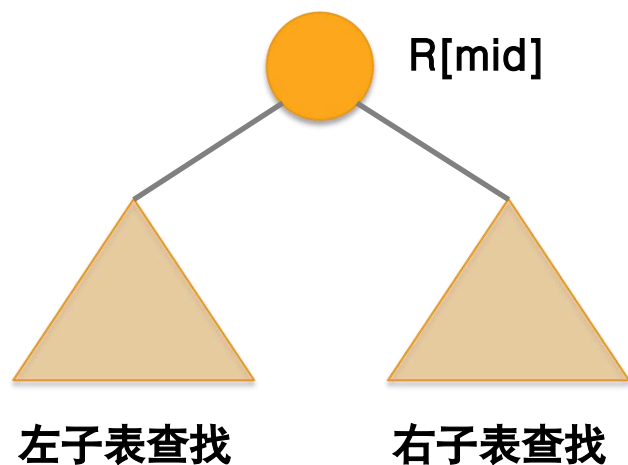
查找成功，关键字为15的记录的逻辑序号为4

关键字比较次数为**3**

二分查找过程

二分查找过程可用二叉树来描述：

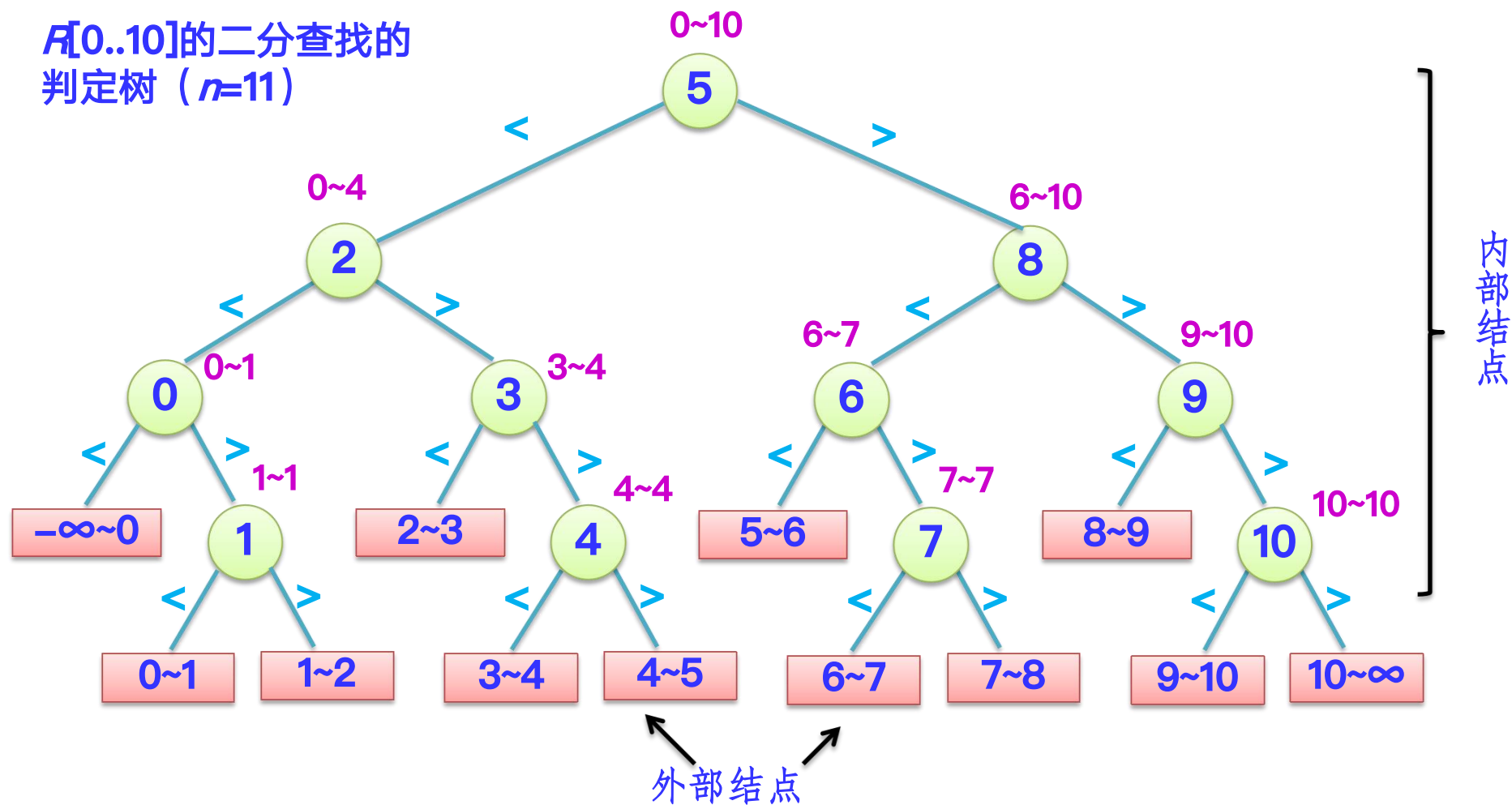
- 把当前查找区间的中间位置上的元素作为**根**；
- 左子表和右子表中的元素分别作为根的**左子树**和**右子树**。



这样的二叉树称为判定树或比较树。



$R[0..10]$ 的二分查找的
判定树 ($n=11$)



外部结点即查找失败对应的结点，是虚拟的

n 个关键字：内部结点为 n 个，外部结点为 $n+1$ 个

示例

【例9.1】对于给定11个数据元素的有序表：

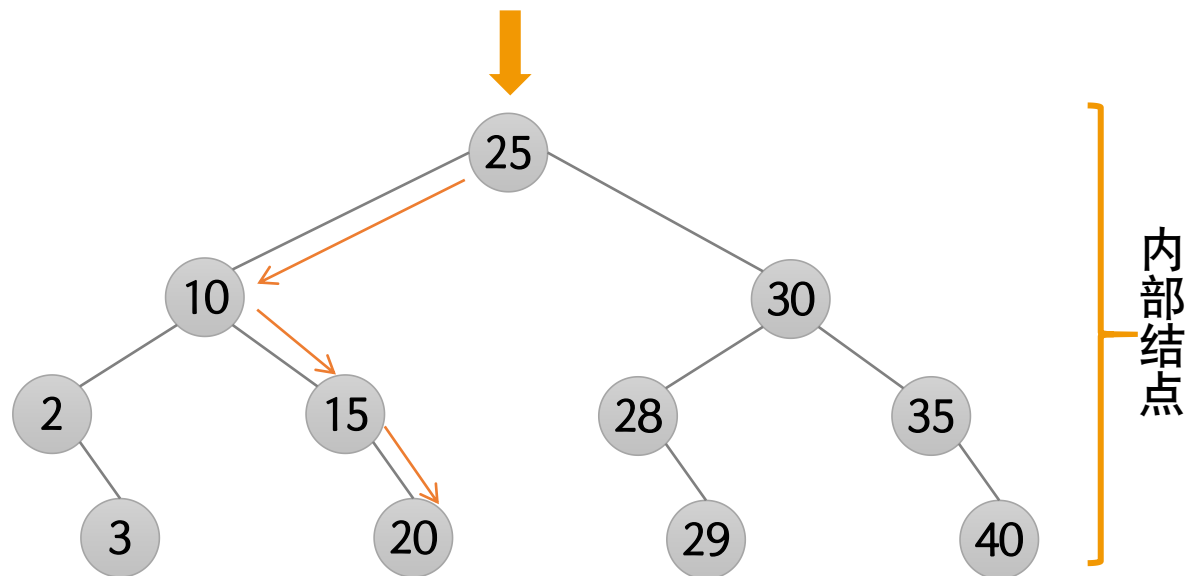
(2, 3, 10, 15, 20, 25, 28, 29, 30, 35, 40)

采用二分查找，试问：

1. 若查找给定值为20的元素，将依次与表中哪些元素比较？
2. 若查找给定值为26的元素，将依次与哪些元素比较？
3. 假设查找表中每个元素的概率相同，求查找成功时的平均查找长度和查找不成功时的平均查找长度。

判定树

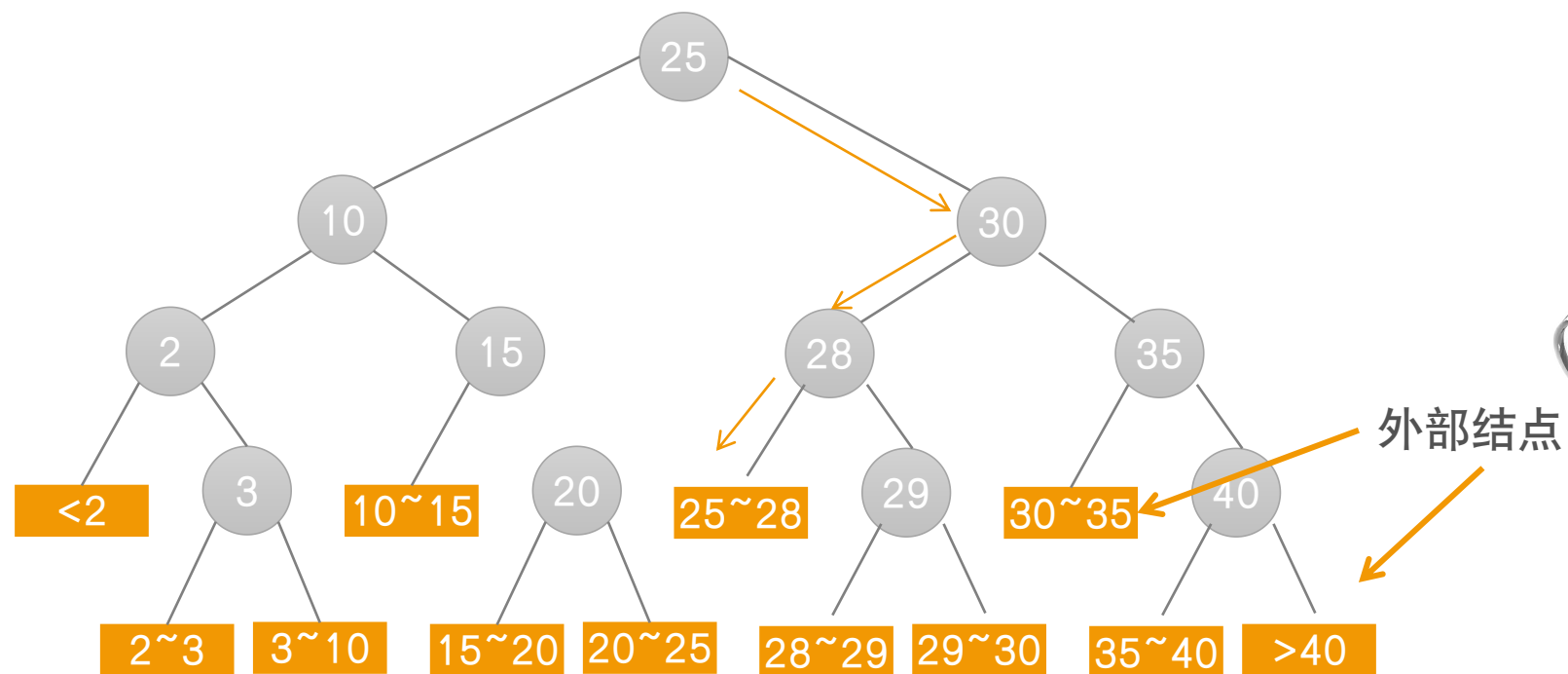
0 1 2 3 4 5 6 7 8 9 10
(2, 3, 10, 15, 20, 25, 28, 29, 30, 35, 40)



(1) 若查找给定值为20的元素，依次与表中25、10、15、20元素比较，共比较4次。

关键字比较的个数

带外部结点的判定树

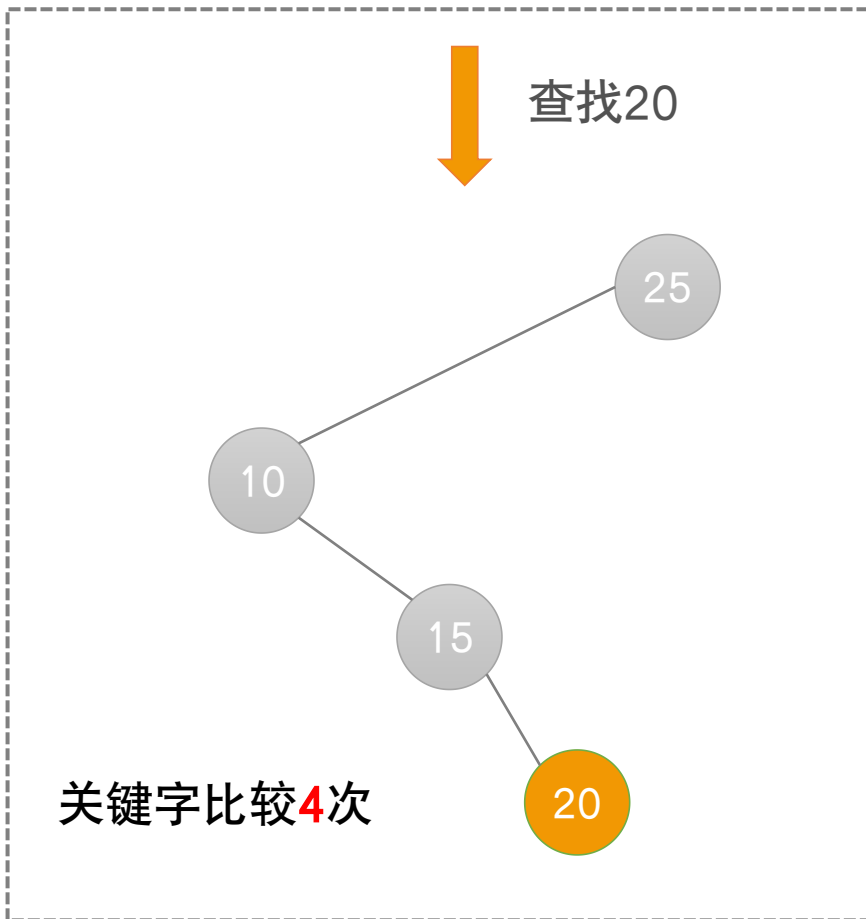


(2) 若查找给定值为26的元素，依次与25、30、28元素比较，共比较3次。

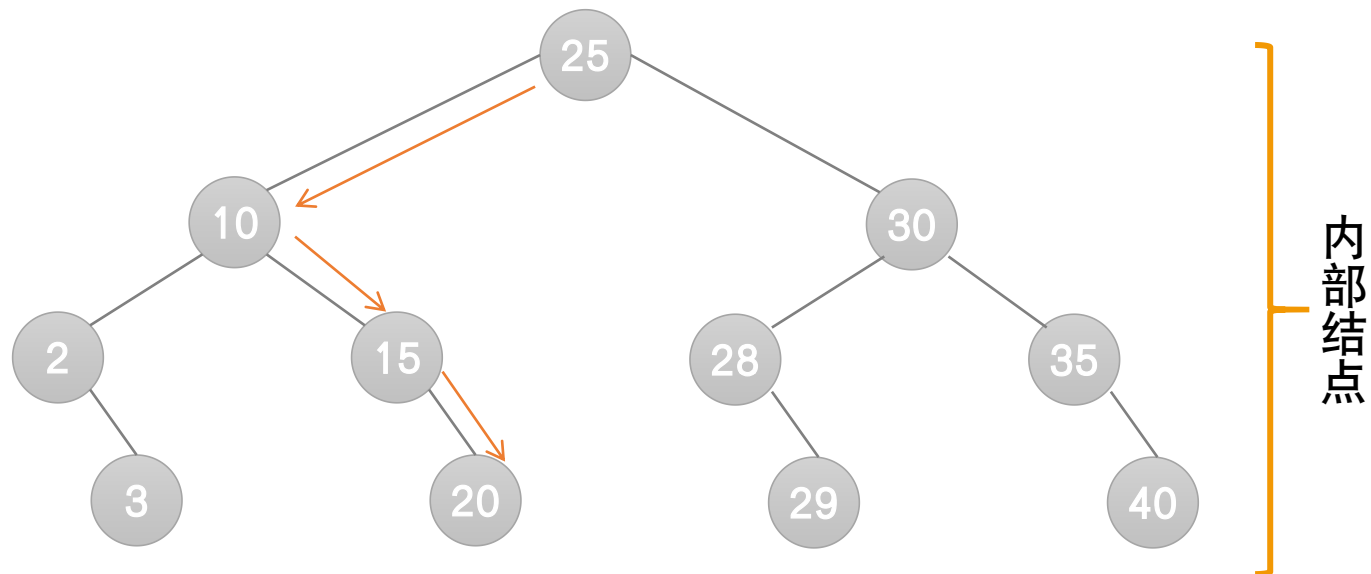
成功二分查找



恰好是走了一条从判定树的根到被查元素的路径，经历比较的关键字次数恰为该元素在树中的层次。



判定树



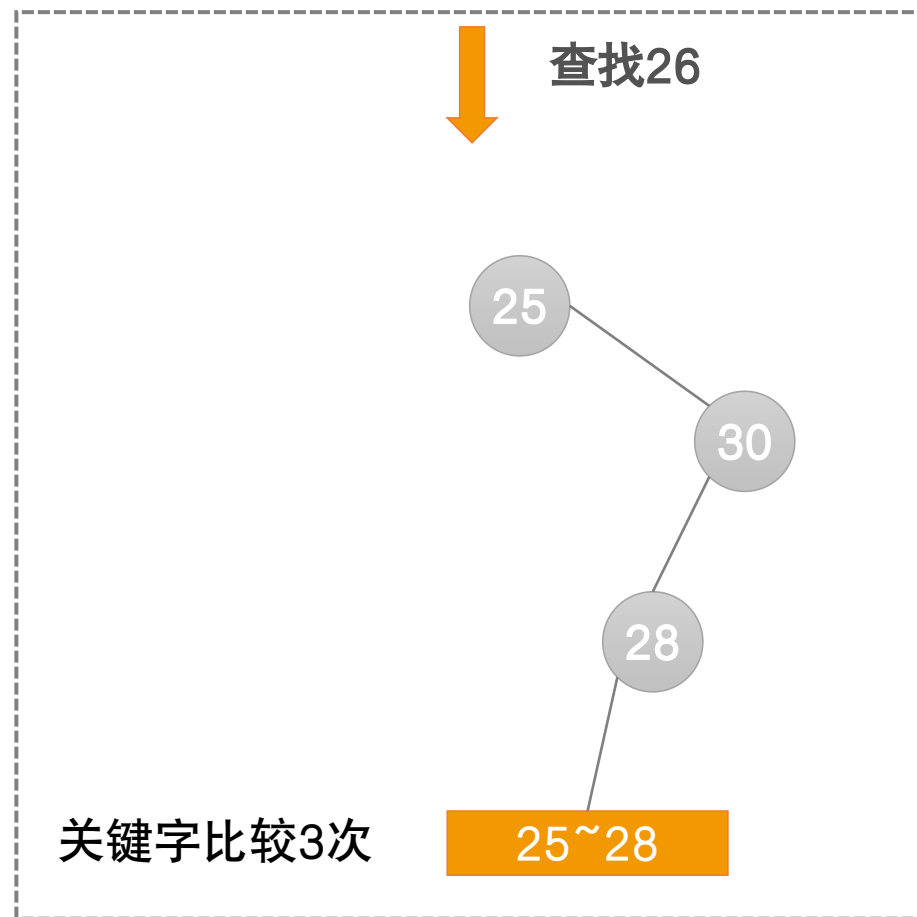
(3) 在查找成功时，会找到某个内部结点，则**成功**时的平均查找长度：

$$ASL_{\text{成功}} = \frac{1 \times 1 + 2 \times 2 + 4 \times 3 + 4 \times 4}{11} = 3$$

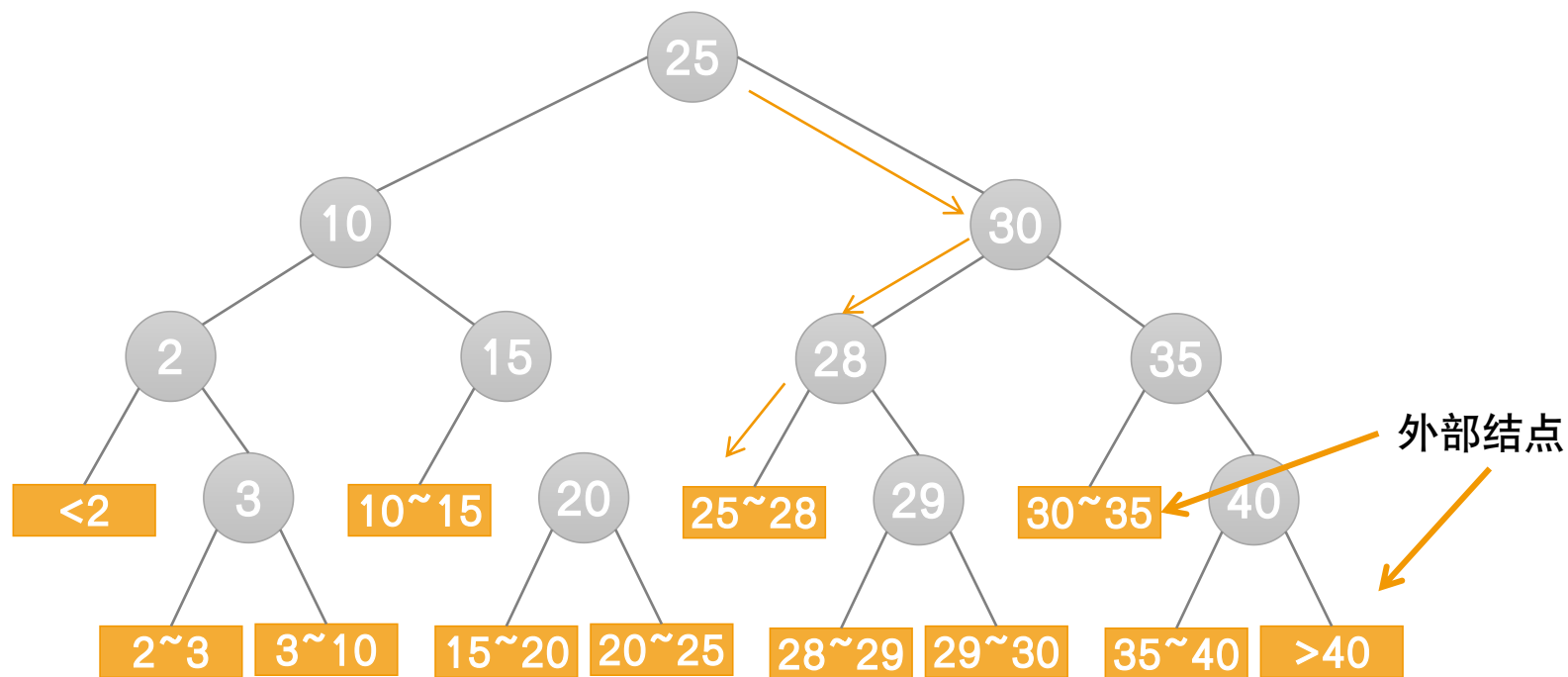
不成功二分查找



比较过程经历了一条从判定树根到某个外部结点的路径，所需的关键字比较次数是该路径上内部结点的总数，即**该外部结点的层次减1**。



带外部结点的判定树



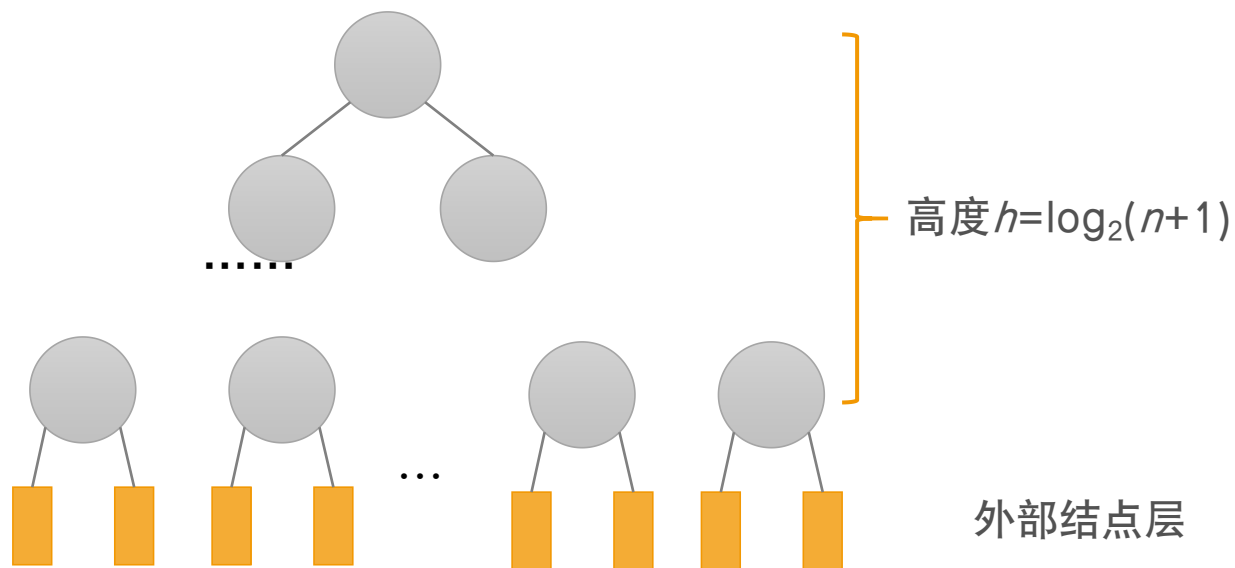
(3) 在查找不成功时，会找到某个外部结点，则不成功时的平均查找长度：

$$ASL_{\text{不成功}} = \frac{4 \times 3 + 8 \times 4}{12} = 3.67$$

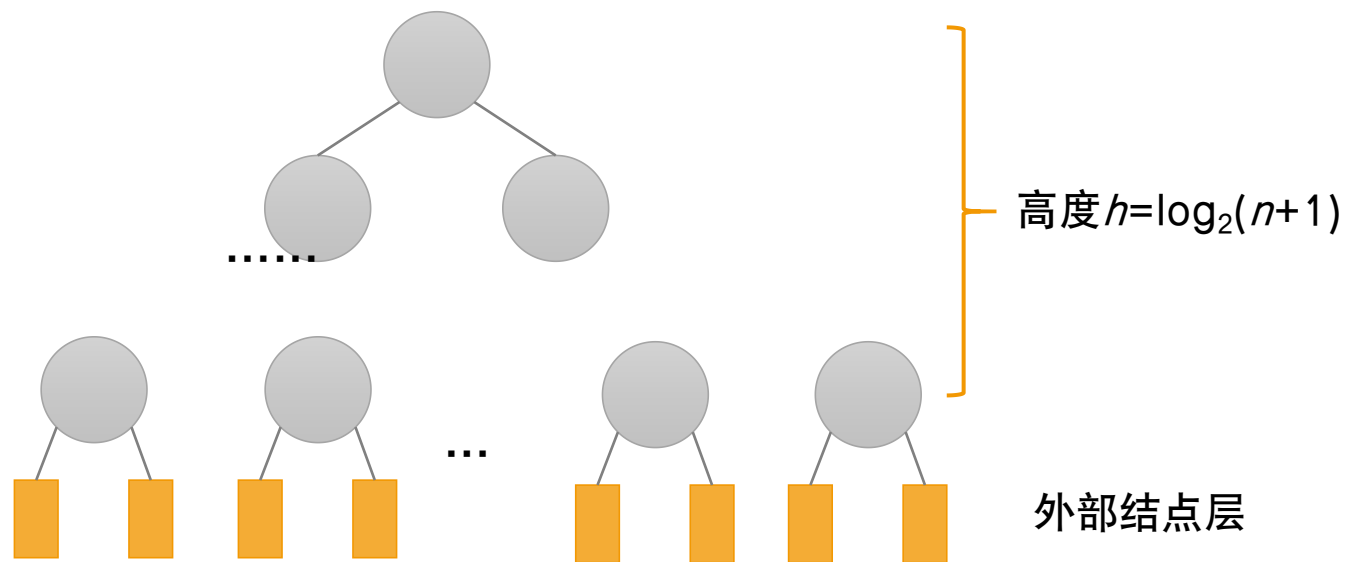
折半查找一般情况分析

当 n 比较大时，将判定树看成：

- 内部结点总数 $n=2^h-1$ 、高度为 $h=\log_2(n+1)$ 的**满二叉树**（高度 h 不计外部结点）。
- 树中第 j 层上的元素个数为 2^{j-1} ，查找该层上的每个元素需要进行 j 次比较。



折半查找一般情况分析



- 对于 n 个元素，二分查找成功时最多的关键字比较次数为： $\lceil \log_2(n+1) \rceil$
- 不成功时最多关键字比较次数为： $\lceil \log_2(n+1) \rceil$ 。

数据结构经典算法的启示

顺序查找算法



利用了数据的有序性

二分查找算法

示例

示例

设有100个元素的有序表，采用折半查找方法，成功时最大的比较次数是（ ）。

A.25 B.50 C.10 D.7

- 成功时最大比较次数为 $\lceil \log_2(n+1) \rceil = \lceil \log_2 101 \rceil = 7$ 。
- 答案： D。

示例

设有100个元素的有序表，采用折半查找方法，不成功时最大的比较次数是（ ）。

A.25 B.50 C.10 D.7

- 不成功时最大比较次数为 $\lceil \log_2(n+1) \rceil = \lceil \log_2 101 \rceil = 7$ 。
- 答案： D。

示例

在一个顺序存储的有序线性表上查找一个数据时，既可以采用折半查找，也可以采用顺序查找，但前者比后者的查找速度（）

- A. 必然快
- B. 取决于表是递增还是递减
- C. 在大部分情况下要快
- D. 必然不快

■ 答案：C。

示例

已知一个有序表为 (11, 22, 33, 44, 55, 66, 77, 88, 99) , 折半查找元素55需要比较几次?

A.1

B.2

C.3

D.4

答: A。

9.2.3 索引存储结构和分块查找

1、索引存储结构

- 索引存储结构 = 主数据表 + 索引表
- 索引表中的每一项称为索引项，索引项的一般形式是：（关键字，地址）
- 关键字唯一标识一个元素，地址作为指向该关键字对应元素的指针，也可以是相对地址。

示例

学号	姓名
1	张三
4	李四
3	王五
2	刘六

学生表



存储

主数据表

存储地址

0
1
2
3

学号	姓名
1	张三
4	李四
3	王五
2	刘六

学号	地址
1	0
4	1
3	2
2	3

索引表

学号	地址
1	0
2	3
3	2
4	1



提取



排序

主数据表

存储地址	学号	姓名
0	1	张三
1	4	李四
2	3	王五
3	2	刘六

索引表

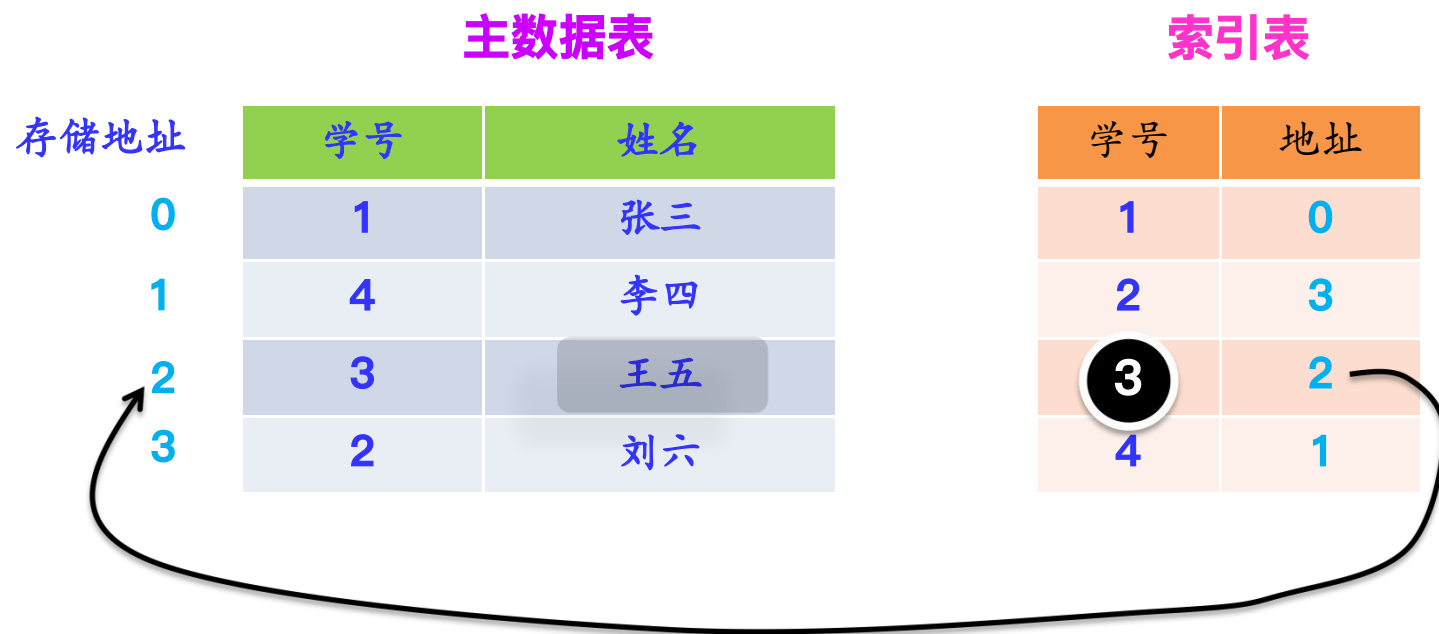
学号	地址
1	0
2	3
3	2
4	1

学生表的索引存储结构

查找过程:

- 在**索引表**（有序）中按关键字查找：采用二分查找。
- 在**主数据块**中查找：跳到主数据块中对应地址的记录。

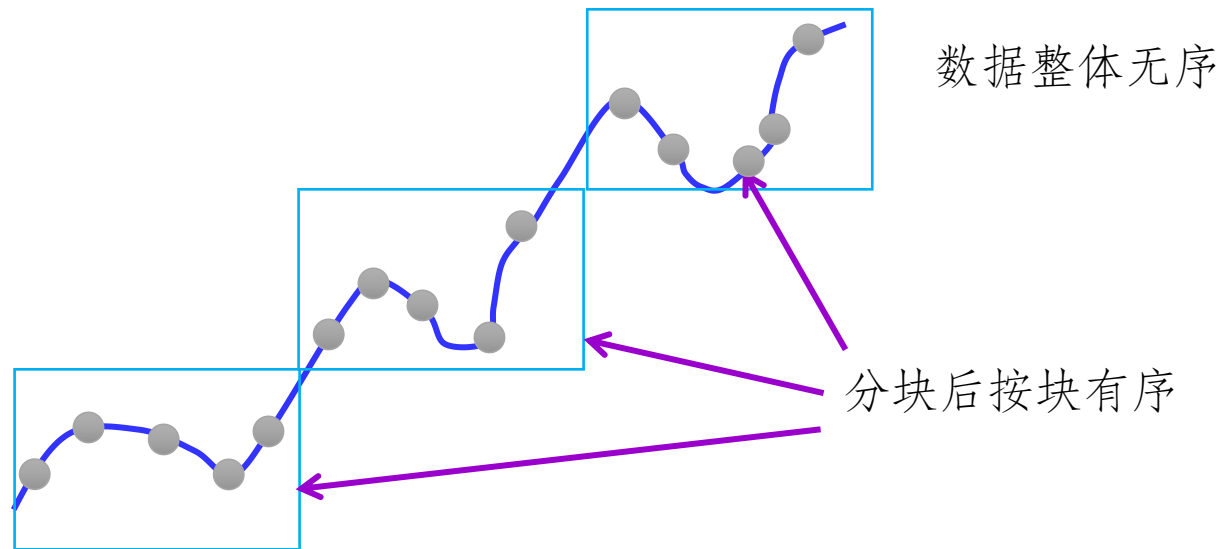
查找学号**3**的姓名:



时间复杂度为 $O(\log_2 n)$

2、分块查找

思路

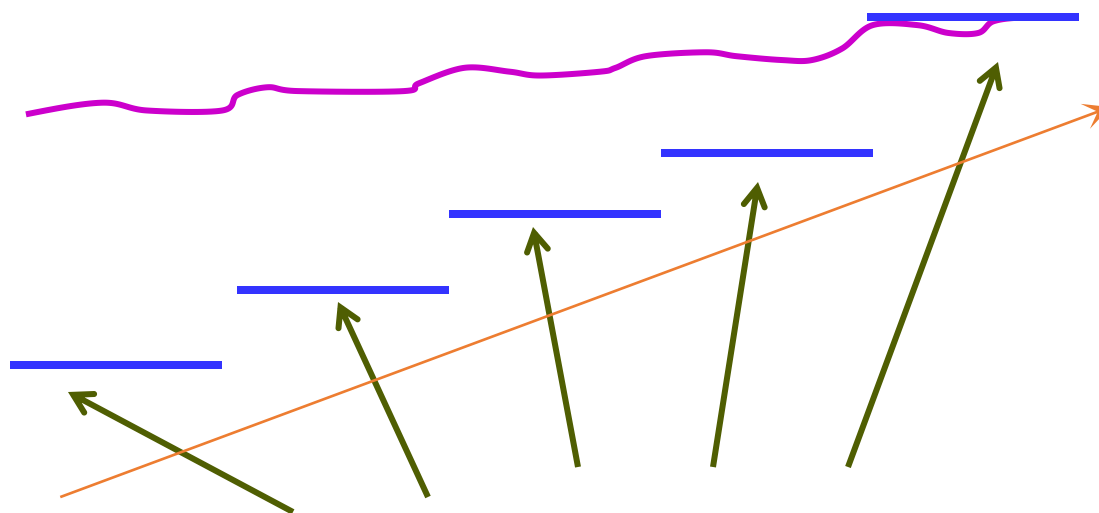


例如，设有一个线性表，其中包含25个记录，其关键字序列为：

8, 14, 6, 9, 10, 22, 34, 18, 19, 31, 40, 38, 54, 66, 46, 71,
78, 68, 80, 85, 100, 94, 88, 96, 87

分块： 将 $n=25$ 个记录分为 $b=5$ 块，每块中有 $s=5$ 个记录。

数据特性：



每块建立一个索引项 $\Rightarrow$ 索引表

分块查找k过程

- 在索引表（有序）中按关键字查找：可以顺序查找块，也可以二分查找块。
- 在对应的数据块（无序）中查找：只能顺序查找块中元素。

分块查找演示

查找关键字为80的记录

索引表

14	34	66	85	100	key
0	5	10	15	20	link

主数据表

8	1	6	9	1	2	3	1	1	3	4	3	5	6	4	7	7	6	8	8	1	9	8	9	8
0	4			0	2	4	8	9	1	0	8	4	6	6	1	8	8	0	5	0	4	8	6	7
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24

分块查找的索引存储结构

- (1) 顺序查索引表，比较4次
- (2) 在对应块中查找，比较4次，共比较8次。

算法的时间复杂度

分块查找含两次查找过程（所以分块查找最少需要2次关键字比较）



整个查找过程的平均查找长度，是两次查找的平均查找长度之和

示例

示例

如果在 n 个元素中查找其中任何一个元素至少要比2次，则所用的查找方法有可能是（ ）。

- A. 折半查找
- B. 分块查找
- C. 顺序查找
- D. 以上都不对

答： B

➤ 第三章

- 栈：先入后出

顺序栈 四要素：

栈空条件： $S \rightarrow TOP == -1$
栈满条件： $S \rightarrow TOP == \text{MaxSize} - 1$
进栈e操作：先 $Top++$ ；后存放
退栈操作：先取值，后 $Top--$

链栈

四要素：

栈空条件： $S \rightarrow \text{Next} == \text{NULL}$
栈满条件（不考虑）
进栈e操作：放在头结点之后
退栈操作：取出首结点的值

- 共享栈
- 简单表达式求值：中缀表达式转后缀表达式；后缀表达式求值

➤ 第三章

• 队列：先入先出

顺序队

四要素：
队空条件：front==rear
队满条件：rear ==MaxSize - 1
进队e操作：rear++; data[rear]=e
元素e出队：front++; e=data[front]

• 环形队列：（防止假溢出）

rear++: rear=(rear+1)%MaxSize
front++: front=(front+1)%MaxSize

队满条件：(rear+1)%MaxSize == front

链队

四要素：
队空条件：q->front==NULL或者q->rear==NULL
队满条件：不考虑
进队e操作：将包含e的结点插入到单链表表尾
退队操作：删除单链表首数据结点

1.若一个栈采用数组 s 存放其元素，MaxSize为 n ，且初始时栈顶指针为 n ，则以下元素 x 进栈的 正确操作是 () C

- A. $top++;s[top]=x;$ B. $s[top]=x;top++;$
C. $top--;s[top]=x;$ D. $s[top]=x;top--;$

2.设有一个栈，元素依次进栈的顺序为 1、2、3、4、5。下列 () 是不可能的出栈序列。 C

- A. 1,2,3,4,5 B. 5,4,3,2,1 C. 3,1,2,4,5 D. 4,3,5,2,1

3. 表达式 $a * (b + c) - d$ 的后缀表达式为: $abc + * d -$

4. 后缀表达式 $20 \ 15 \ 2 \ + \ - \ 27 \ 9 \ / \ *$ 运算后的值为_____。 9



数据结构

教师: 刘芳岑

邮箱: liufc@hainanu.edu.cn

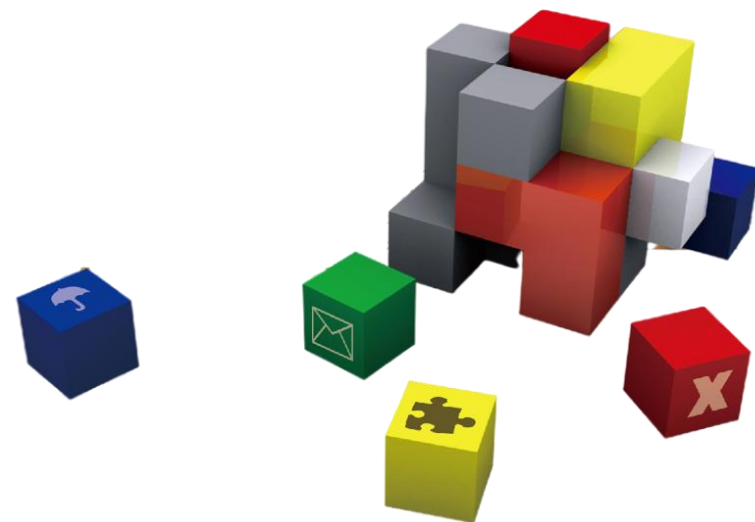
学院: 计算机科学与技术学院

时间: 2026春季学期

9.3 树表的查找



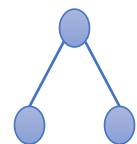
- 以二叉树或其他树作为查找表的组织形式，称为**树表**。
- 树表主要采用**链式存储结构**。
- 这类存储结构不仅适合于查找，也适合于插入和删除操作。属于**动态查找表**。



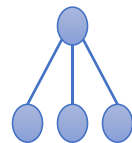
常见的树表：

- 二叉排序树
- 平衡二叉树
- B树
- B+树

二叉树



m 叉树



9.3.1 二叉排序树

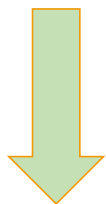
二叉排序树（简称BST）又称二叉查找（搜索）树，其定义为：
二叉排序树或者是空树，或者是满足如下性质（BST性质）的二叉树：

- ❶ 若它的左子树非空，则左子树上所有结点值（指关键字值）均小于根结点值。
- ❷ 若它的右子树非空，则右子树上所有结点值均大于根结点值。
- ❸ 左、右子树本身又各是一棵二叉排序树。

注意：二叉排序树中没有相同关键字的结点。

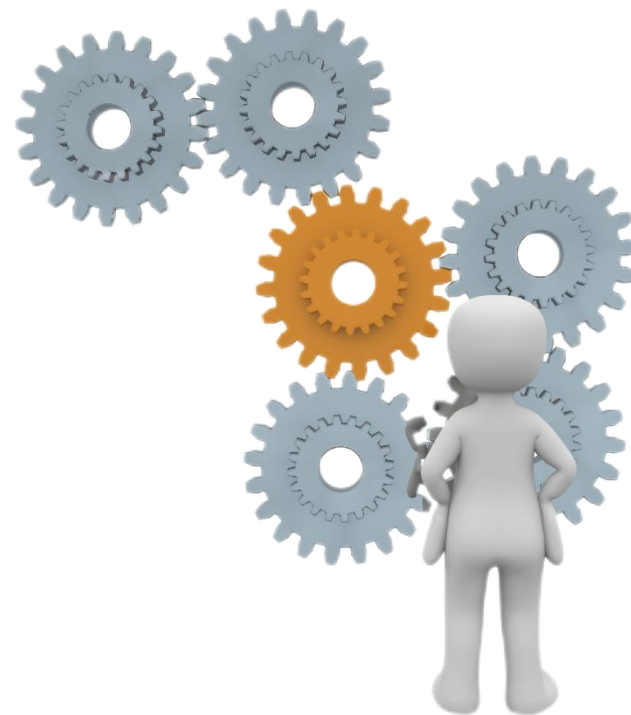
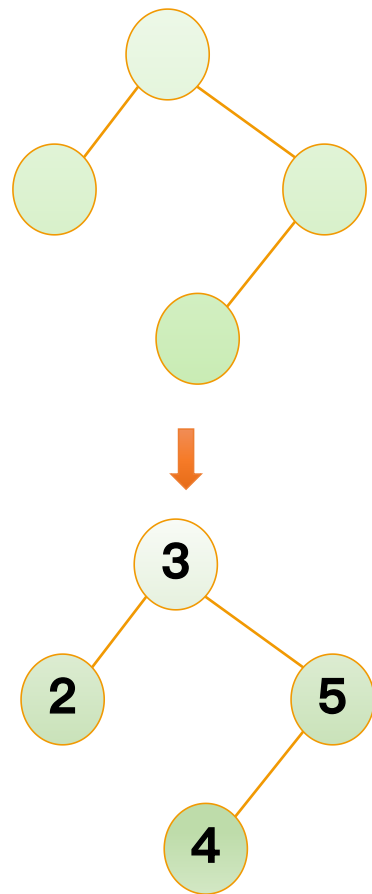


二叉树结构

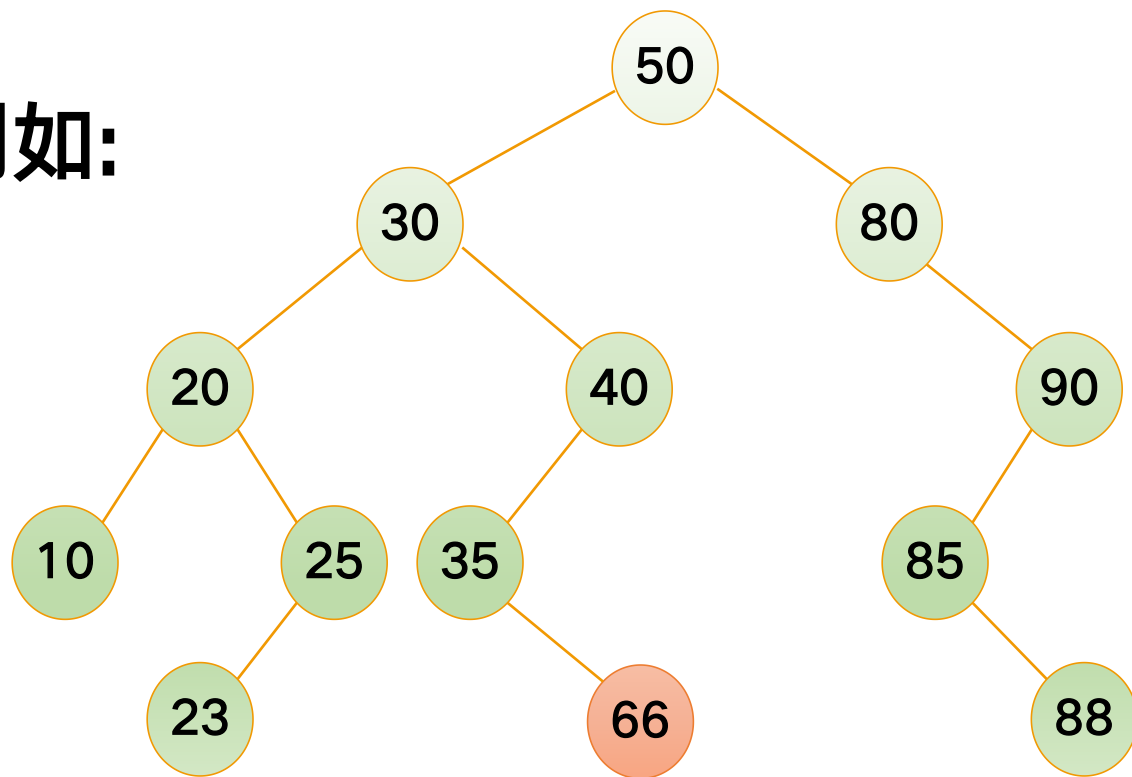


满足BST性质：
结点值约束

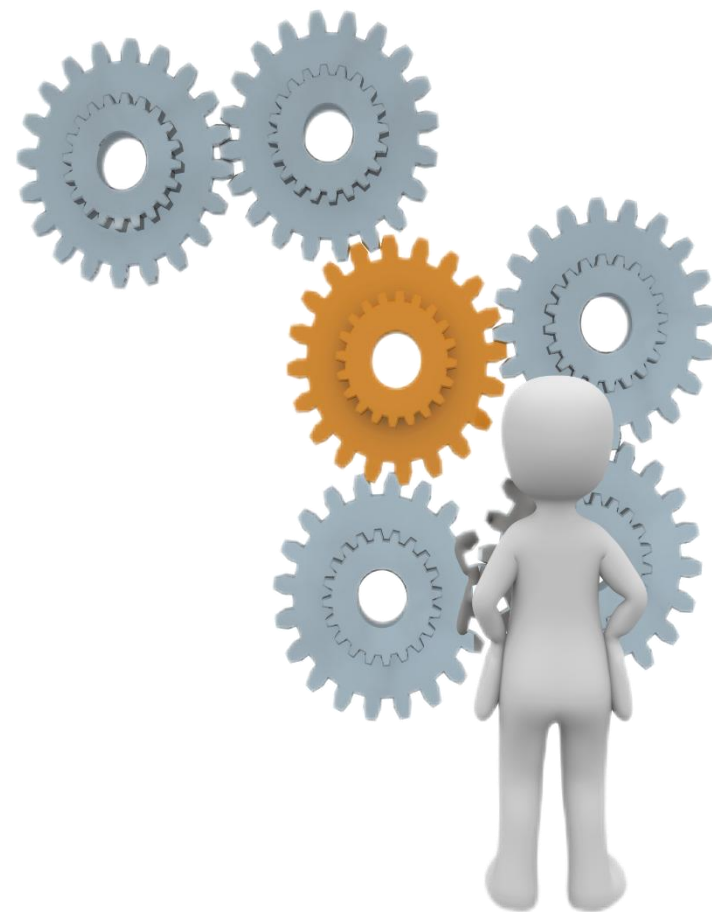
二叉排序树



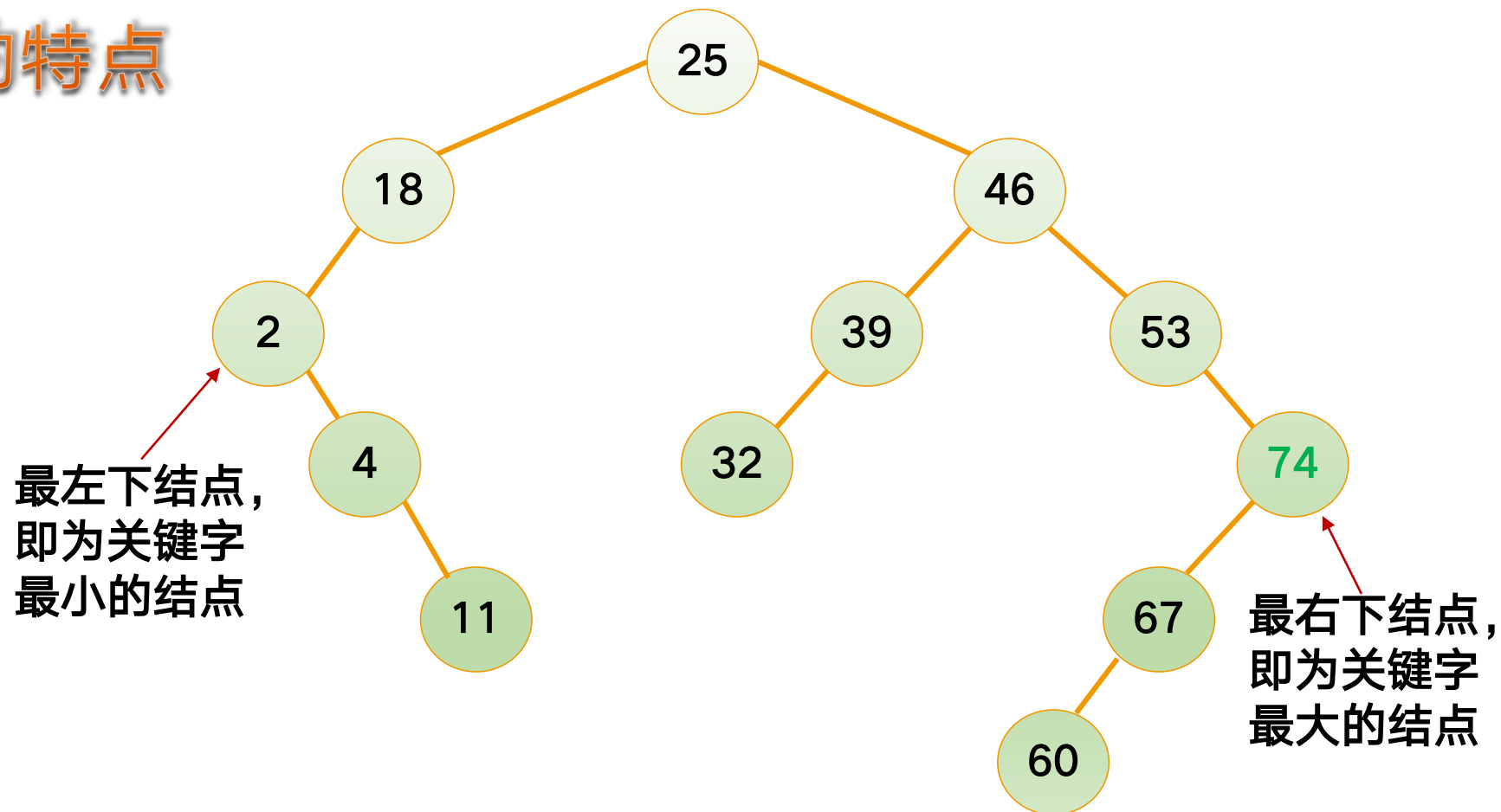
例如:



不是二叉排序树。



二叉排序树的特点



中序序列: 2 4 11 18 25 32 39 46 53 60 67 74

特点:

- 二叉排序树的中序序列是一个递增有序序列
- 根结点的最左下结点是关键字最小的结点
- 根结点的最右下结点是关键字最大的结点



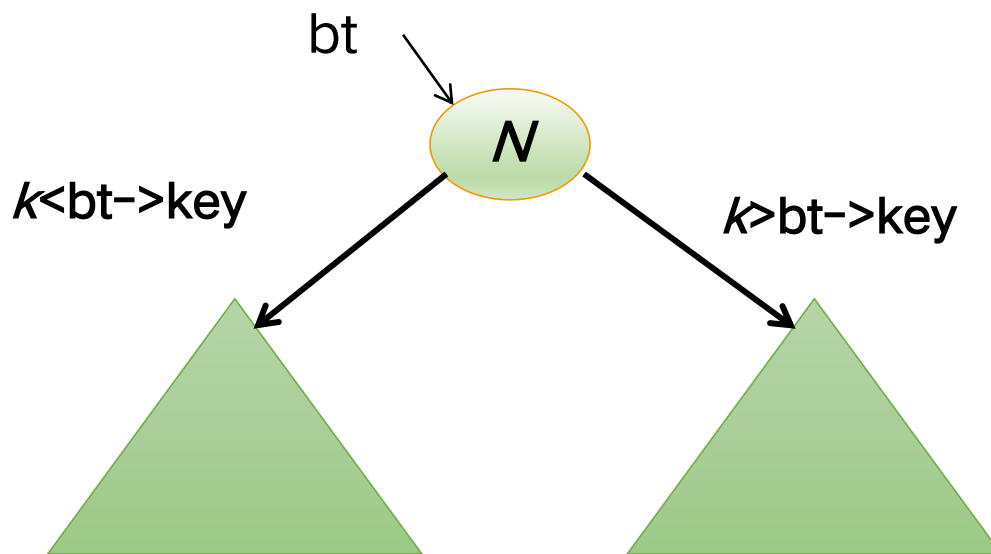
二叉排序树的结点类型如下：

```
typedef struct node
{
    KeyType key;           //关键字项
    InfoType data;        //其他数据域
    struct node *lchild, *rchild; //左右孩子指针
} BSTNode;
```



1、二叉排序树的查找

- 二叉排序树可看做是一个有序表（中序序列是递增有序序列）。
- 在二叉排序树上进行查找，和二分查找类似，也是一个逐步缩小查找范围的过程。

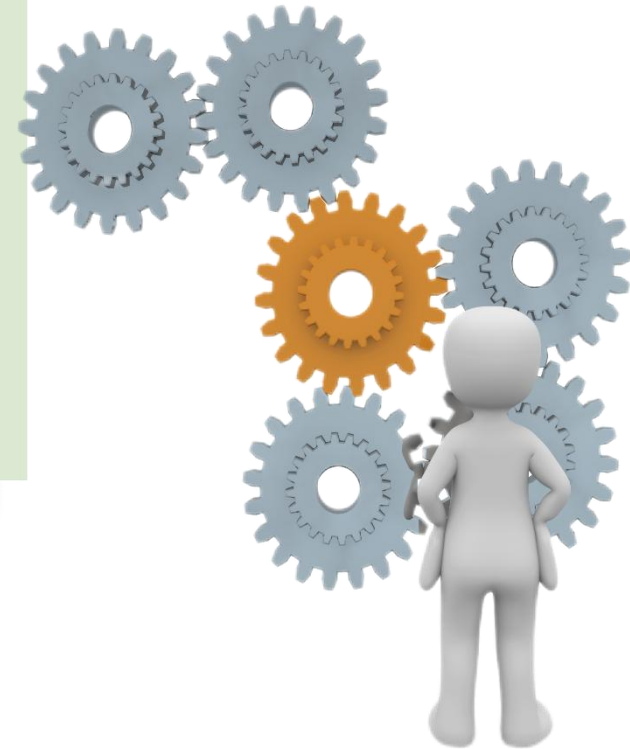


每一层只和一个结点进行关键字比较！

- 查找算法**SearchBST**(bt,k): 在二叉排序树bt上查找关键字为k的结点, 成功时返回该结点地址, 否则返回NULL。

```
BSTNode *SearchBST(BSTNode *bt, KeyType k)
{  if (bt==NULL || bt->key==k)           //递归出口
    return bt;
    if (k<bt->key)
        return SearchBST(bt->lchild, k); //在左子树中递归查找
    else
        return SearchBST(bt->rchild, k); //在右子树中递归查找
}
```

↑
递归算法

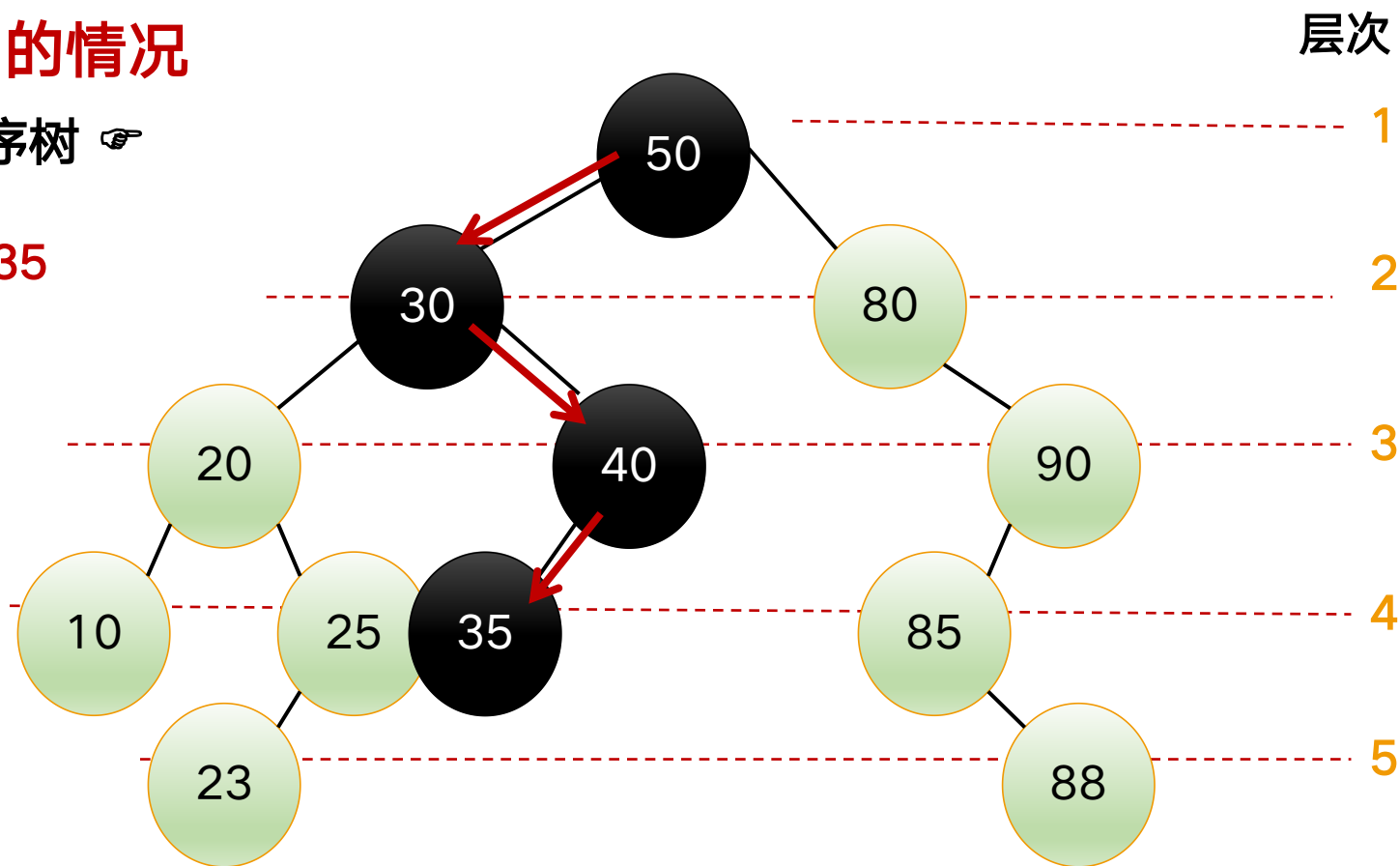


二叉排序树查找性能分析

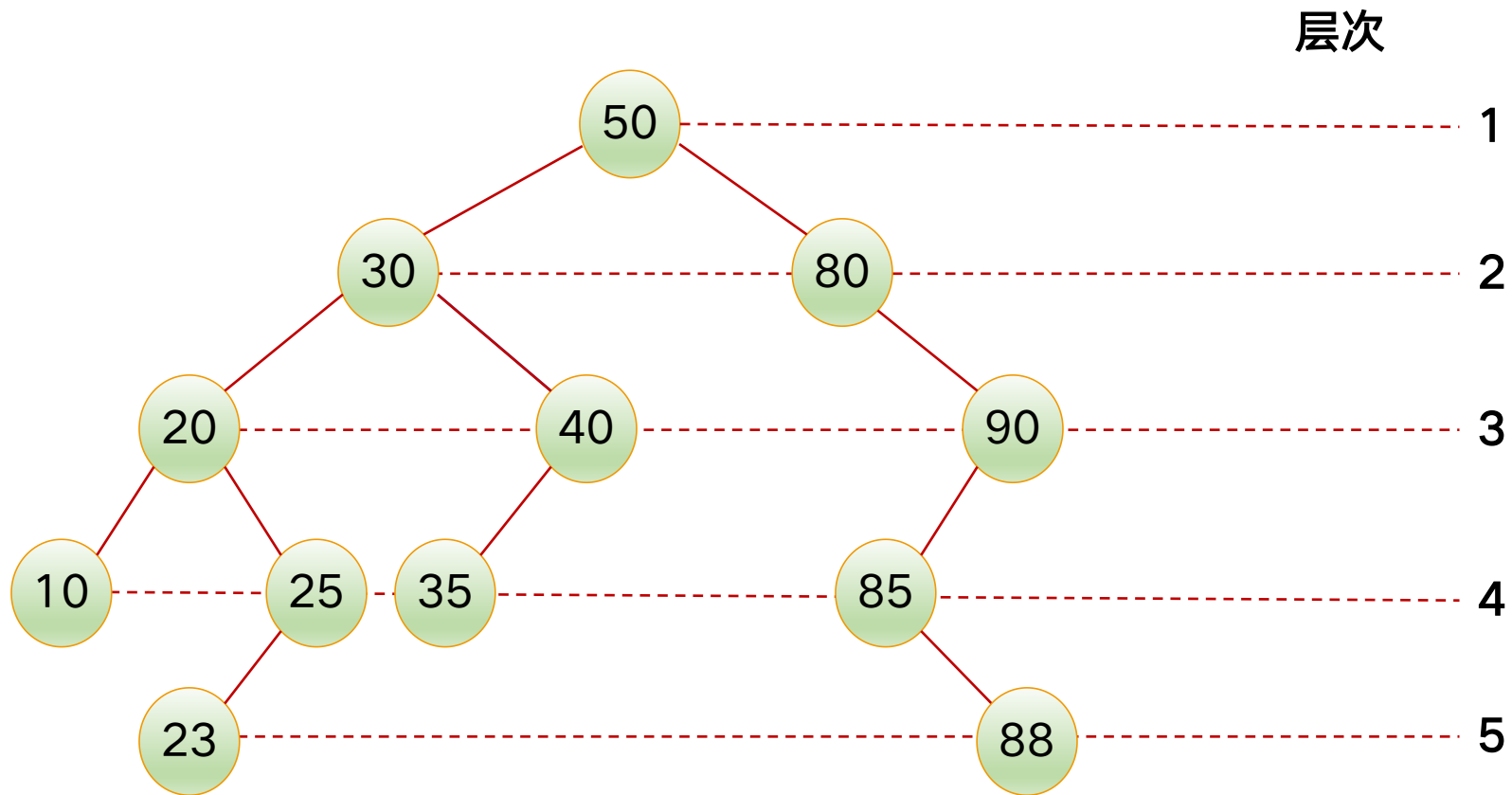
❶ 查找成功的情况

一棵二叉排序树 📌

查找: 35

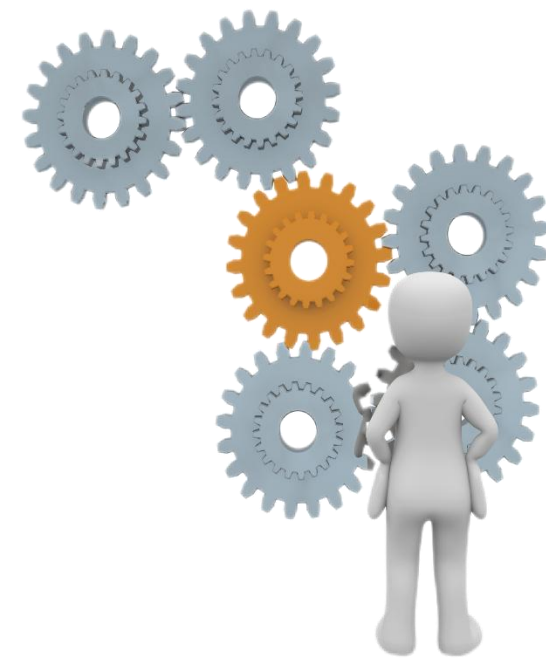


- 从根结点开始，查找最后落在某个**内部结点**中
- 比较次数为该内部结点的层次
- 查找路径是二叉排序树的一部分，本身也是一棵二叉排序树

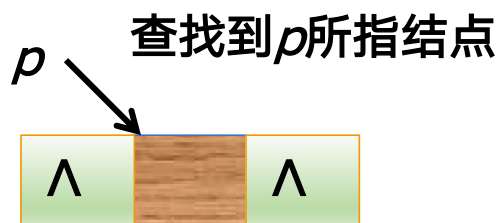


在等概率假设下，查找成功的平均查找长度

$$ASL_{\text{成功}} = \frac{1 \times 1 + 2 \times 2 + 3 \times 3 + 4 \times 4 + 2 \times 5}{12} = 3.33$$

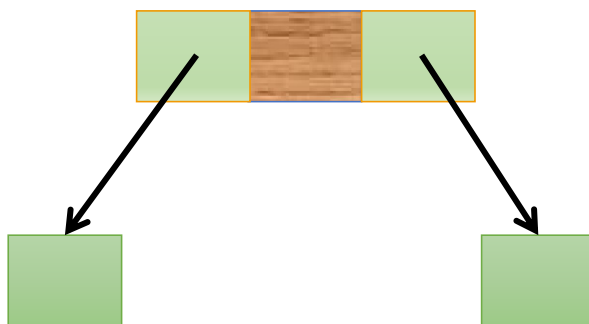


② 查找失败的情况

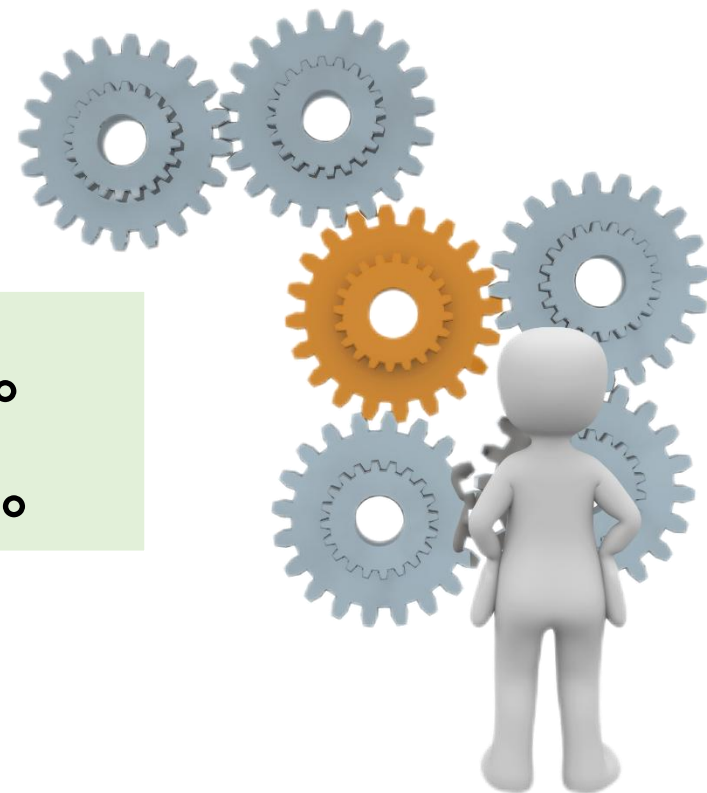


- 若 $k < p \rightarrow \text{key}$, 并且 $p \rightarrow \text{lchild} = \text{NULL}$, 查找失败。
- 若 $k > p \rightarrow \text{key}$, 并且 $p \rightarrow \text{rchild} = \text{NULL}$, 查找失败。

加上外部结点



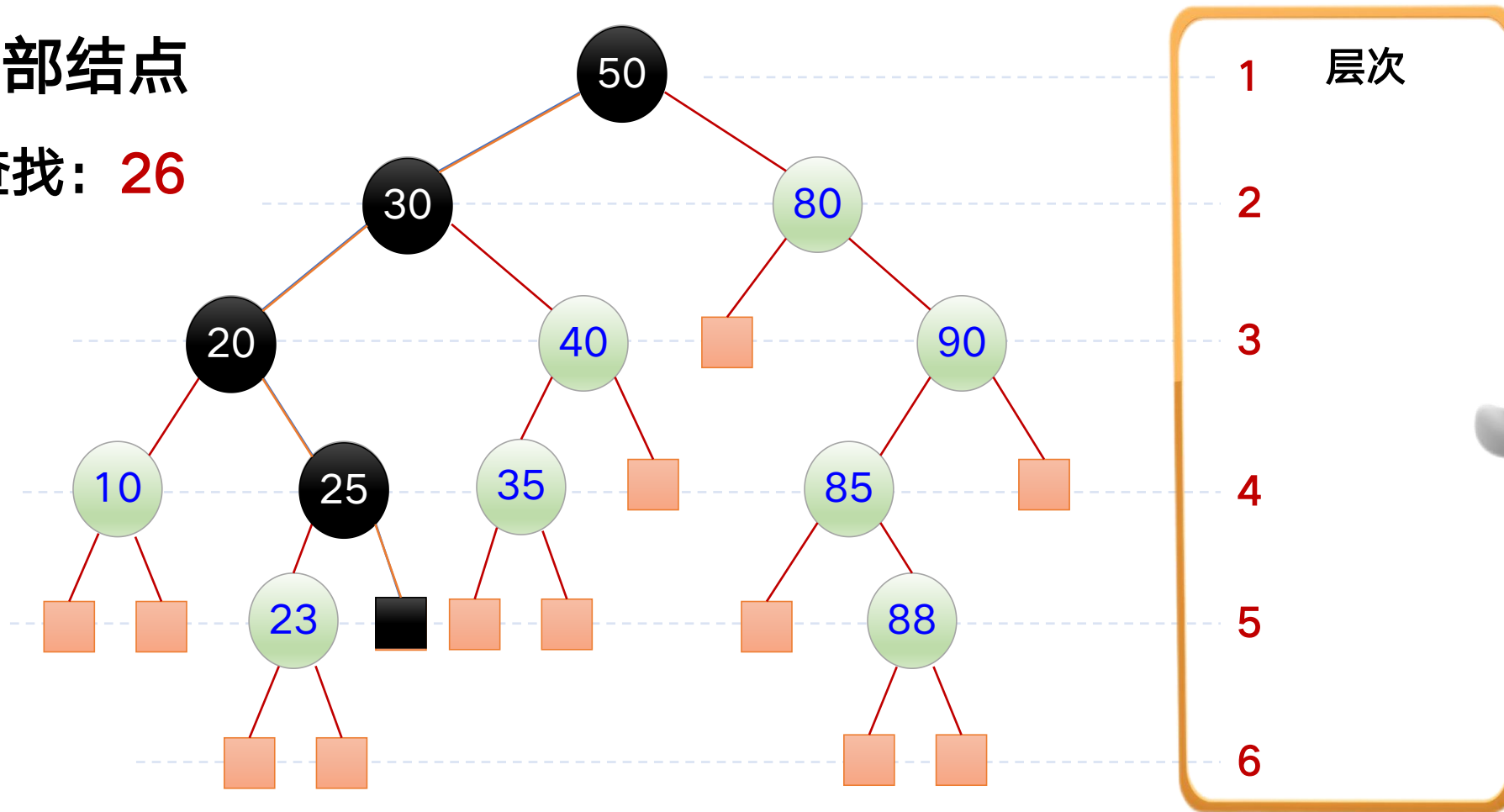
一个外部结点对应某内部结点的一个NULL指针



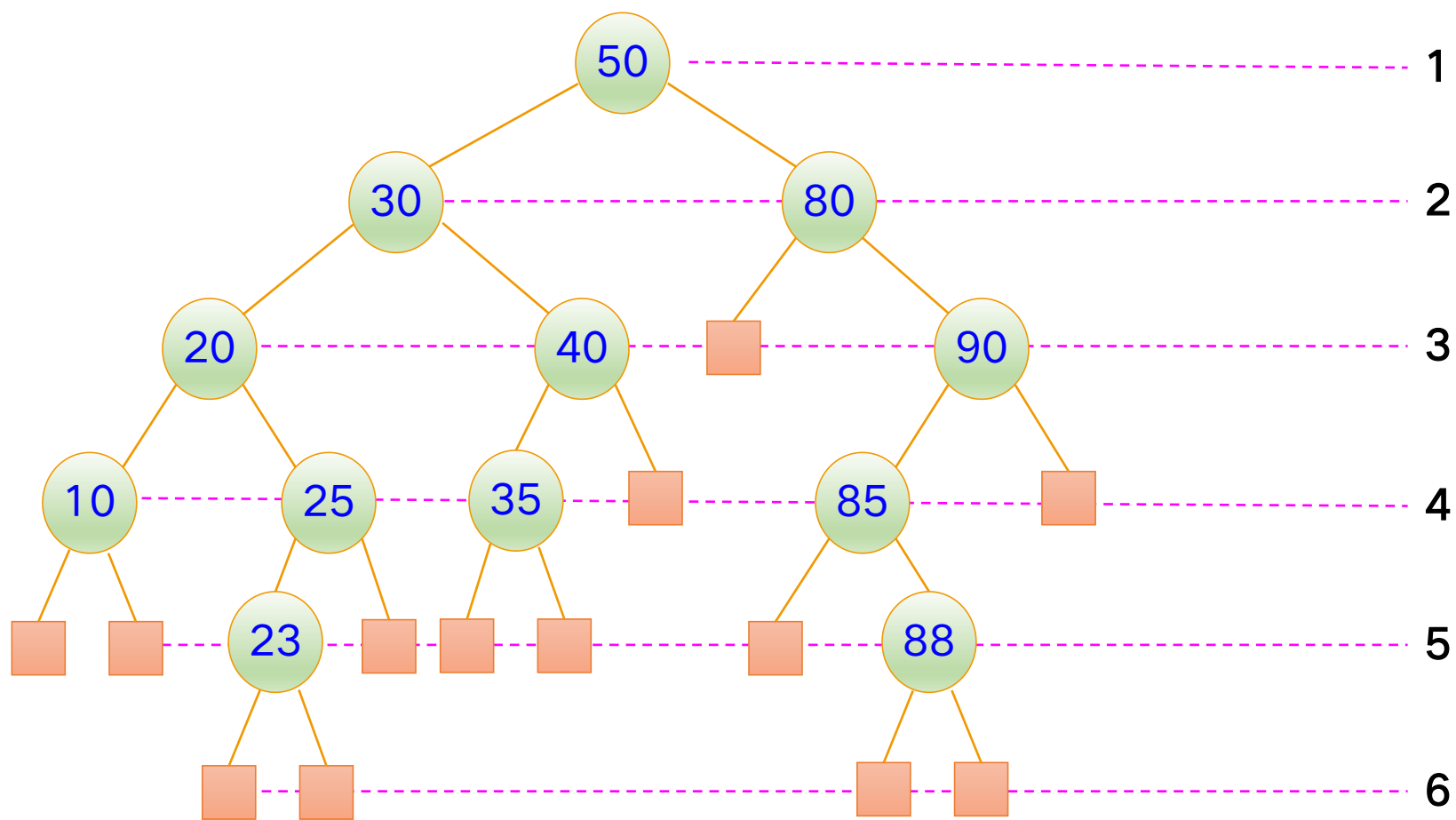
② 查找失败的情况

加上外部结点

查找: 26



- 从根结点开始，查找最后落在某个外部结点中
- 比较次数为该外部结点的层次减1



在等概率假设下，查找不成功的平均查找长度

$$ASL_{\text{不成功}} = \frac{1 \times 2 + 2 \times 3 + 6 \times 4 + 4 \times 5}{13} = 4$$

示例

在含有27个结点的二叉排序树上，查找关键字为35的结点，以下（ ）是可能的关键字比较序列？

A.28,36,18,46,35

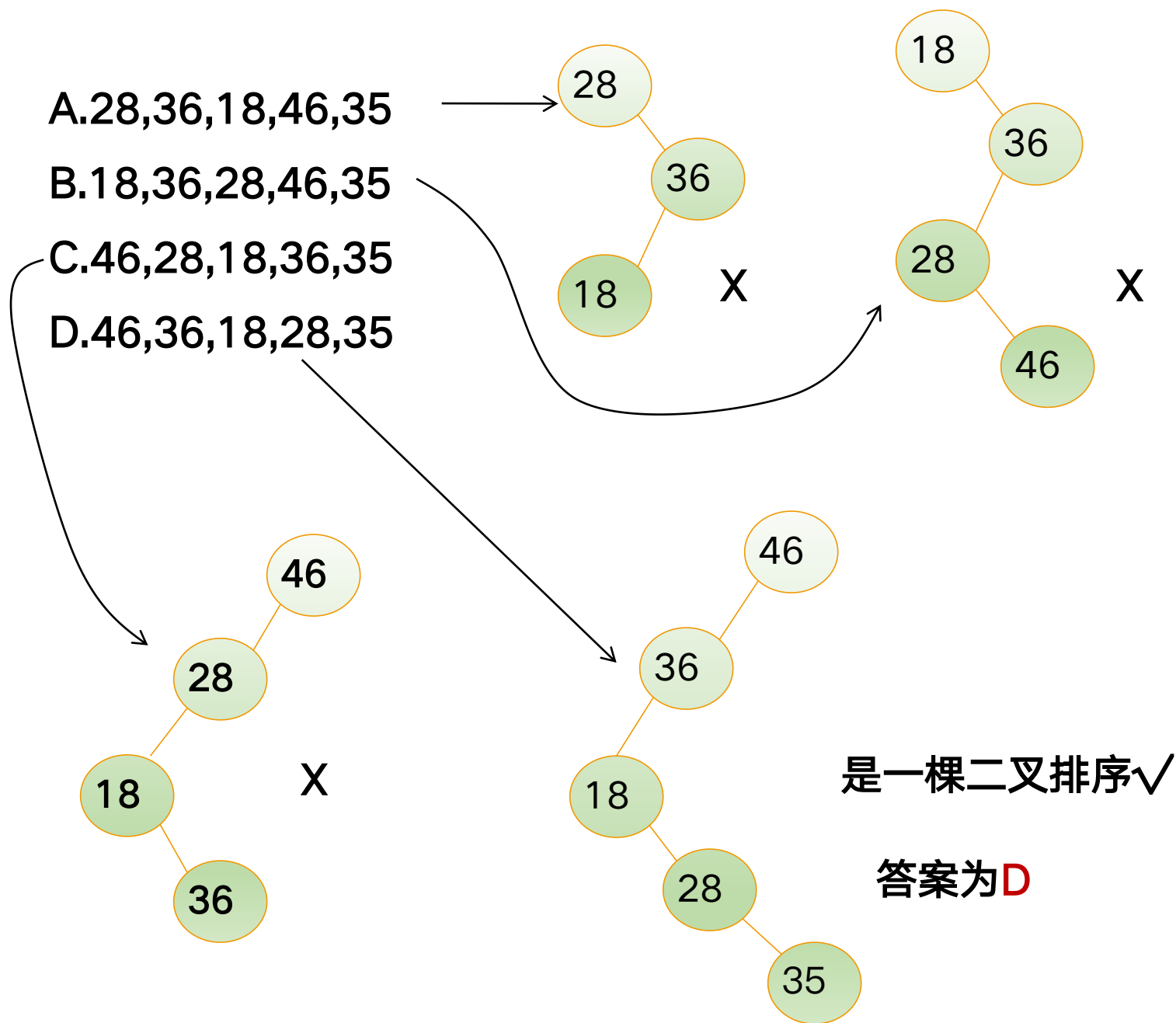
B.18,36,28,46,35

C.46,28,18,36,35

D.46,36,18,28,35

判断标准：在二叉排序树中的查找路径是原来二叉排序树的一部分，也一定构成一棵二叉排序树。





2、二叉排序树的插入和生成

在二叉排序树中插入一个关键字为 k 的新结点，要保证插入后仍满足BST性质。

插入过程：

- (1) 若二叉排序树 T 为空，则创建一个key域为 k 的结点，将它作为根结点；
- (2) 否则将 k 和根结点的关键字比较，若两者相等，则说明树中已有此关键字 k ，无须插入，直接返回0；
- (3) 若 $k < T \rightarrow \text{key}$ ，则将 k 插入根结点的左子树中。
- (4) 否则将它插入右子树中。

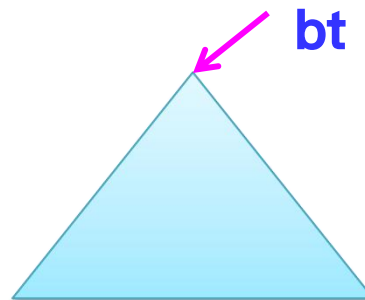
对应的递归算法InsertBST()如下:

```
int InsertBST(BSTNode *&p, KeyType k)
{
    if (p==NULL)          //原树为空, 新插入的记录为根结点
    { p=(BSTNode *)malloc(sizeof(BSTNode));
      p->key=k;p->lchild=p->rchild=NULL;
      return 1;
    }
    else if (k==p->key)    //存在相同关键字的结点, 返回0
      return 0;
    else if (k<p->key)
      return InsertBST(p->lchild, k); //插入到左子树中
    else
      return InsertBST(p->rchild, k); //插入到右子树中
}
```



先序遍历的思想

关键字数组
 $A[0..n-1]$



```
BSTNode *CreatBST(KeyType A[], int n) //返回树根指针
{ BSTNode *bt=NULL;           //初始时bt为空树
  int i=0;
  while (i<n)
  { InsertBST(bt, A[i]); //将A[i]插入二叉排序树T中
    i++;
  }
  return bt;           //返回建立的二叉排序树的根指针
}
```

注意：任何结点插入到二叉排序树时，都是以叶结点插入的。

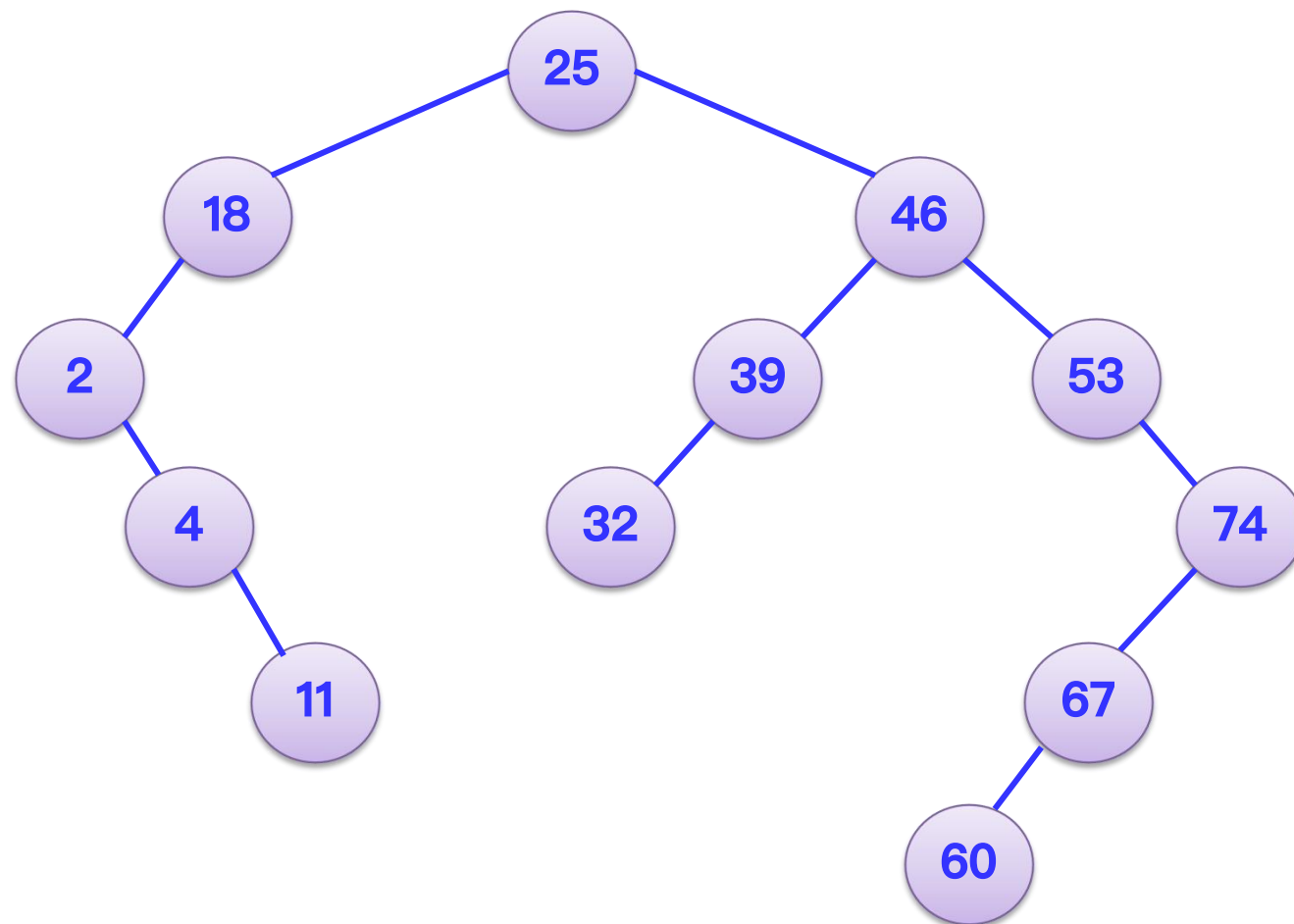
【例9-3】 已知一组关键字为：

{25, 18, 46, 2, 53, 39, 32, 4, 74, 67, 60, 11}

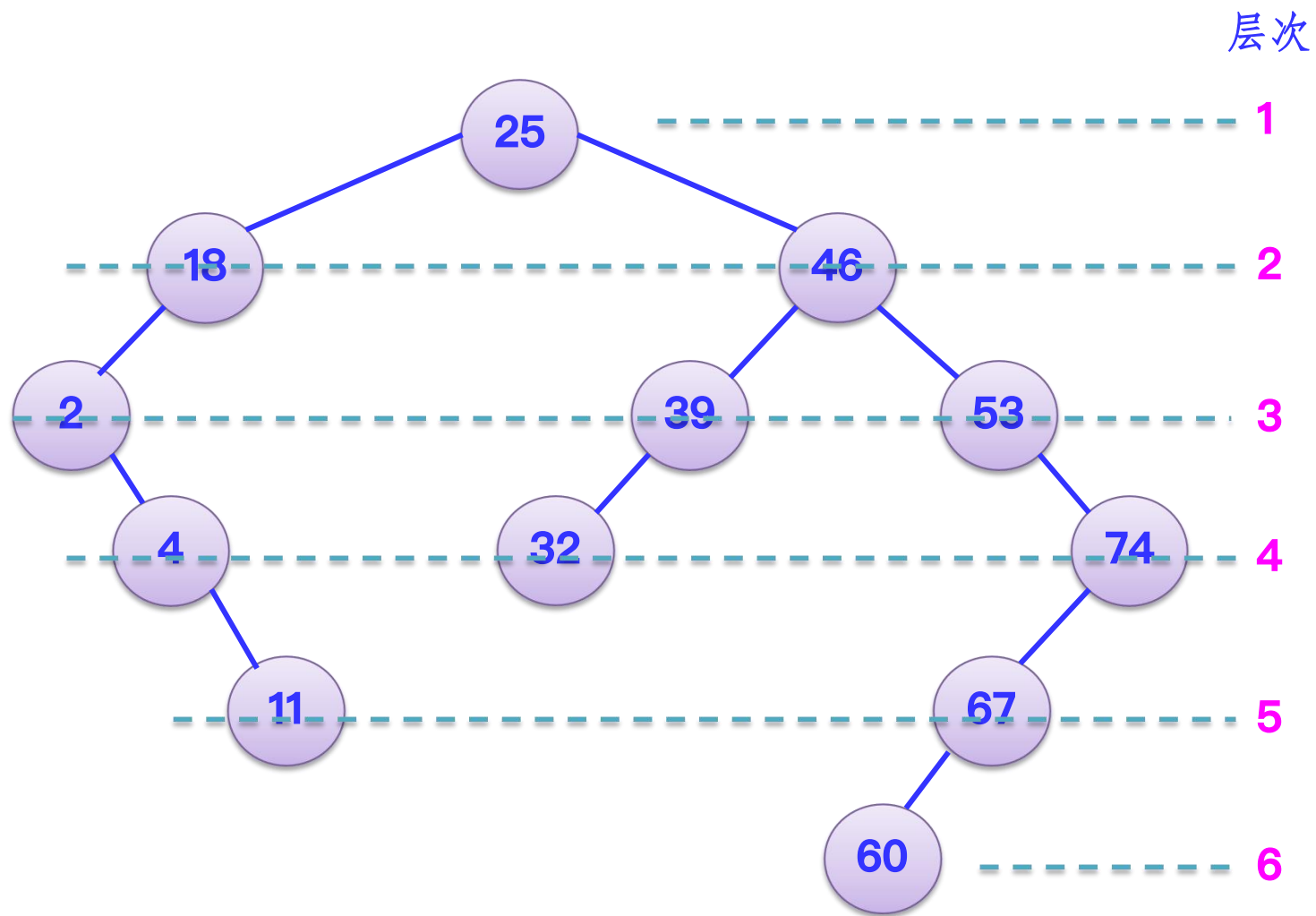
按表中的元素顺序依次插入到一棵初始为空的二叉排序树中，画出该二叉排序树。

求在等概率的情况下查找成功的平均查找长度和查找不成功的平均查找长度。

序列: 25 18 46 2 53 39 32 4 74 67 60 11



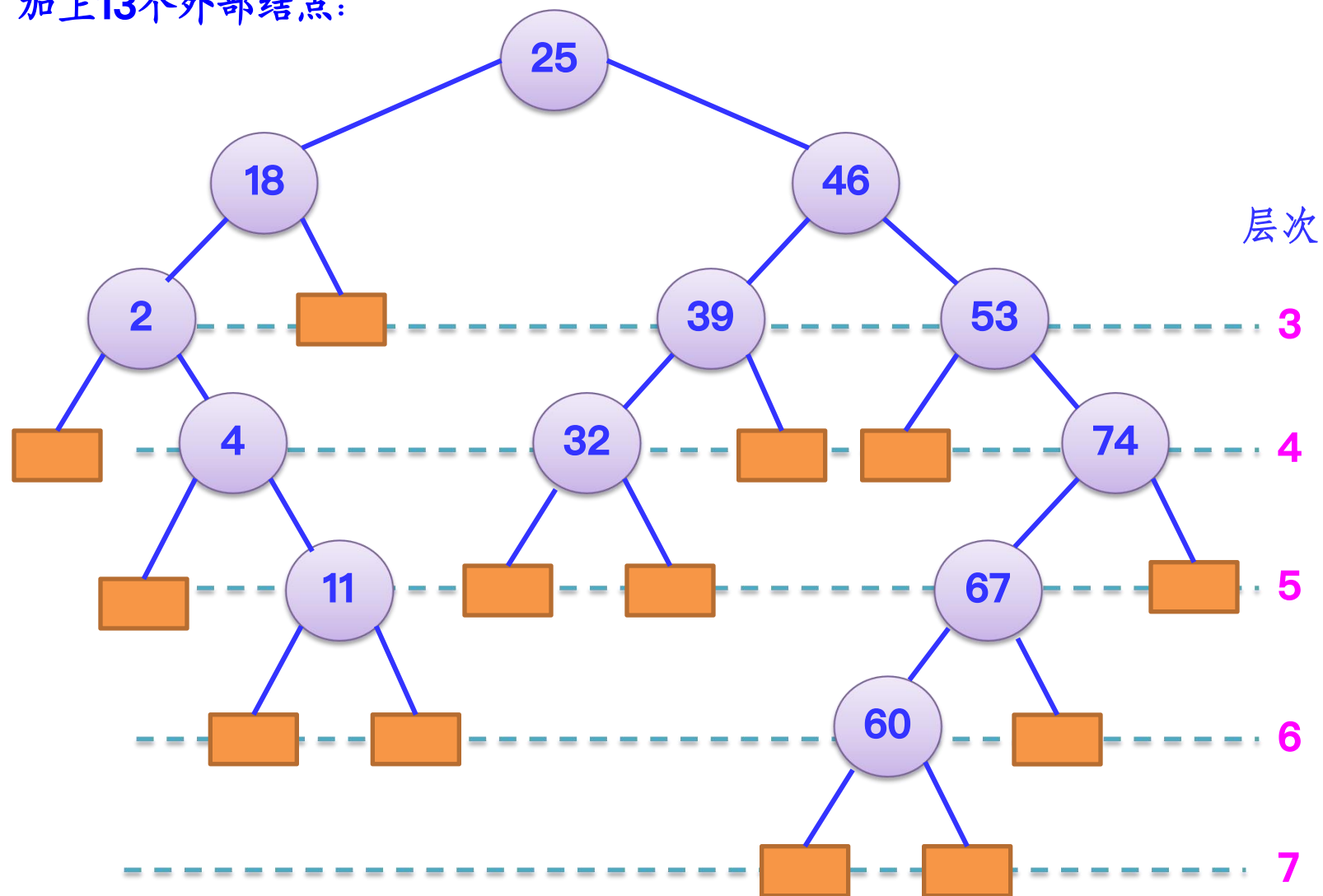
二叉排序树创建完毕



成功找到第*i*
层的结点，需
要*i*次比较

$$ASL_{\text{成功}} = \frac{1 \times 1 + 2 \times 2 + 3 \times 3 + 3 \times 4 + 2 \times 5 + 1 \times 6}{12} = 3.5$$

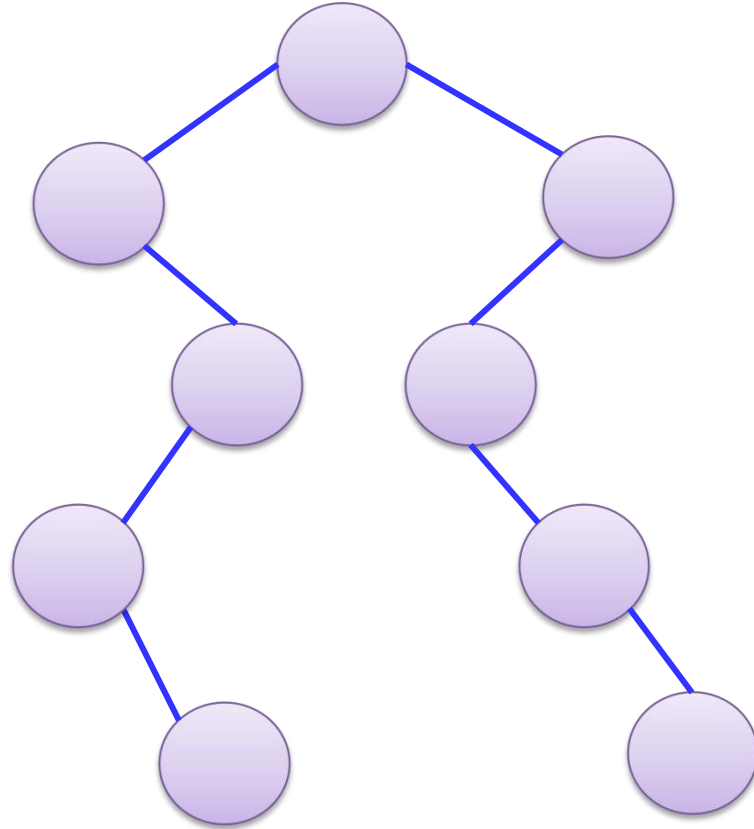
加上13个外部结点:



落在第
 i 层的
外部结
点, 需
要 $i-1$
次比较

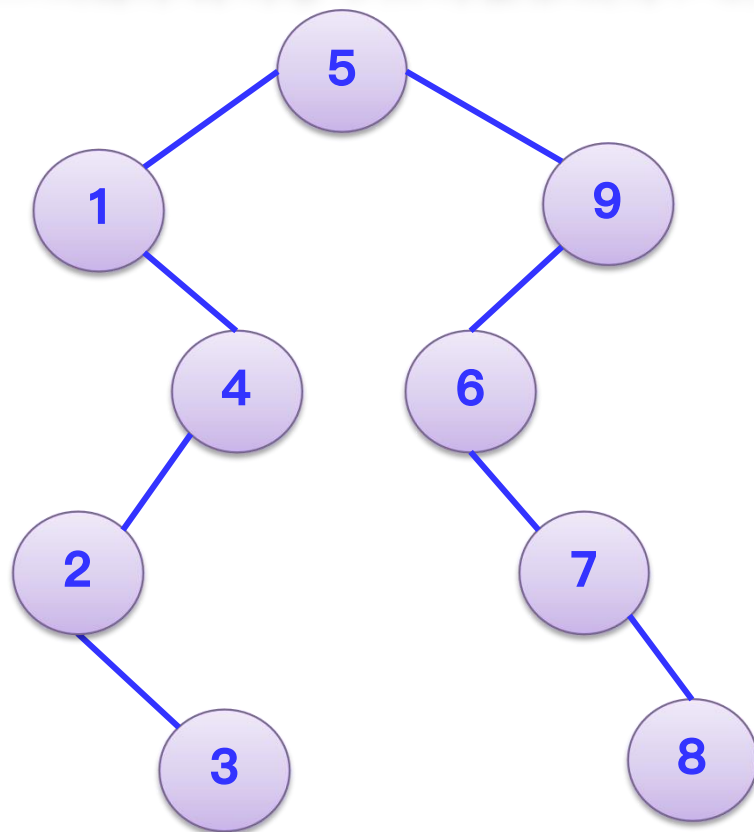
$$ASL_{\text{不成功}} = \frac{1 \times 2 + 3 \times 3 + 4 \times 4 + 3 \times 5 + 2 \times 6}{13} = 4.15$$

已知下面二叉排序树的各节点的值依次为1-9，请标出各节点的值。





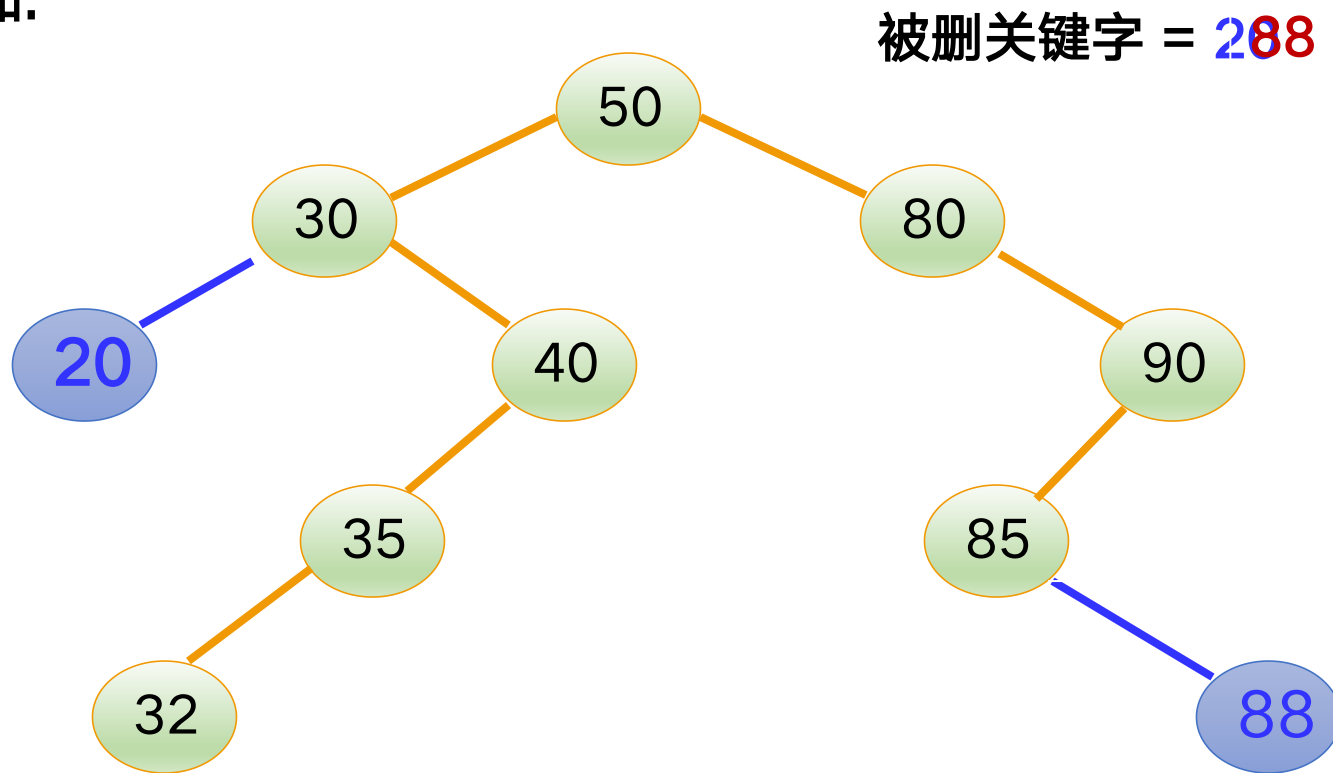
已知下面二叉排序树的各节点的值依次为1-9，请标出各节点的值。



3、二叉排序树的结点删除

(1) 被删除的结点是叶子结点：直接删去该结点。

例如：



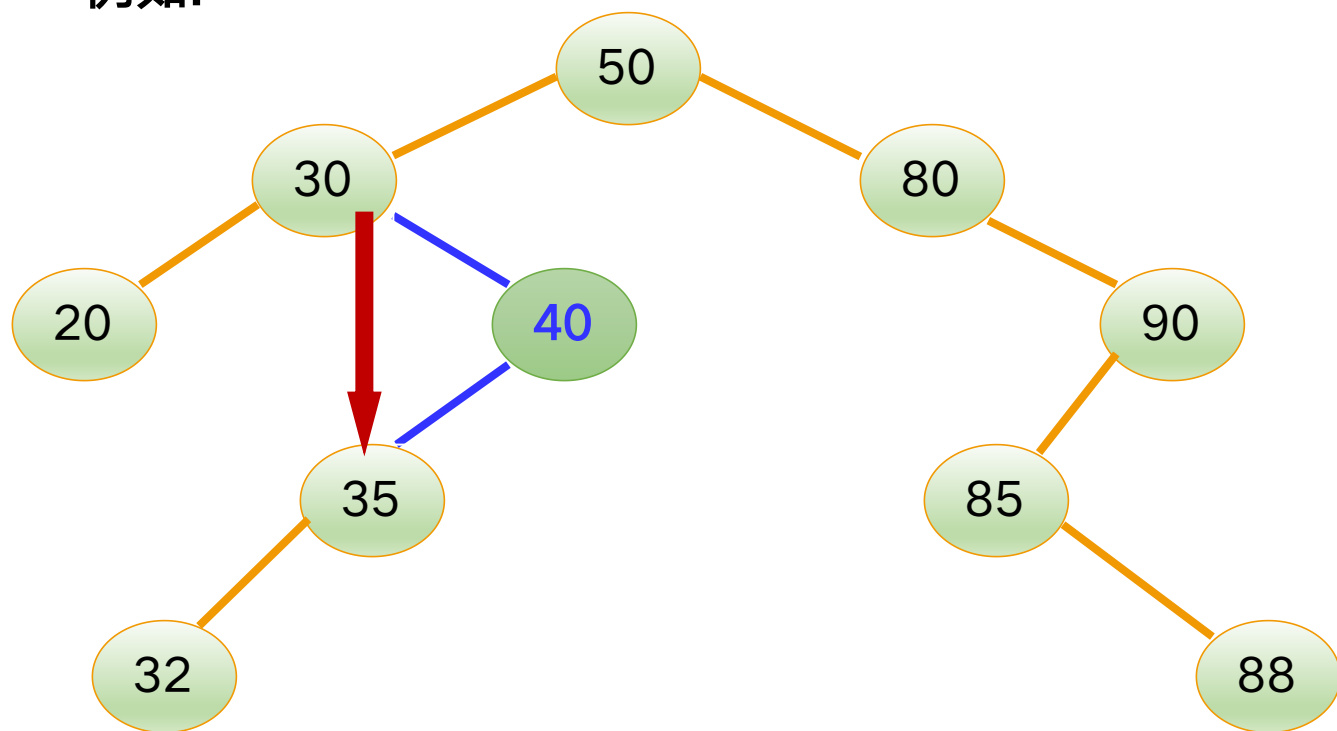
其双亲结点中相应指针域的值改为“空”



(2) 被删除的结点只有左子树，用其左孩子替换它（结点替换）。

被删关键字=40

例如：



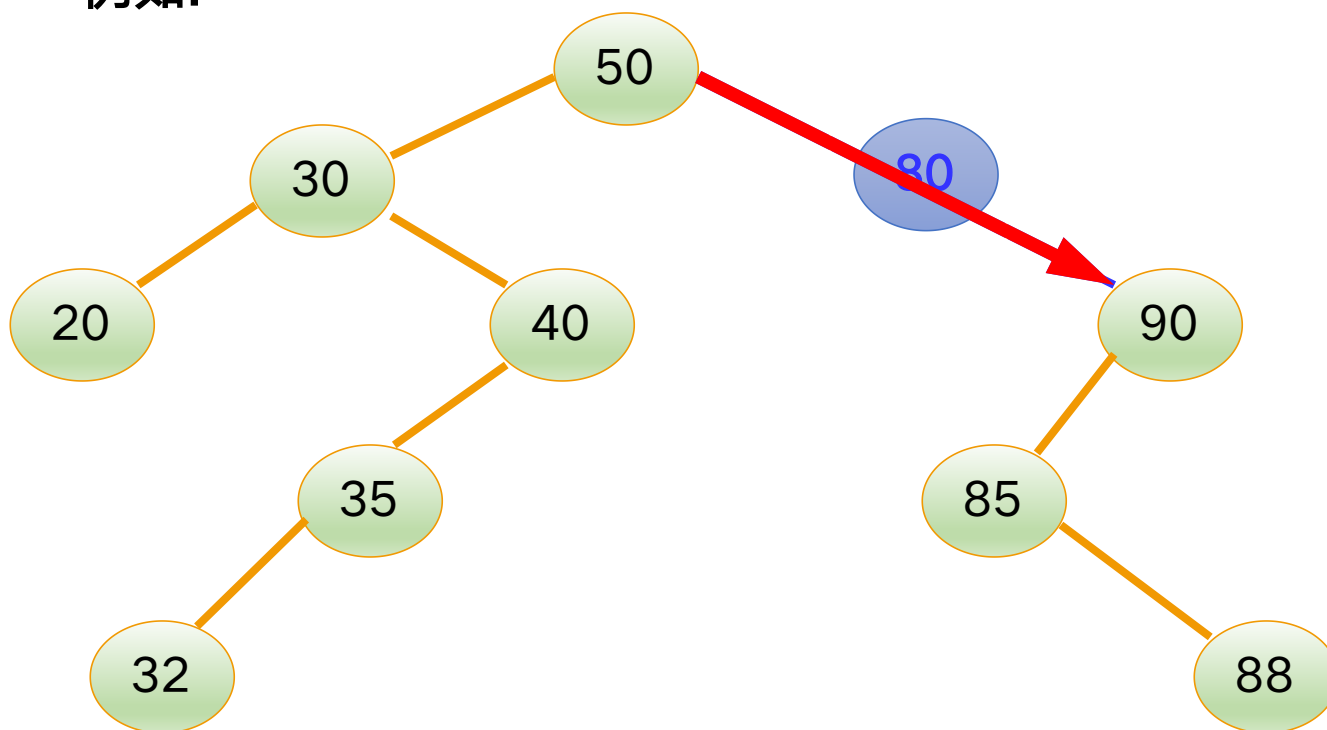
其双亲结点的相应指针域的值改为“指向被删除结点的左孩子”。



(3) 被删除的结点只有右子树，用其右孩子替换它（结点替换）。

例如：

被删关键字=80



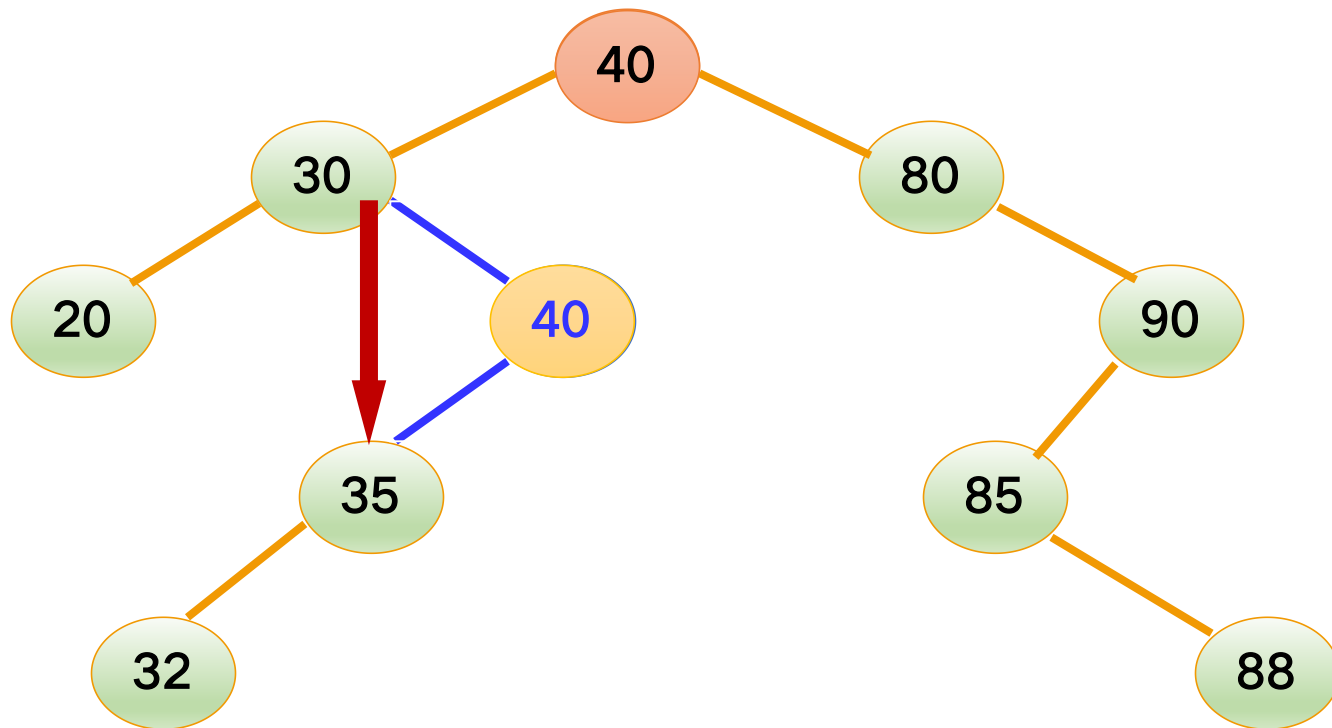
其双亲结点的相应指针域的值改为“指向被删除结点的右孩子”。



(4) 被删除的结点既有左子树，也有右子树

例如：

被删关键字 = 50



以其中**序前驱值**替换之（值替换），然后再**删除该前驱结点**。前驱是左子树中最大的结点。

也可以用其**后继**替换之，然后再删除该后继结点。后继是右子树中最小的结点。



示例

在任意一棵非空二叉排序树 T_1 中，删除某结点 p 之后形成二叉排序树 T_2 ，再将 p 插入 T_2 形成二叉排序树 T_3 。下列关于 T_1 与 T_3 的叙述中，正确的是（ ）。

- | | |
|---|---|
| I. 若 p 是 T_1 的叶结点，则 T_1 与 T_3 不同 | × |
| II. 若 p 是 T_1 的叶结点，则 T_1 与 T_3 相同 | ✓ |
| III. 若 p 不是 T_1 的叶结点，则 T_1 与 T_3 不同 | ✓ |
| IV. 若 p 不是 T_1 的叶结点，则 T_1 与 T_3 相同 | × |
- A. 仅I、III B. 仅I、IV C. 仅II、III D. 仅II、IV

答案： C。



9.3.2 平衡二叉树 (AVL)

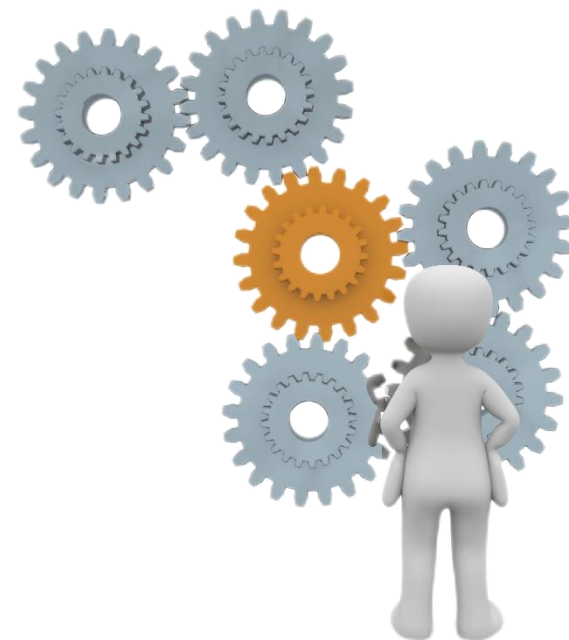
1、什么是平衡二叉树

若一棵二叉树中每个结点的左、右子树的高度至多相差1，则称此二叉树为**平衡二叉树 (AVL)**。

平衡因子：该结点左子树的高度减去右子树的高度。



所有结点的平衡因子的绝对值小于或等于1，该二叉树称为**平衡二叉树**。



二叉树结构



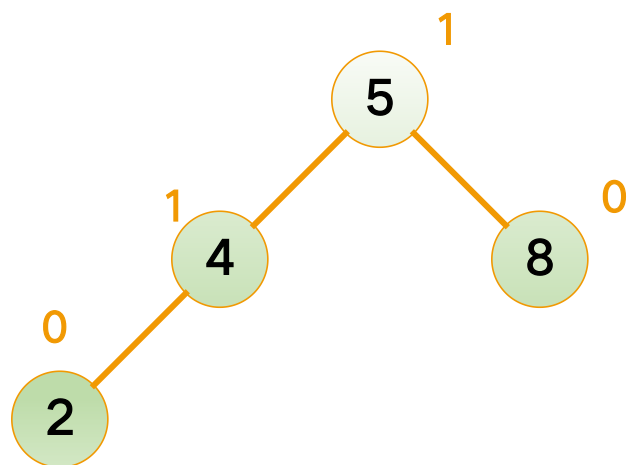
满足BST性质

二叉排序树

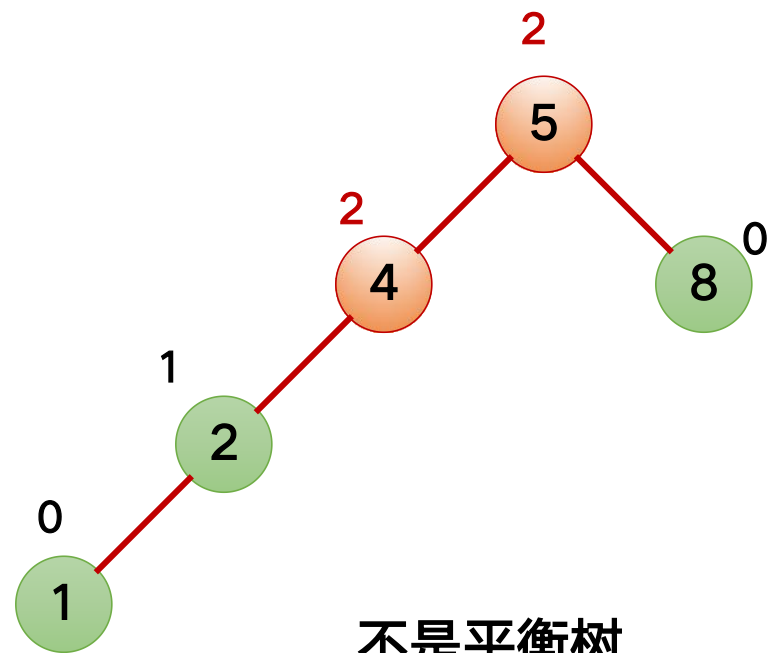


所有结点的平衡因子的绝对值 ≤ 1 ：结构约束

平衡二叉树



是平衡树



不是平衡树

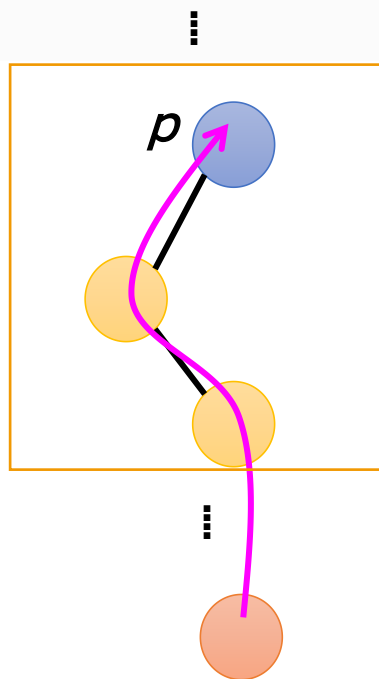
2、平衡二叉树的插入过程

- 平衡二叉树中插入关键字为 k 的新结点方式与二叉排序树相似。
- 但插入后可能破坏平衡二叉树的平衡性，解决方法是**调整**。



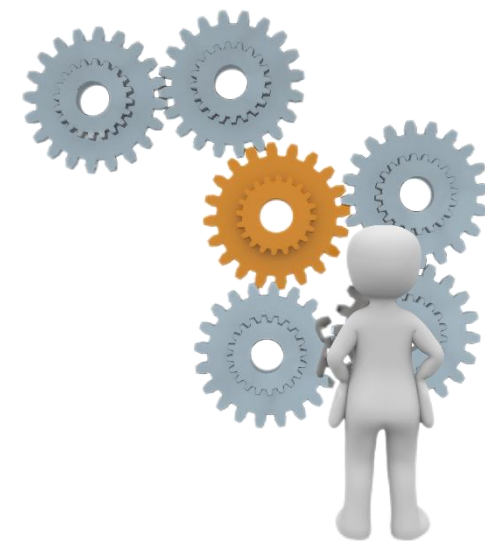
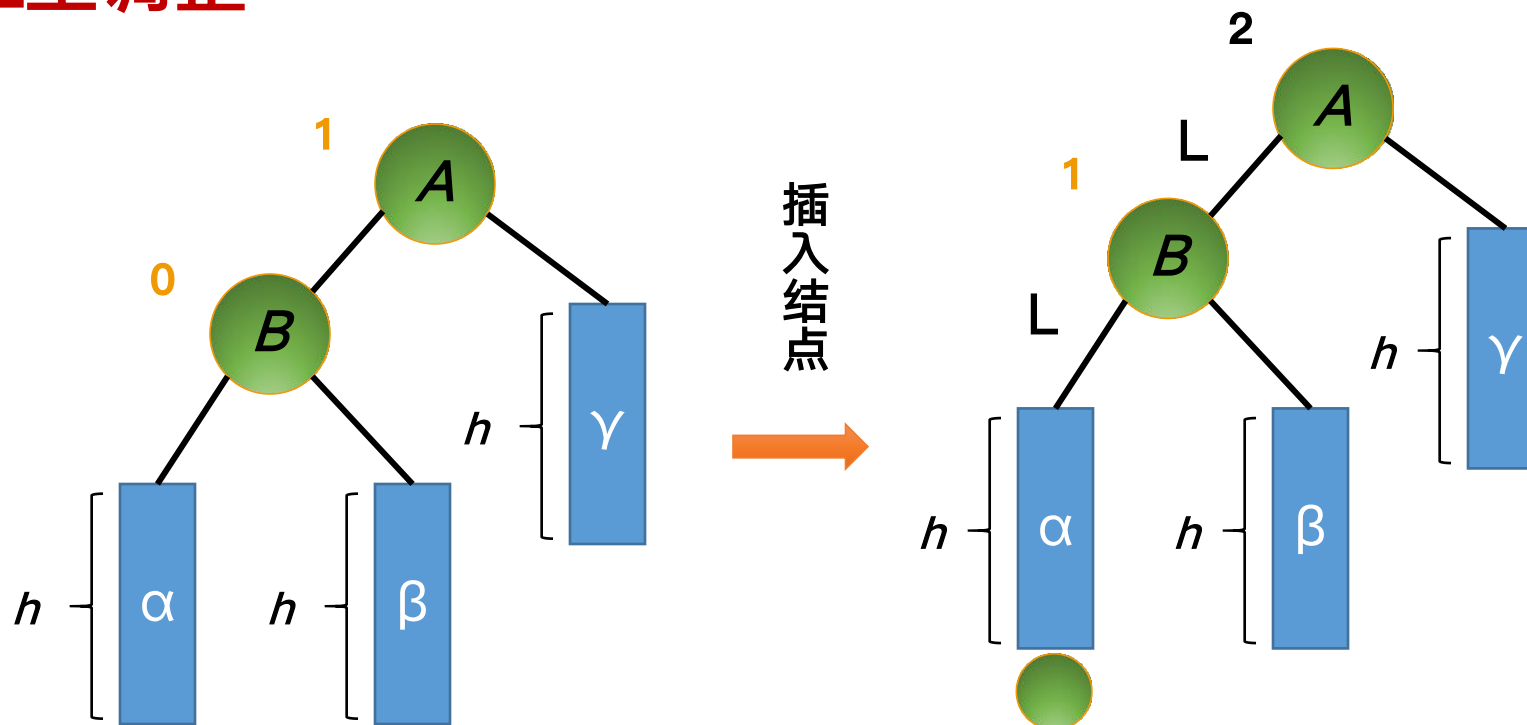
插入过程：

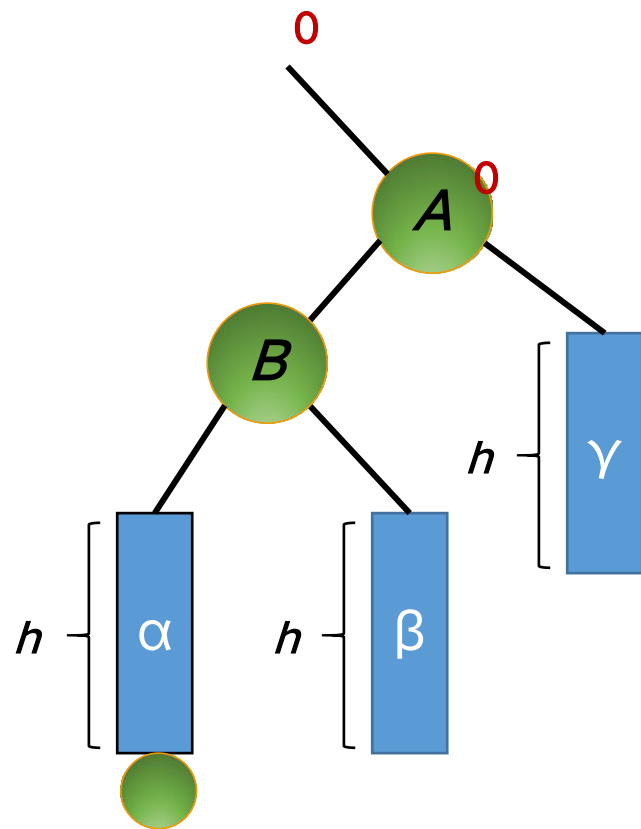
- ❶ 插入一个新结点（总是作为叶子结点插入的）。
- ❷ 从该结点到根结点方向求结点的平衡因子。
- ❸ 若均为平衡的（ $|\text{平衡因子}| \leq 1$ ），不需要调整。
- ❹ 若找到一个失去平衡的结点 p ，选择 p 和路径上另外两个结点进行调整。构成4种调整操作。



3层结点构成LL、
RR、LR和RL关系

(1) LL型调整





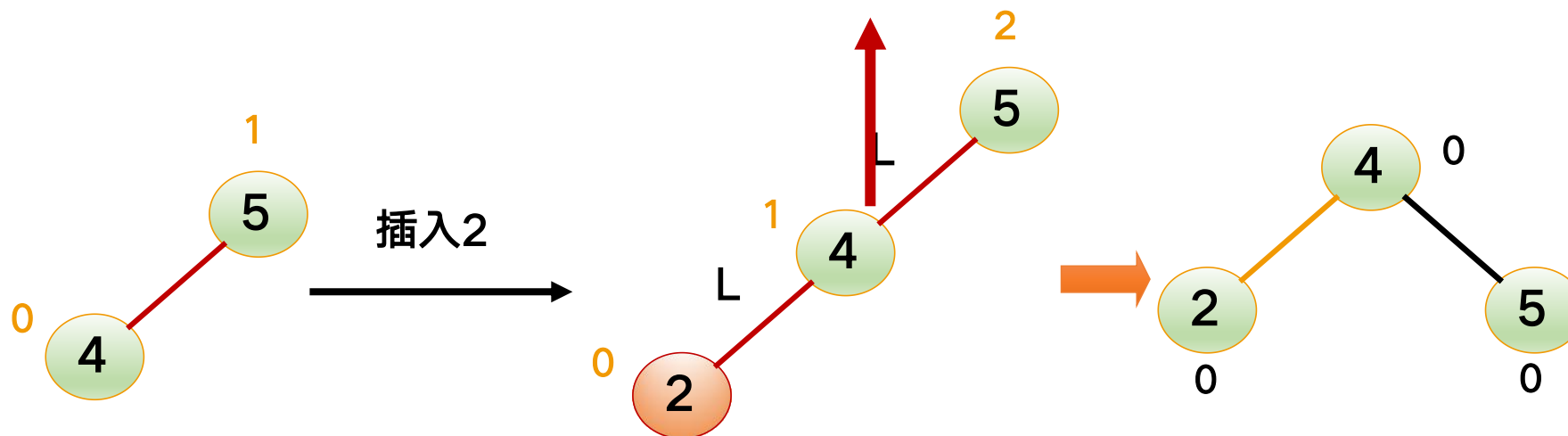
LL调整后的结果

LL型调整过程：

- B结点带左子树 α 一起上升
- A结点成为B的右孩子
- 原来B结点的右子树 β 作为A的左子树



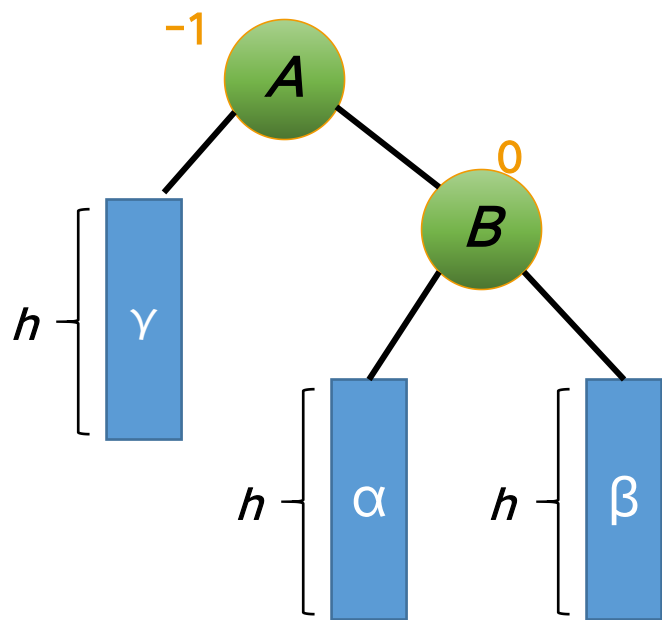
AVL树LL调整演示



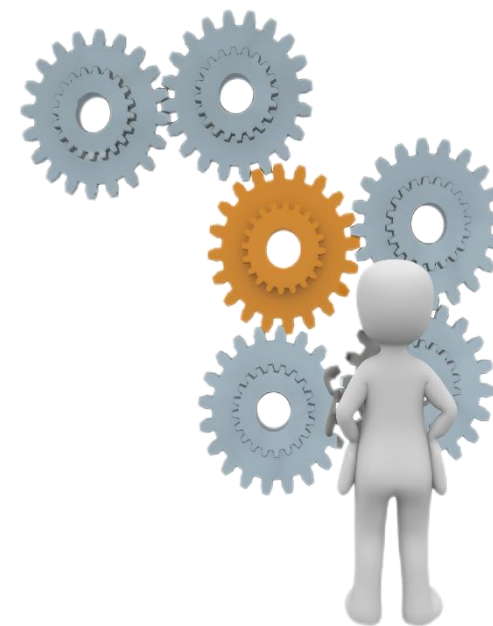
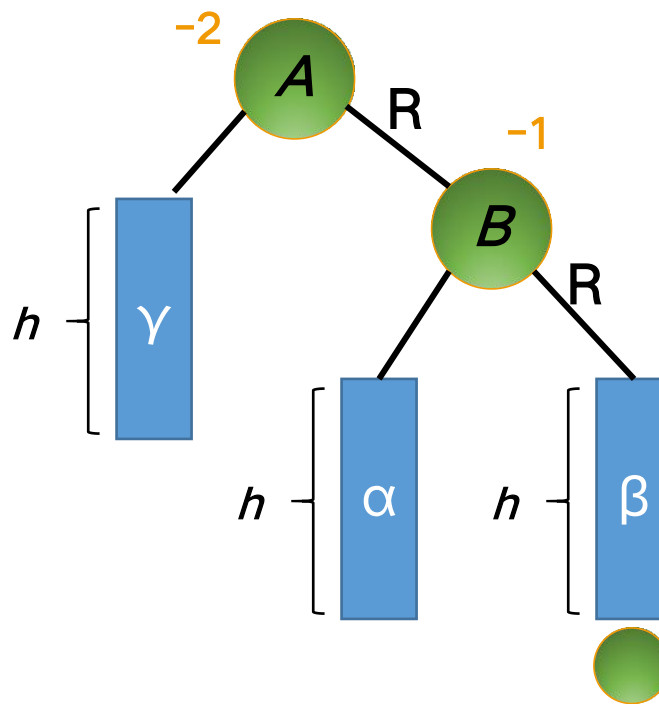
插入关键字2的结果

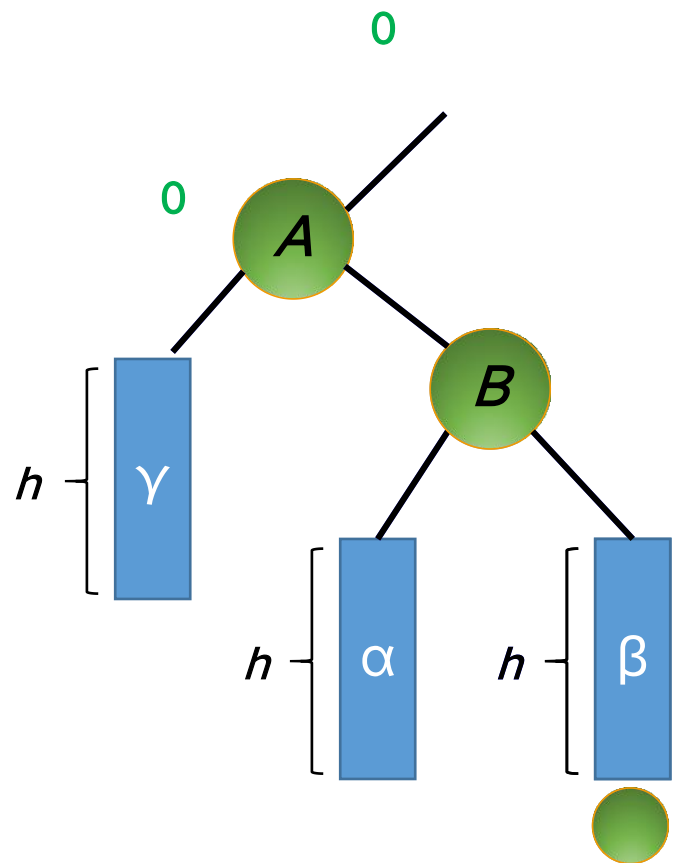
调整完毕

(2) RR型调整



插入结点





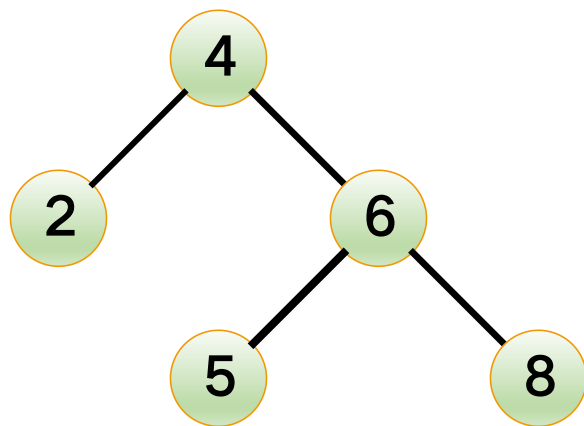
RR调整后的结果

RR型调整过程：

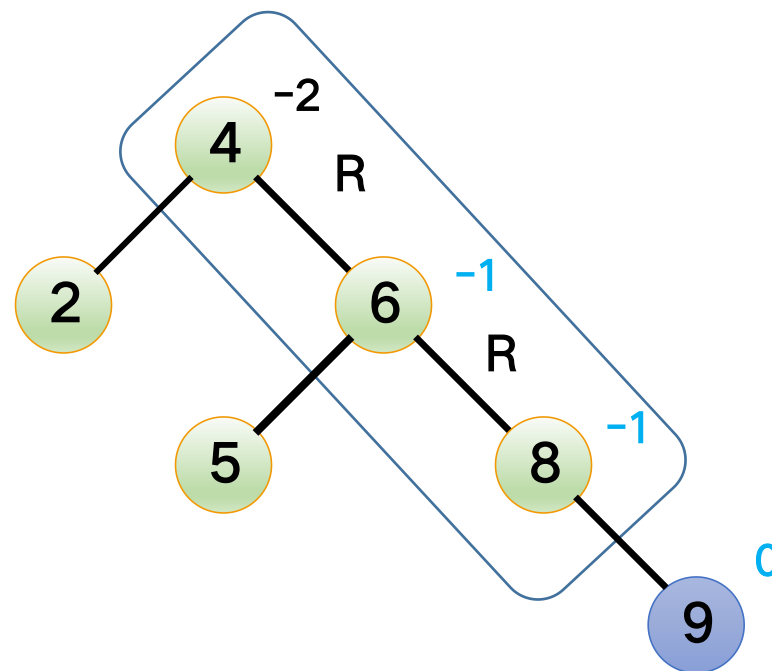
- B结点带右子树 β 一起上升
- A结点成为B的左孩子
- 原来B结点的左子树 α 作为A的右子树

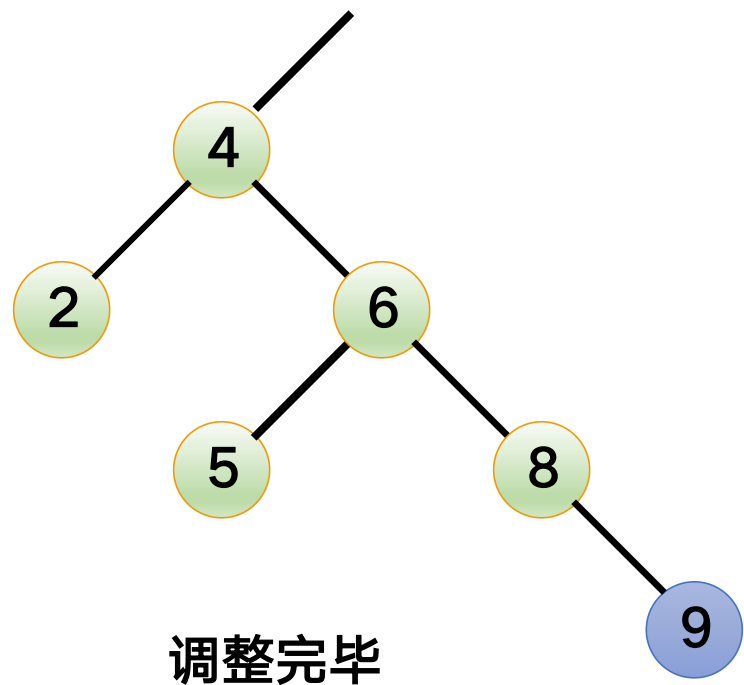


AVL树RR调整演示



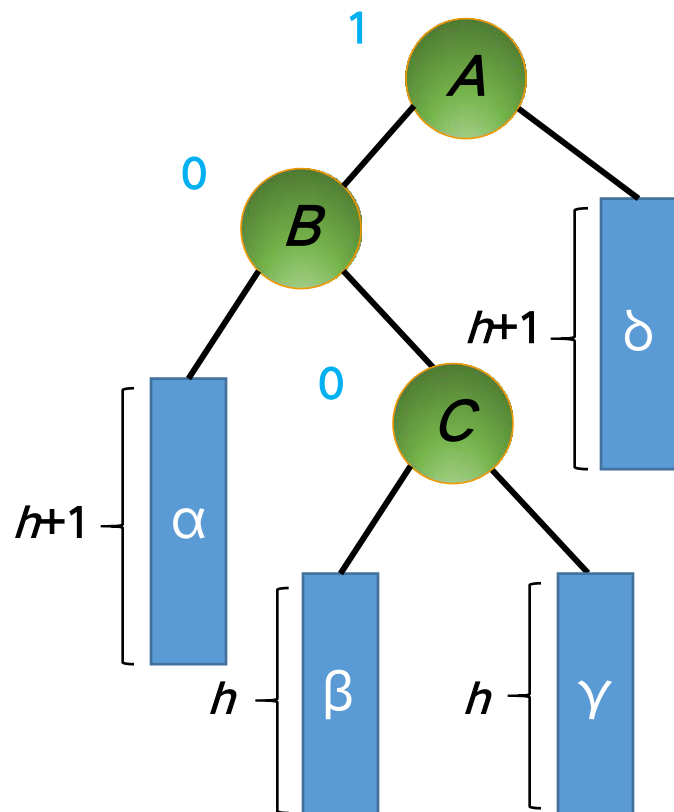
插入9



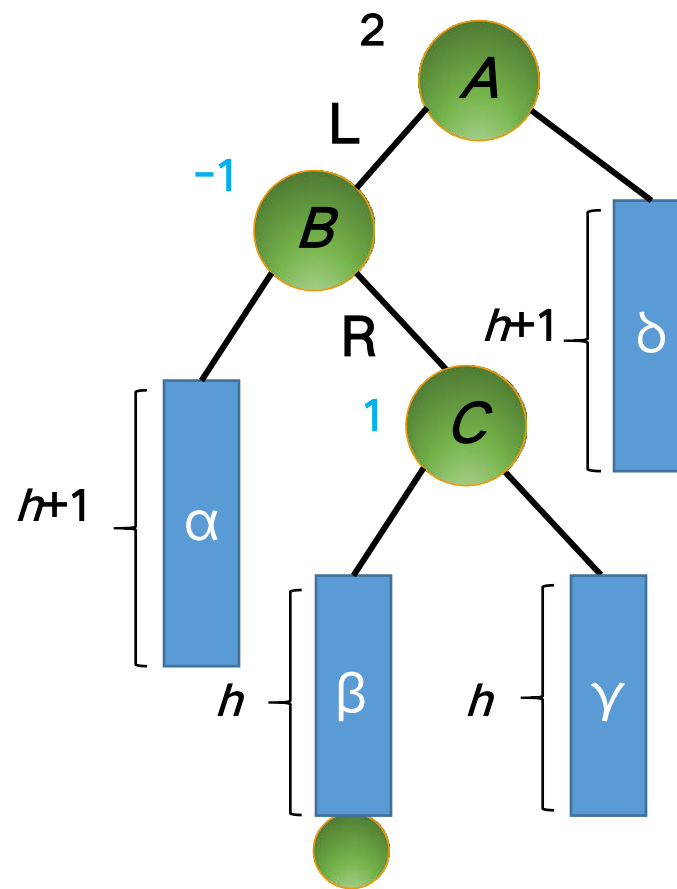


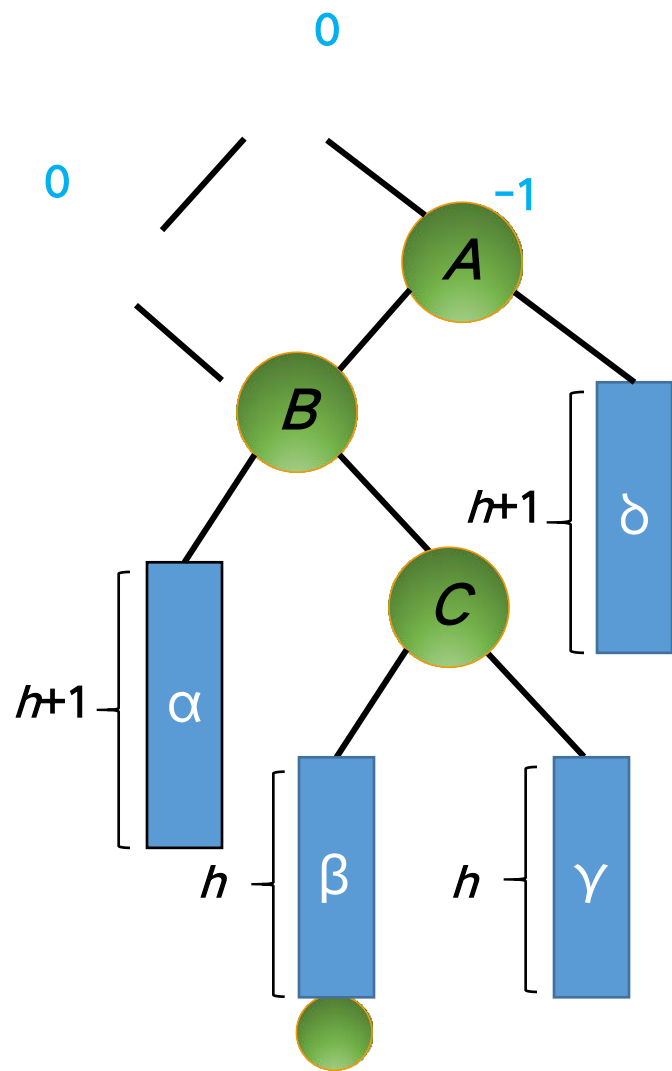
调整完毕

(3) LR型调整



插入结点



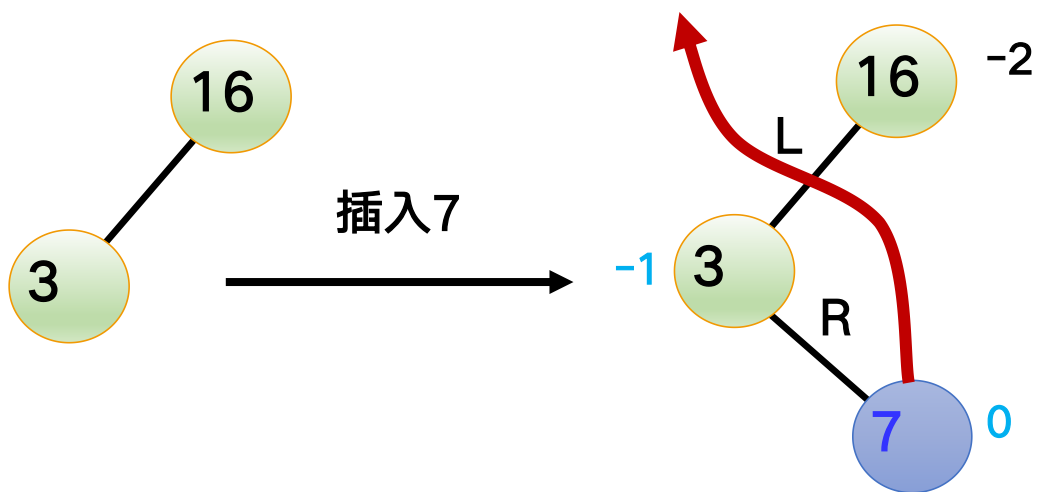


LR调整后的结果

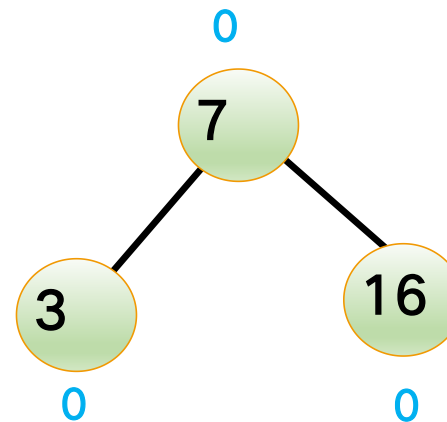
LR型调整过程:

- C结点穿过A、B结点上升
- B结点成为C的左孩子，A结点成为C的右孩子
- 原来C结点的左子树 β 作为B的右子树；原来C结点的右子树 γ 作为A的左子树

AVL树LR调整演示



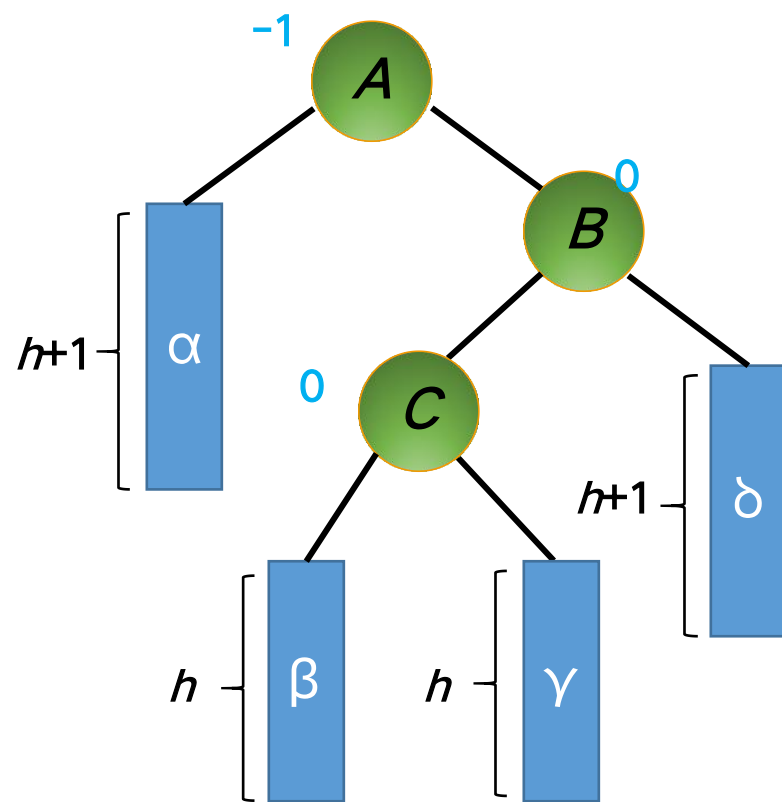
插入关键字7的结果



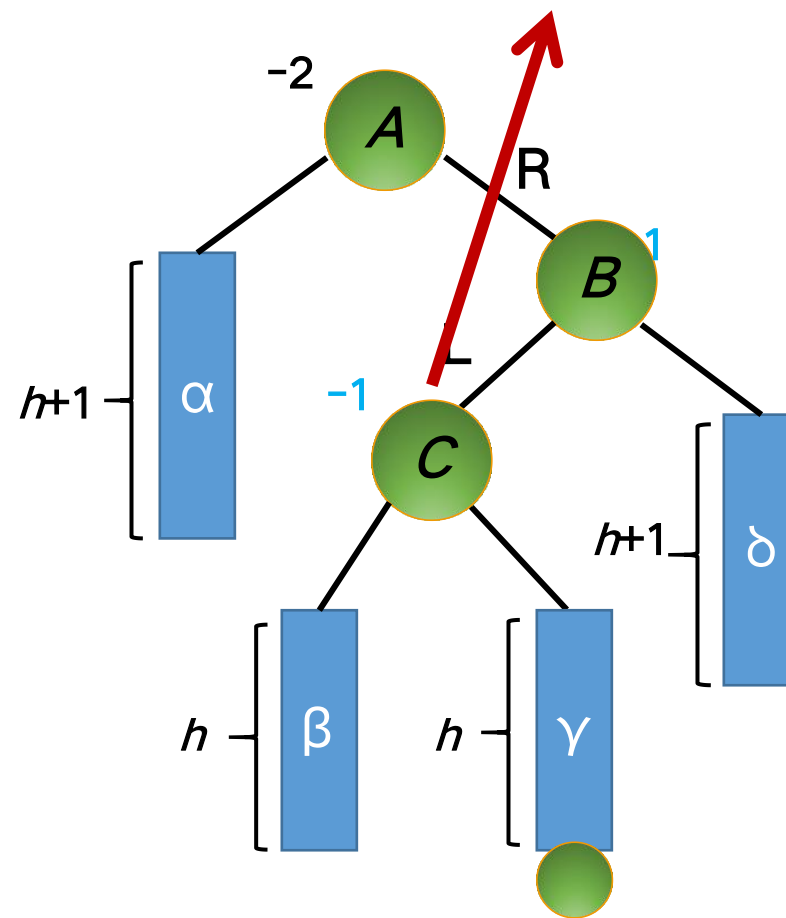
调整完毕



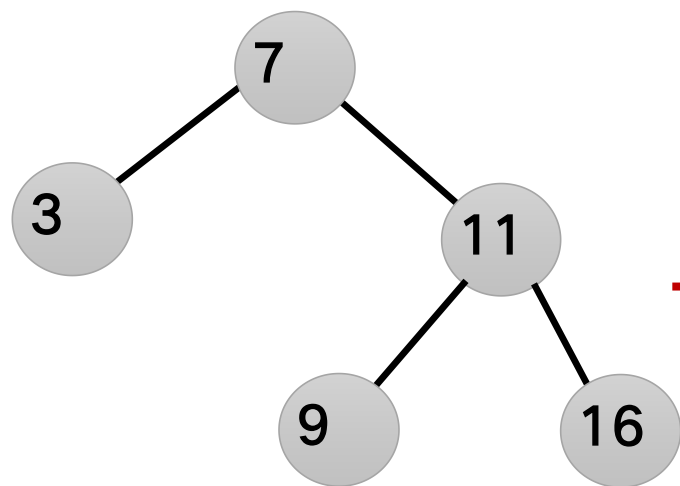
(4) RL型调整



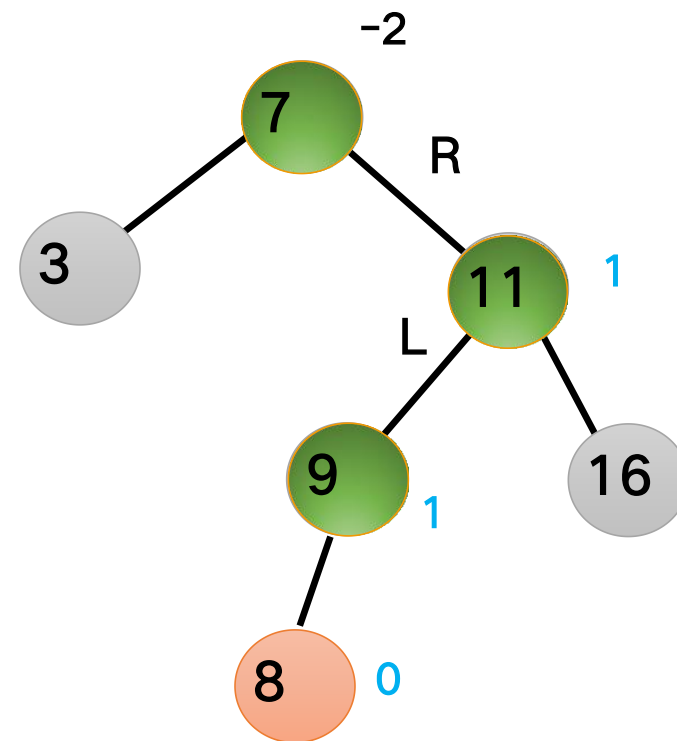
插入结点



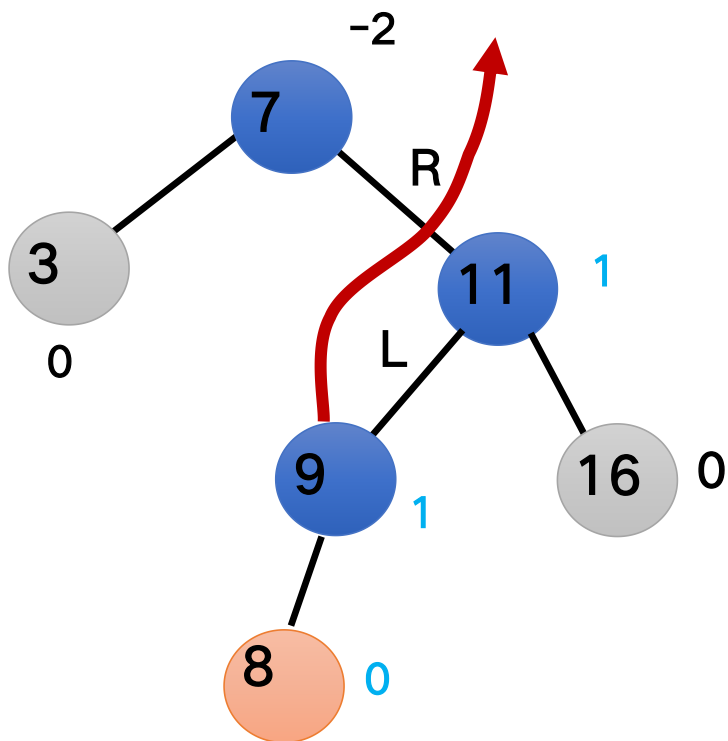
AVL树RL调整演示



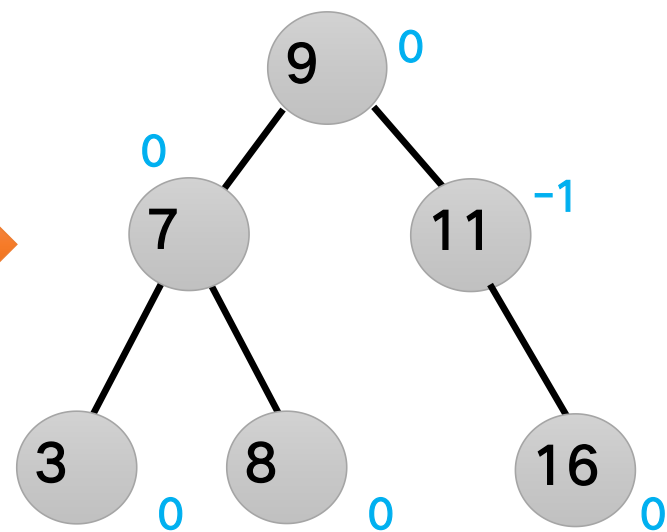
插入8



插入关键字8的结果

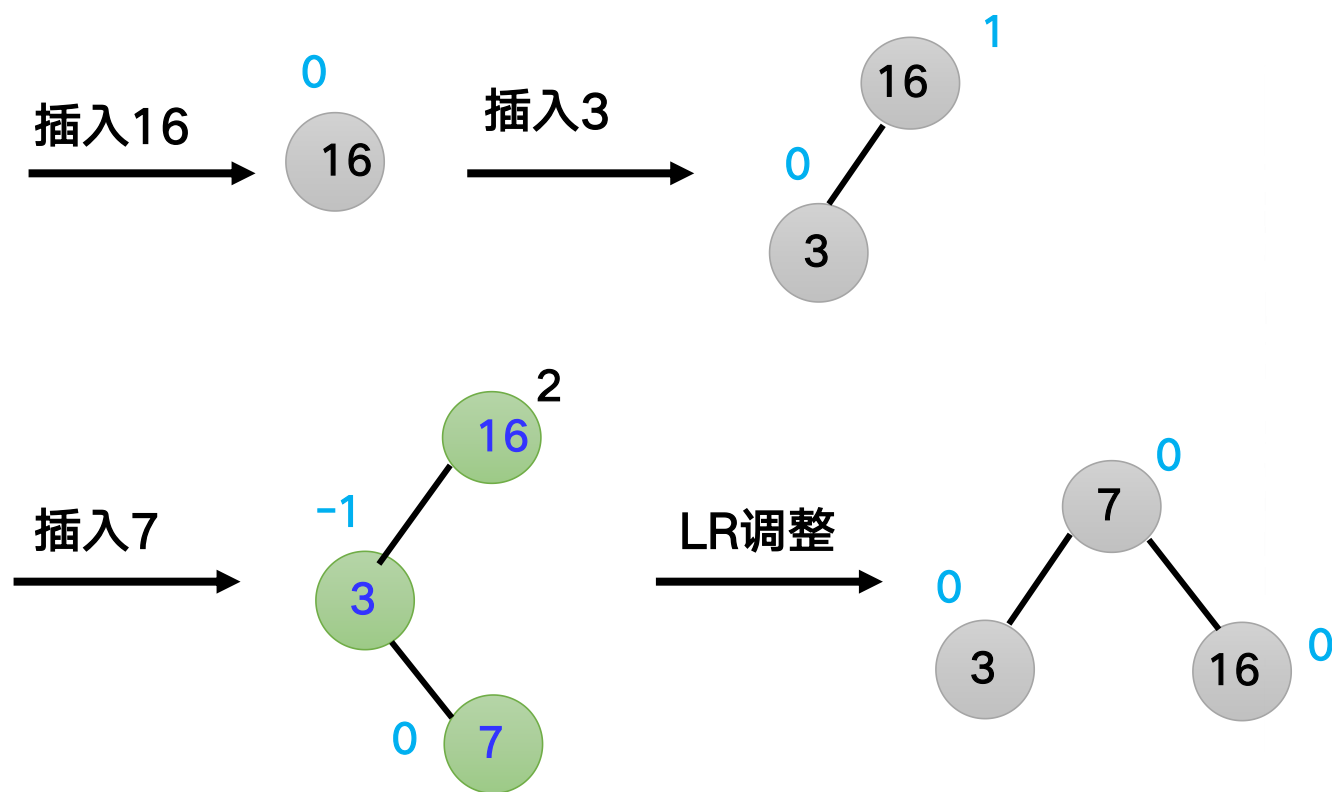


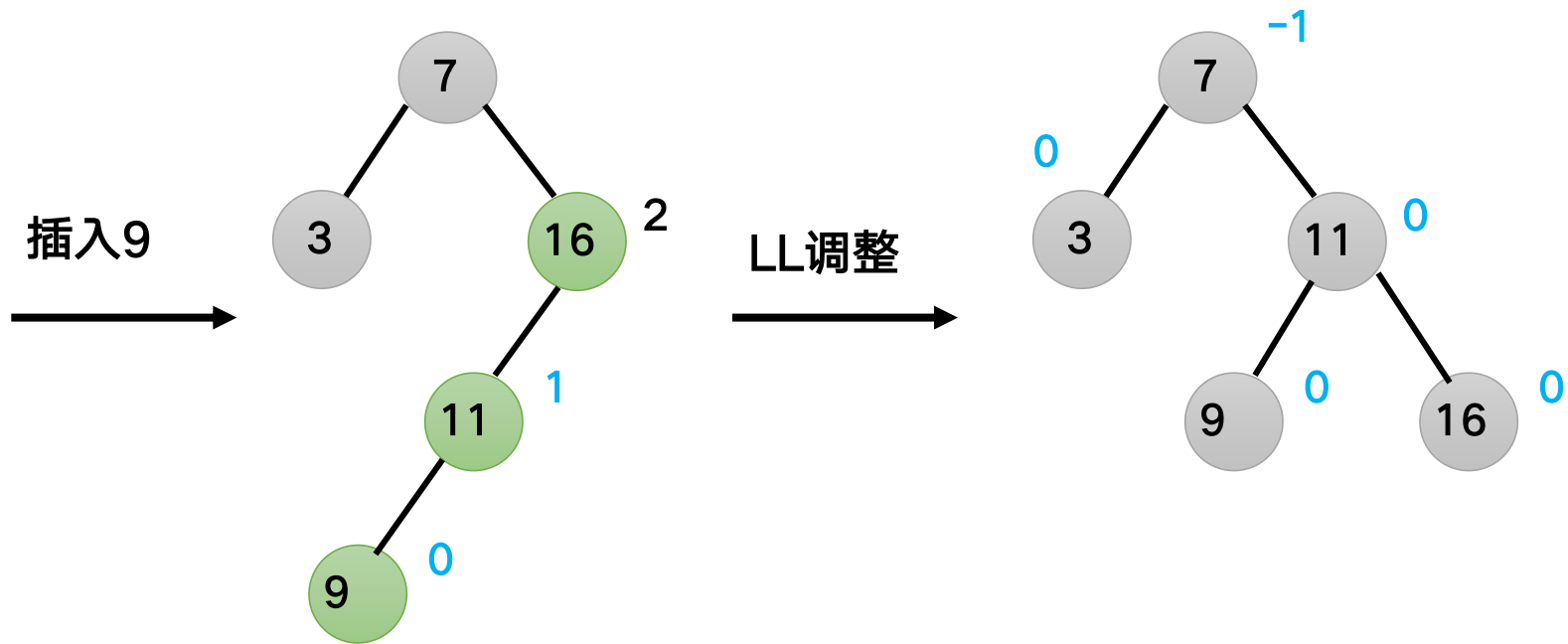
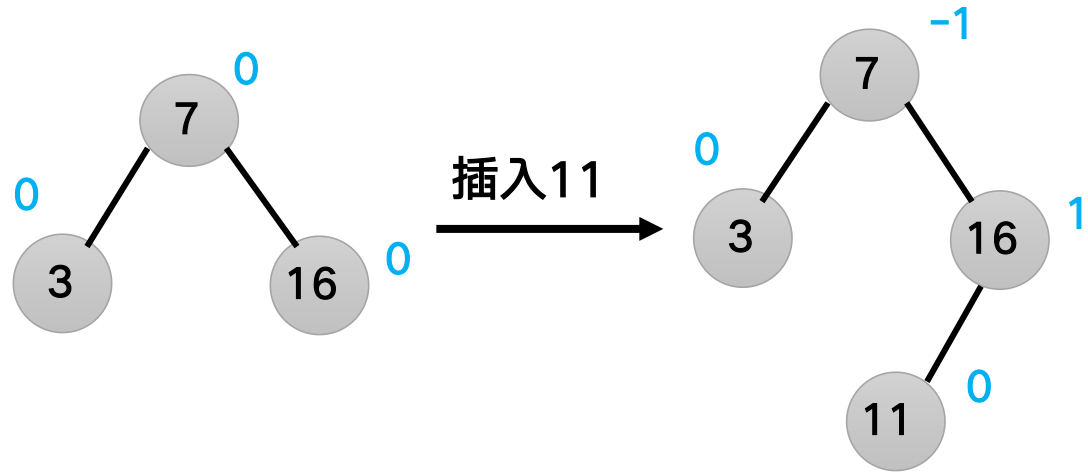
RL调整

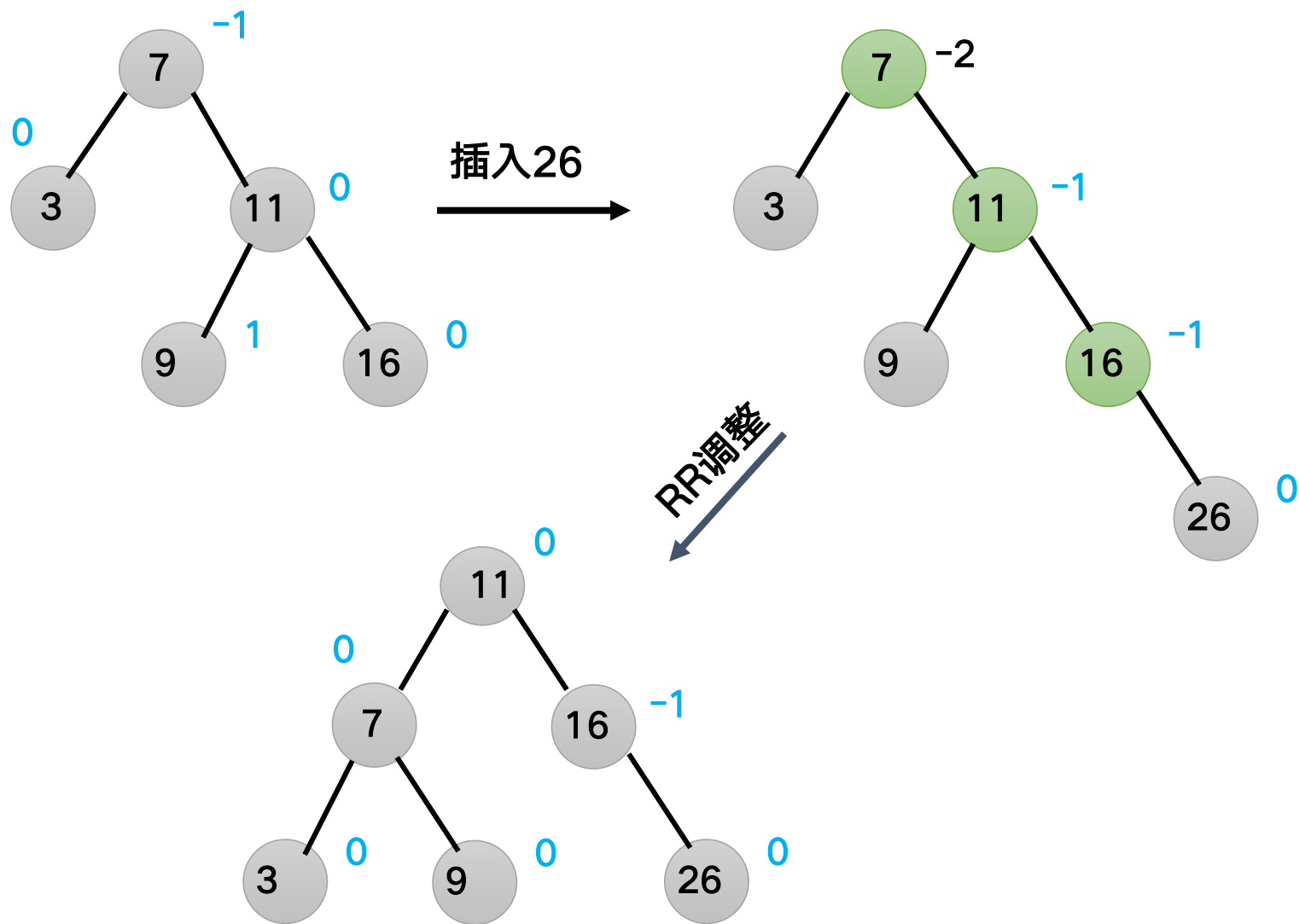


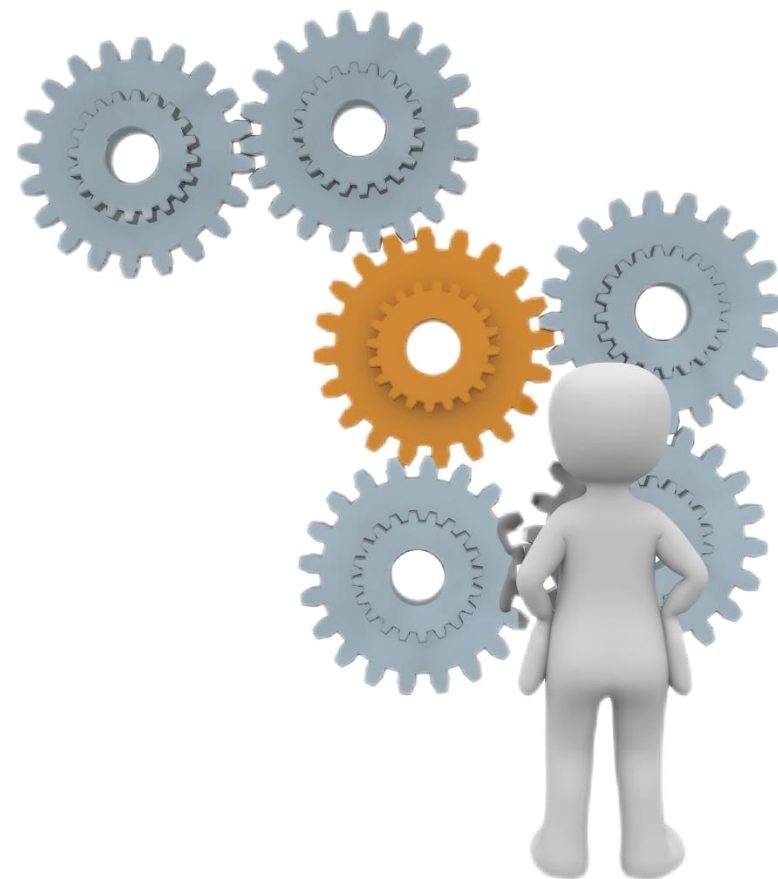
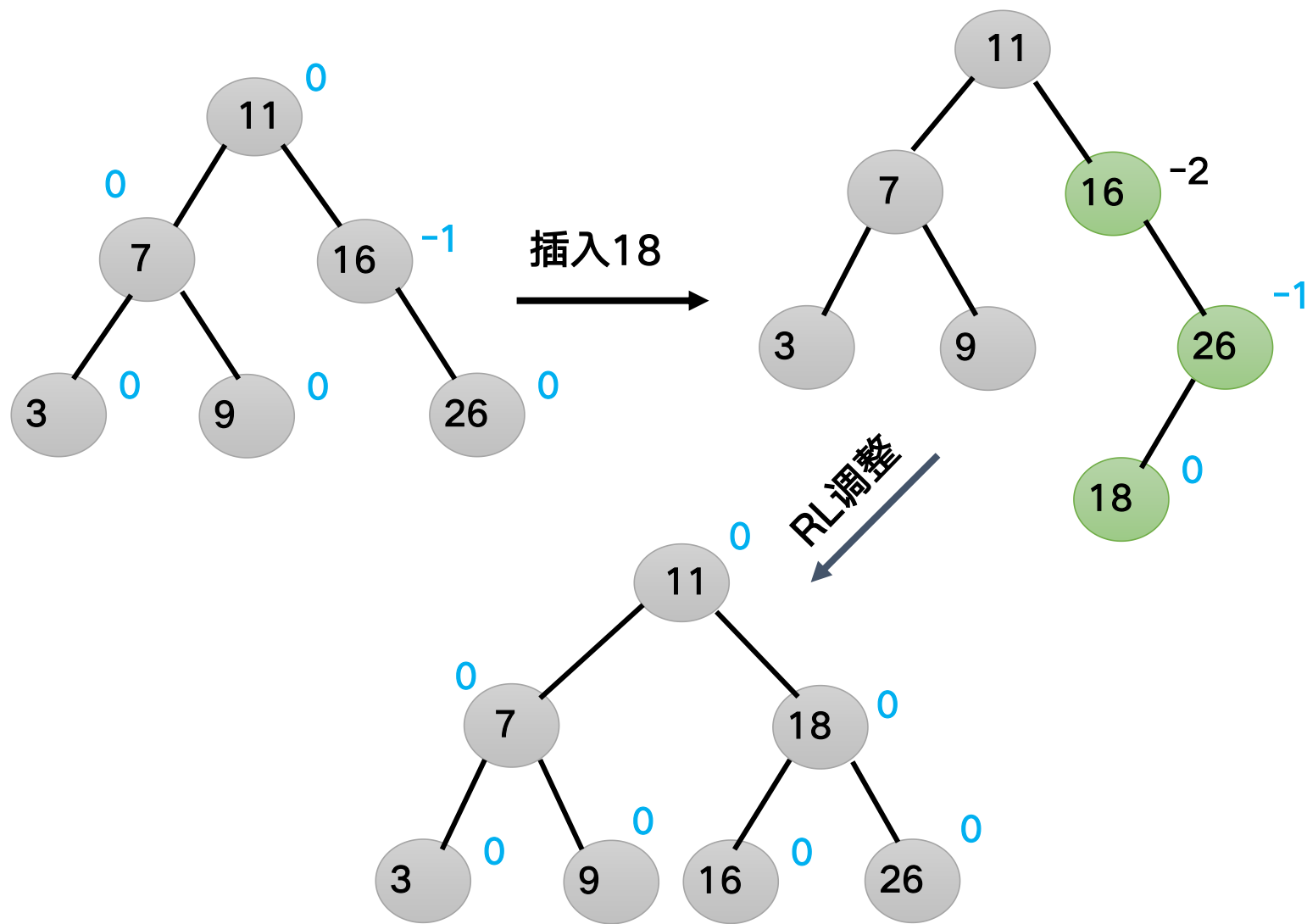
调整完毕

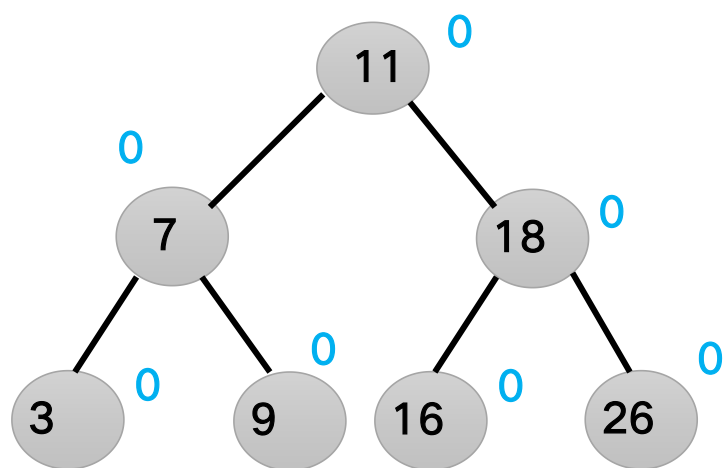
【例9.5】 输入关键字序列(16, 3, 7, 11, 9, 26, 18, 14, 15), 给出构造一棵AVL树的步骤。



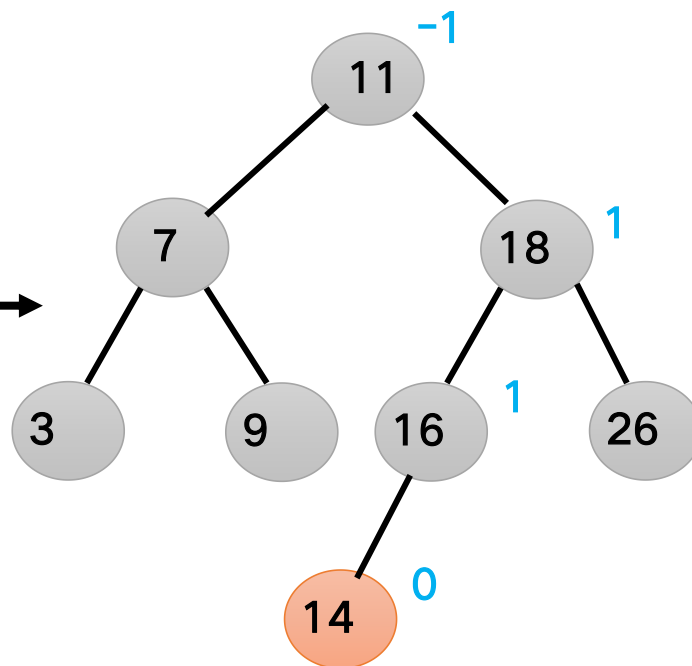


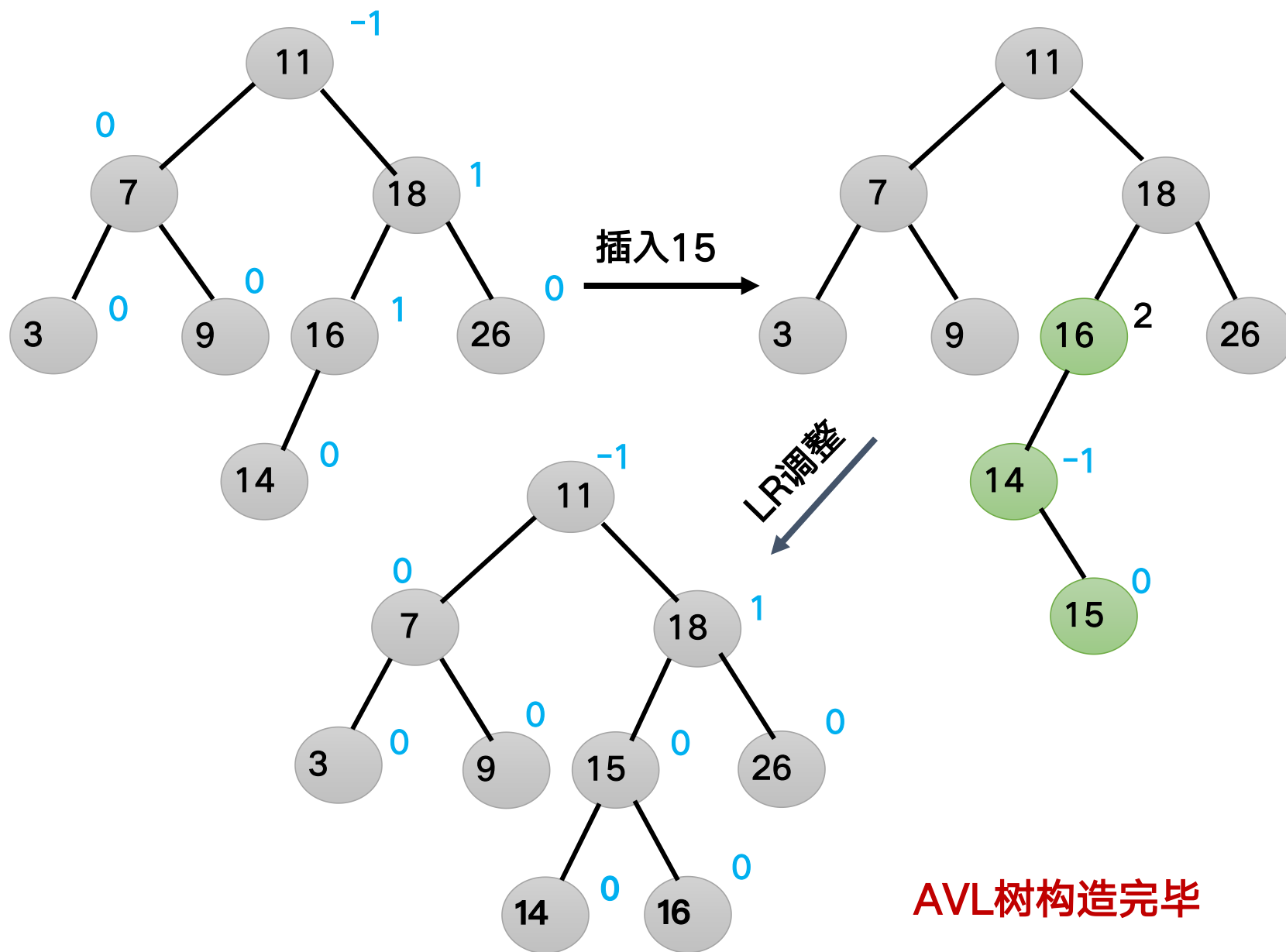






插入14





AVL树构造完毕

3、平衡二叉树的删除过程

- 平衡二叉树中删除关键字为 k 的结点方式与二叉排序树删除相似。
- 但删除后可能破坏平衡二叉树的平衡性，解决方法是**调整**。



删除过程

(1) 查找：先在平衡二叉树中查找到关键字为 k 的结点 p 。

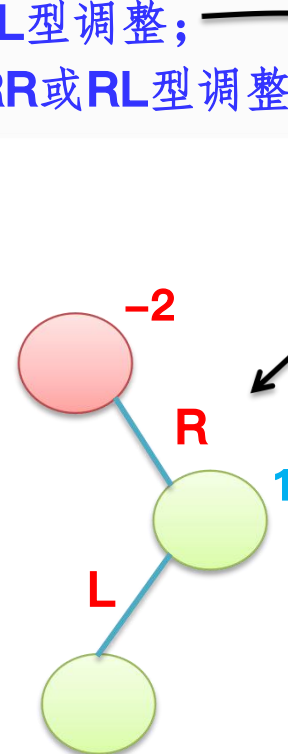
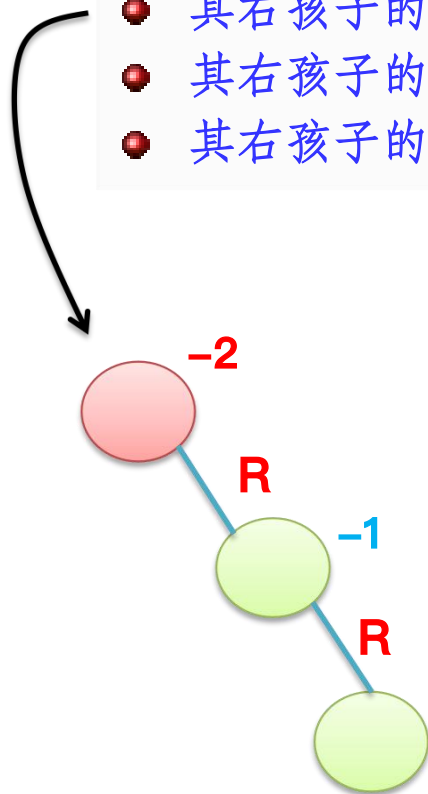
(2) 删除：删除 p 结点分以下几种情况。

- 叶子结点：直接删除该结点。
- 单分支结点：用 p 结点的左或右孩子结点替代 p 结点（结点替换）。
- 双分支结点：用 p 结点的中序前驱（或中序后继）结点 q 的值替换 p 结点的值。再删除结点 q 。

(3) 调整：若被删除的是结点 q ，则从根结点到结点 q 的路径之逆向根结点方向查找第一个失去平衡的结点：

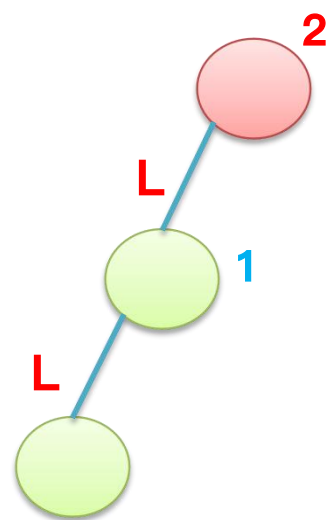
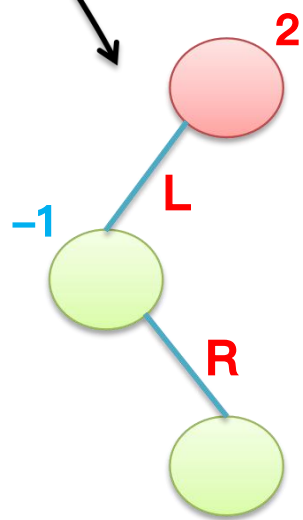
- 若所有结点都是平衡的，则不需要调整。
- 假设找到某个结点的平衡因子为 -2 ：

- 其右孩子的平衡因子是 -1 ，则作 **RR** 型调整；
- 其右孩子的平衡因子是 1 ，则作 **RL** 型调整；
- 其右孩子的平衡因子是 0 ，则作 **RR** 或 **RL** 型调整均可。



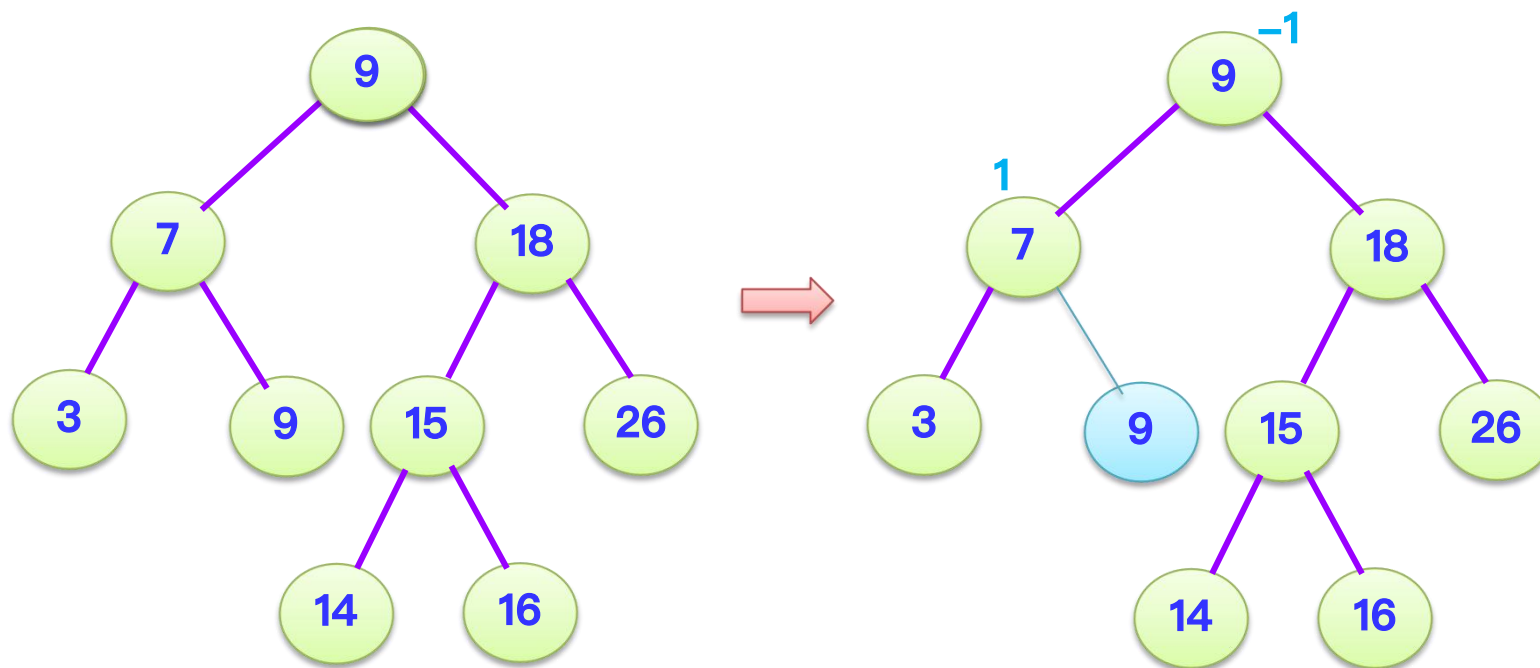
■ 假设找到某个结点的平衡因子为**2**:

- 其左孩子的平衡因子是**-1**，则作**LR**型调整；
- 其右孩子的平衡因子是**1**，则作**LL**型调整；
- 其右孩子的平衡因子是**0**，则作**LR**或**LL**型调整均可。



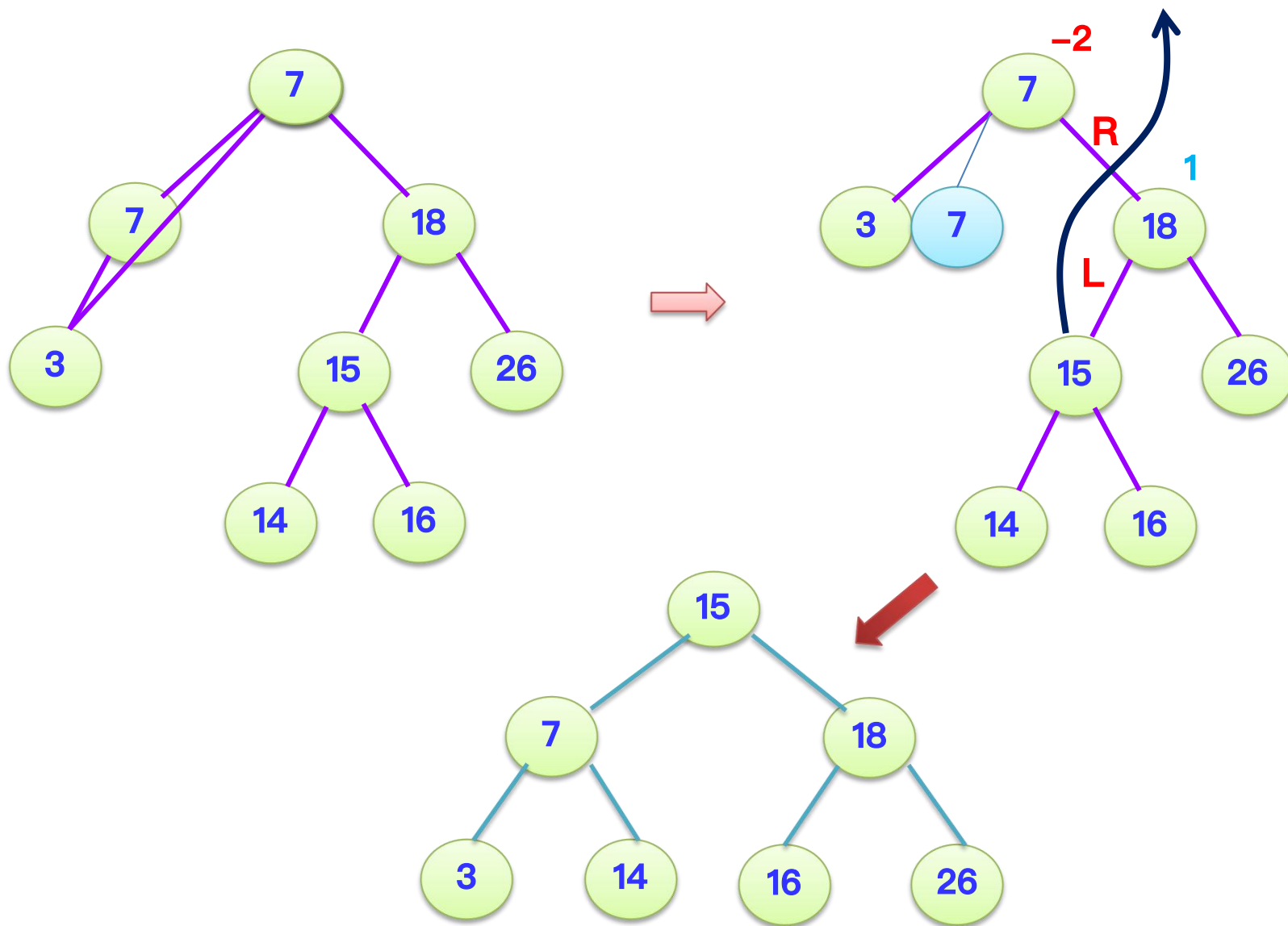
【例9-6】 对例9.5生成的AVL树，给出依次删除结点11、9和15的步骤。

删除结点11

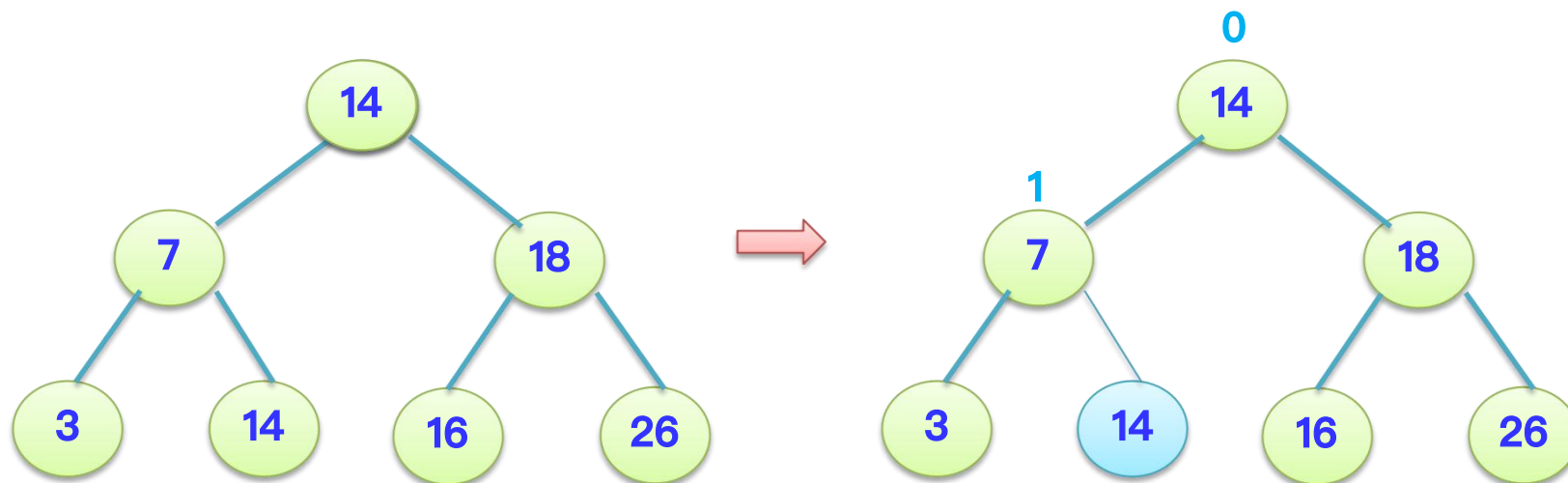


都是平衡的，不需要调整

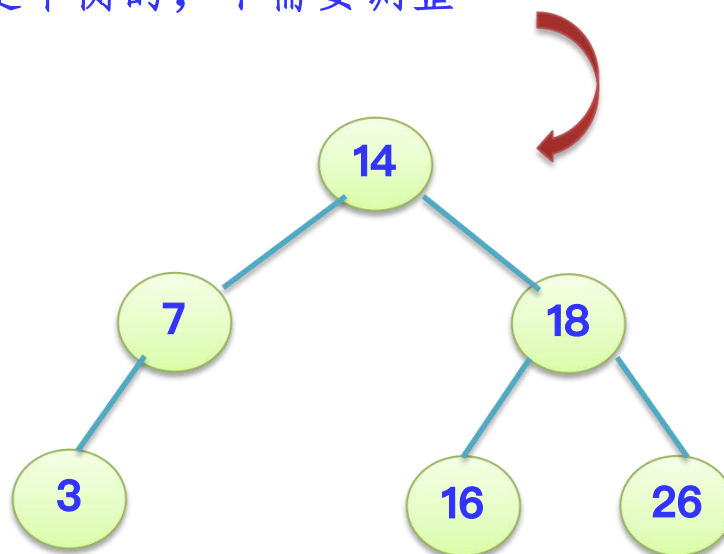
删除结点9



删除结点15



都是平衡的，不需要调整

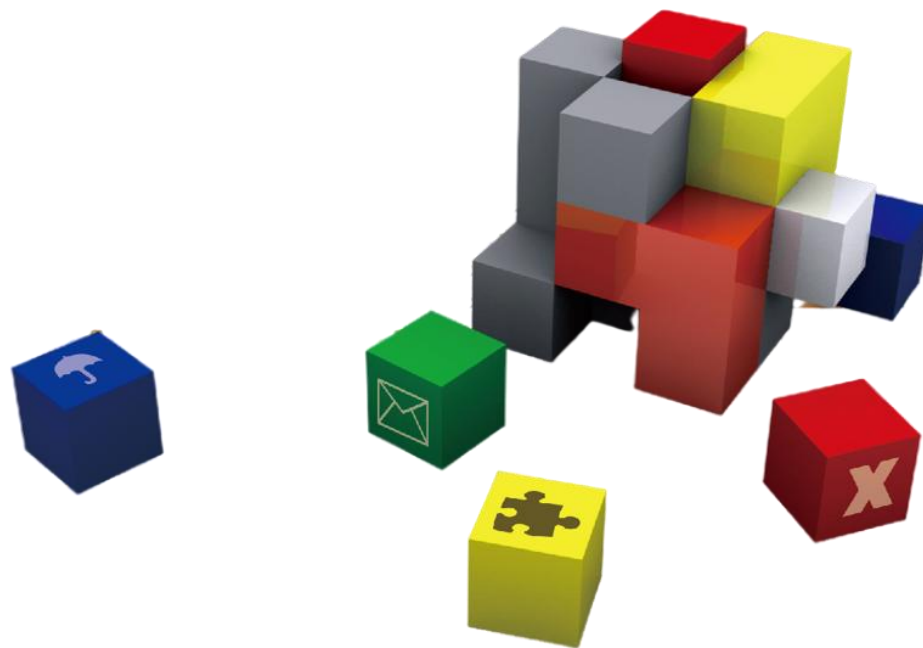


4、平衡二叉树的查找

- 在平衡二叉树上进行查找的过程和在二叉排序树上进行查找的过程完全相同，因此在平衡二叉树上进行查找关键字的比较次数不会超过平衡二叉树的高度。

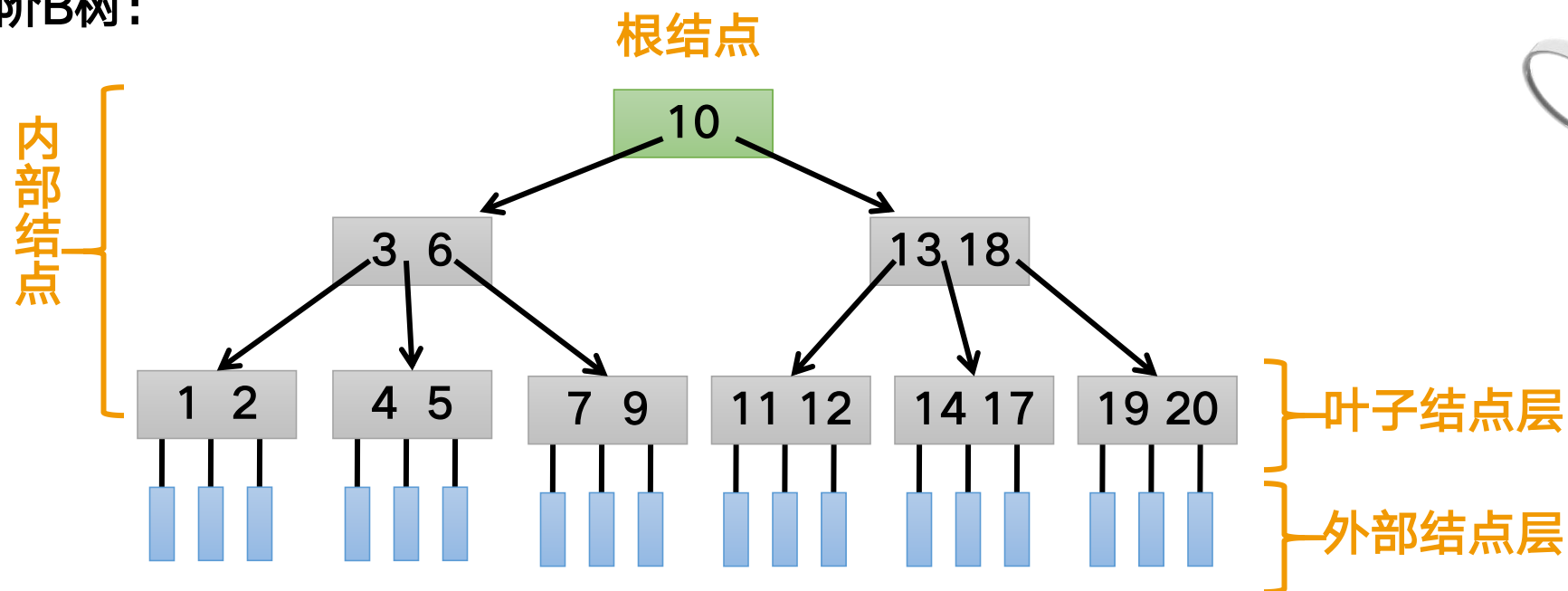
9.3.4 B树

B树又称为**多路平衡查找树**，是一种组织和维护**外存文件系统**非常有效的数据结构。



1、B树的定义

一棵3阶B树：

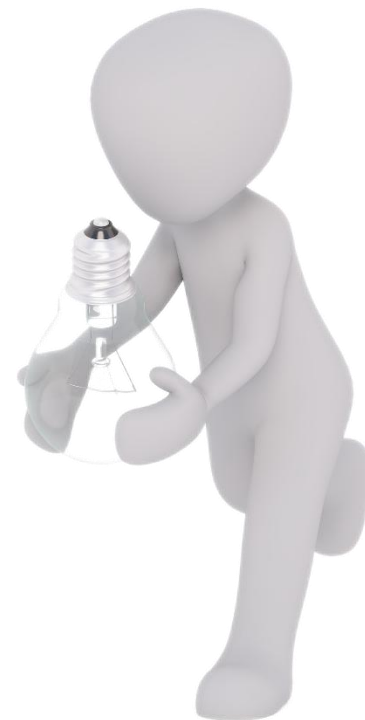
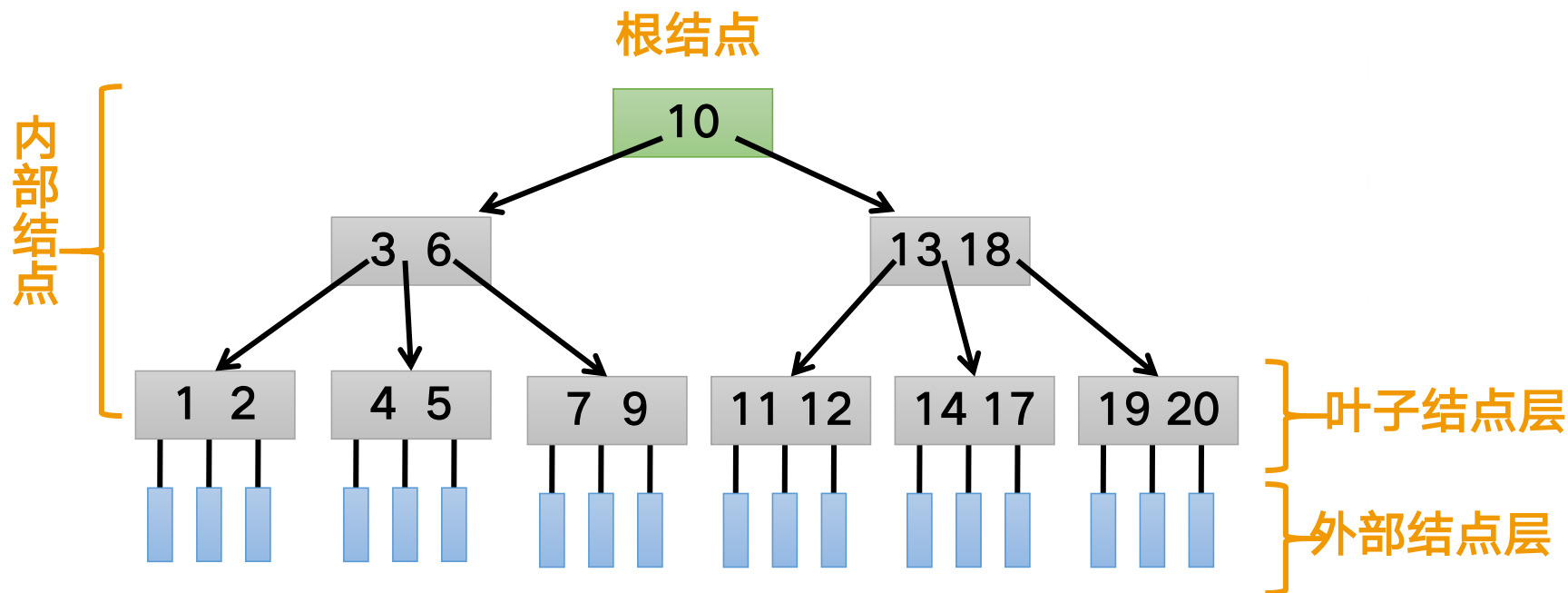


一棵 m 阶B树或者是一棵空树，或者是满足要求的 m 叉树：

① 树中每个结点至多有 m 个孩子结点（即至多有 $m-1$ 个关键字）

最多关键字个数 $\text{Max} = m-1$

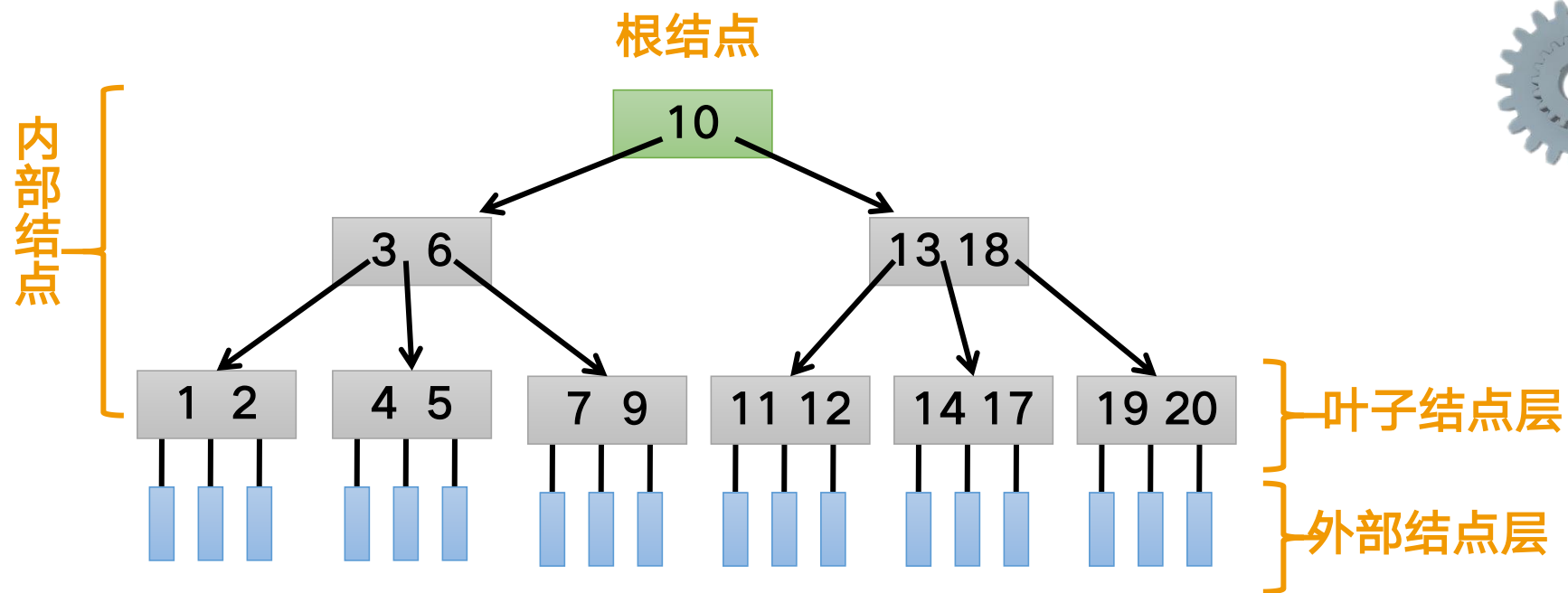
一棵3阶B树:



- ② 除根结点外，其他非叶子结点至少有 $\lceil m/2 \rceil$ 个孩子结点（即至少有 $\lceil m/2 \rceil - 1 = \lfloor (m-1)/2 \rfloor$ 个关键字）；

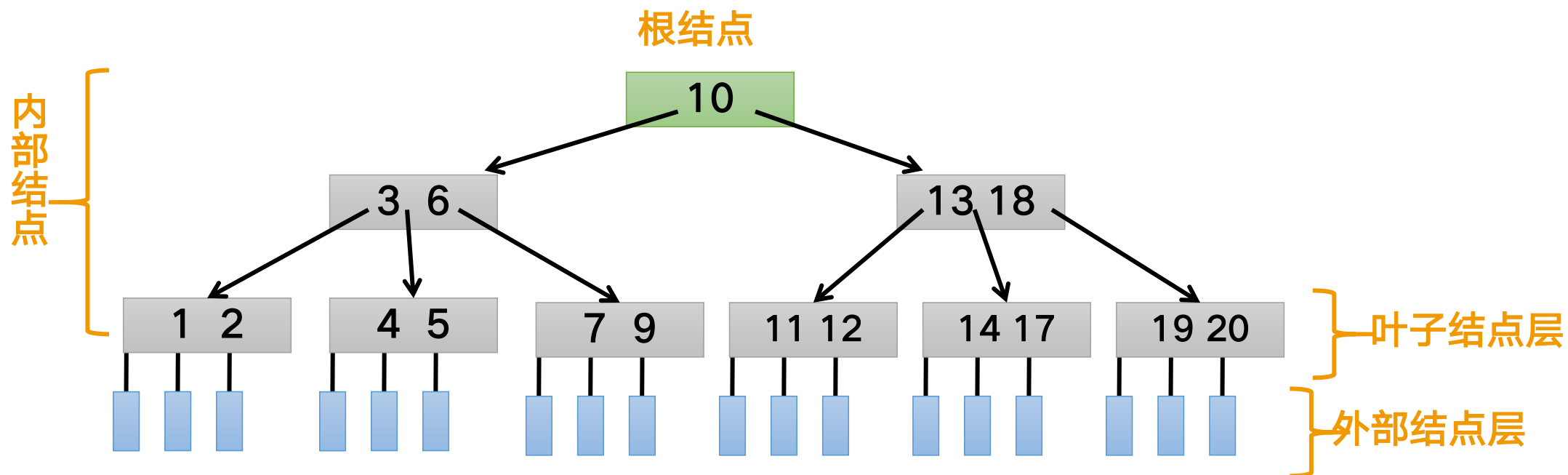
$$\text{最少关键字个数Min} = \lceil m/2 \rceil - 1$$

一棵3阶B树：

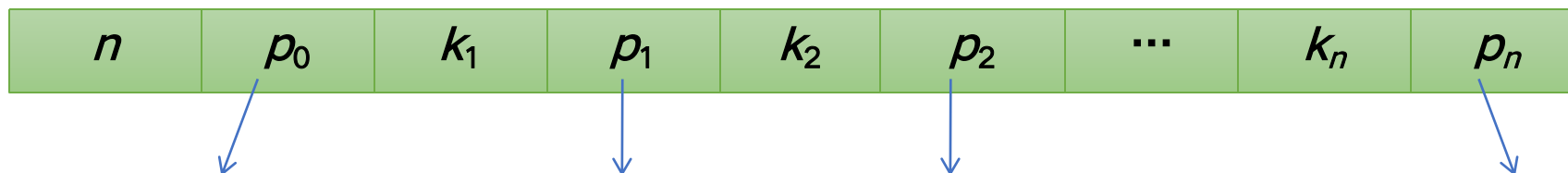


③ 若根结点不是叶子结点，则根结点至少有两个孩子结点；

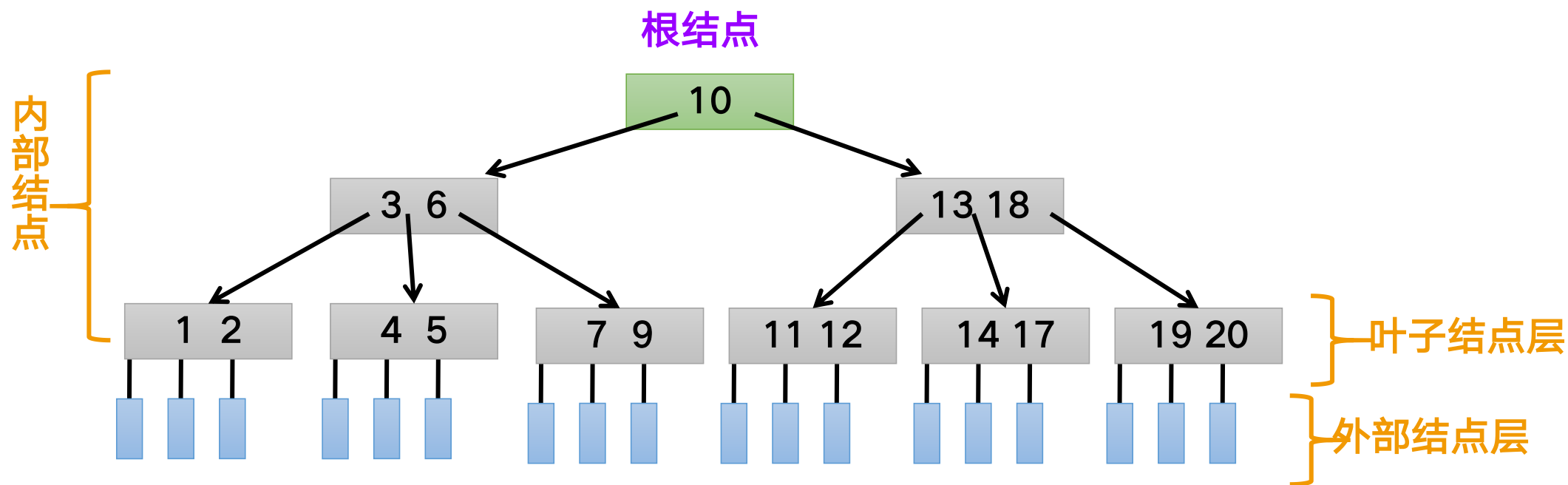
一棵3阶B树:



④ 每个结点的结构如下，结点中按关键字大小顺序排列：



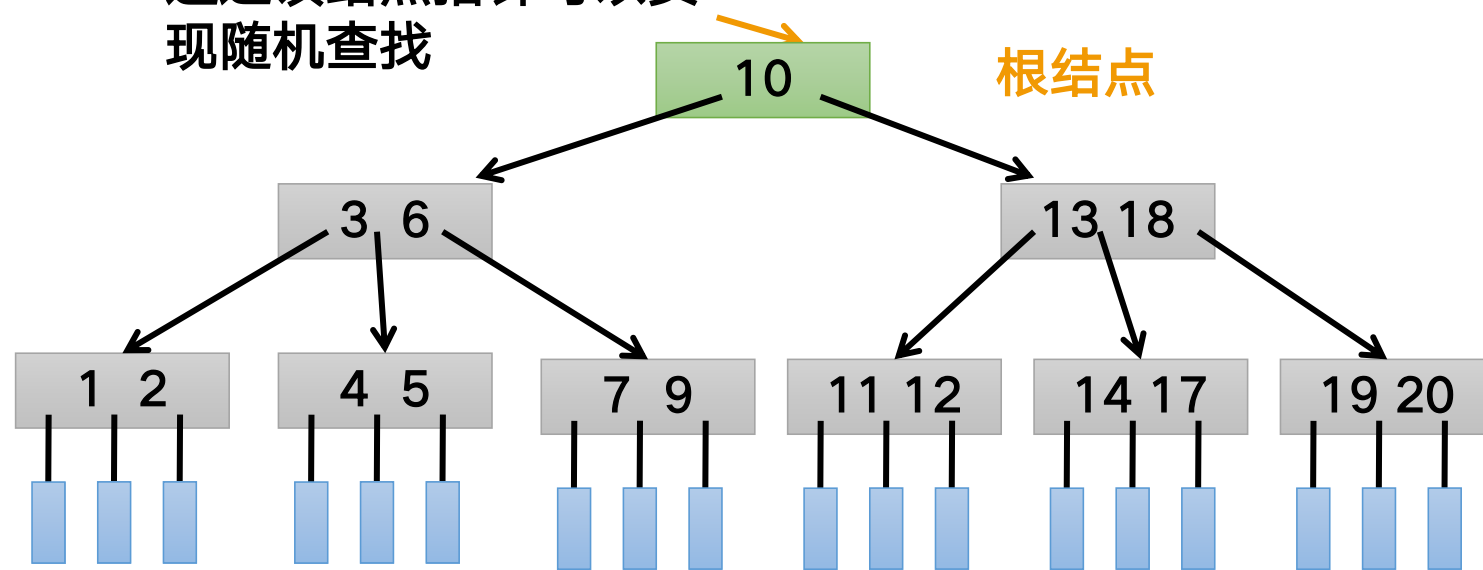
一棵3阶B树:



⑤ 所有外部结点都在同一层上。B树是所有结点的平衡因子均等于0的多路查找树。

注意：在计算B树的高度时，有时计入最底层的外部结点

通过该结点指针可以实现随机查找



一棵3阶B树

- 非根非外部结点的关键字个数：1 ~ 2。
- 非根非外部结点的孩子结点个数：2 ~ 3。

说明：外部结点就是失败结点，指向它的指针为空，不含有任何信息，是虚设的。一棵B树中总有 n 个关键字，则外部结点个数为 $n+1$ 。



在B树的存储结构中，结点的类型声明如下：

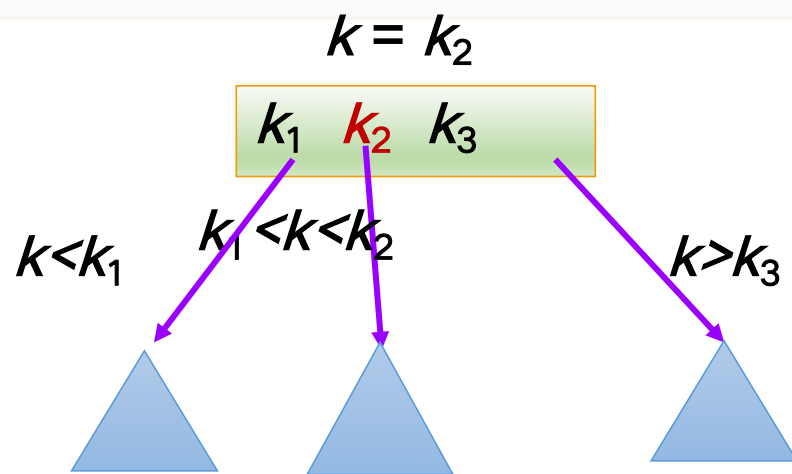
```
#define MAXM 10      //定义B树的最大的阶数
typedef int KeyType;  //KeyType为关键字类型

typedef struct node
{
    int keynum;        //结点当前拥有的关键字的个数
    KeyType key[MAXM]; // [1..keynum]存放关键字
    struct node *parent; //双亲结点指针
    struct node *ptr[MAXM]; //孩子结点指针数组[0..keynum]
} BTNode;
```

2、B树的查找

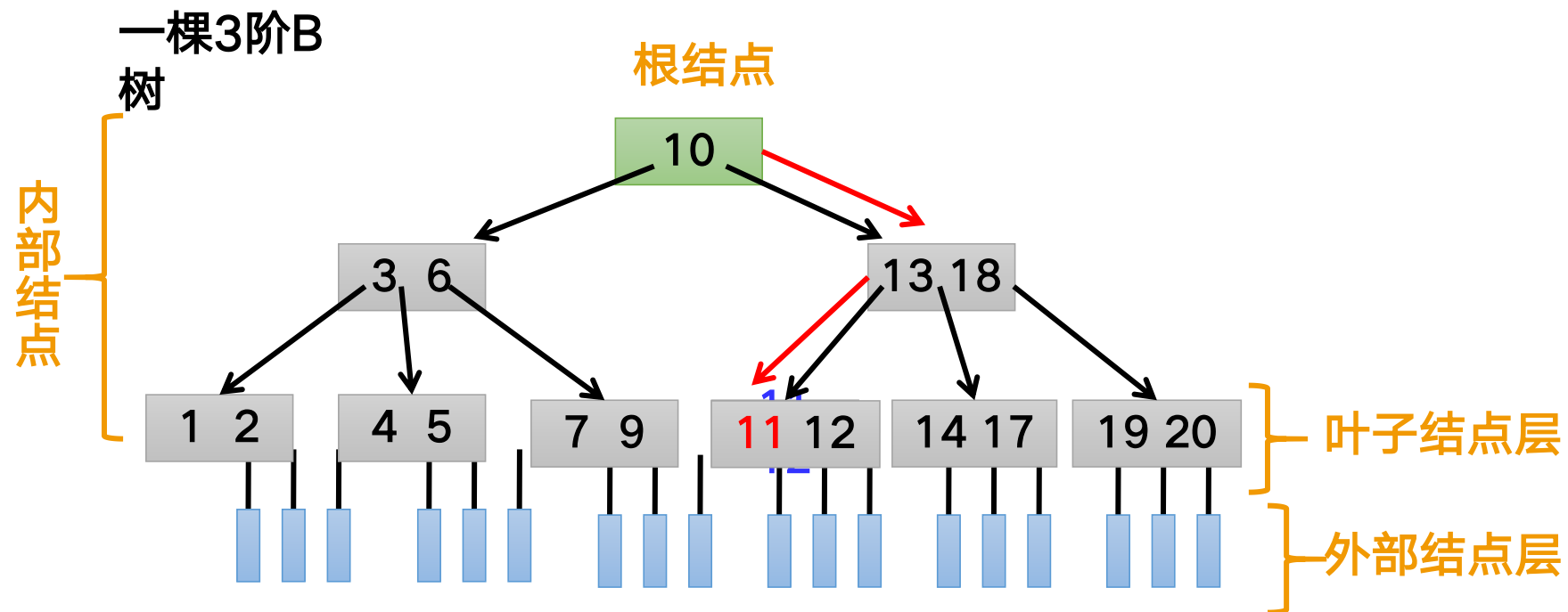
将 k 与根结点中的 $key[i]$ 进行比较：

- ① 若 $k=key[i]$ ，则查找成功；
- ② 若 $k<key[1]$ ，则沿着指针 $ptr[0]$ 所指的子树继续查找；
- ③ 若 $key[i]<k<key[i+1]$ ，则沿着指针 $ptr[i]$ 所指的子树继续查找；
- ④ 若 $k>key[n]$ ，则沿着指针 $ptr[n]$ 所指的子树继续查找。



说明：当查找到某个叶结点时，若相应的指针为空，落入一个外部结点，表示查找失败。

① 成功的查找



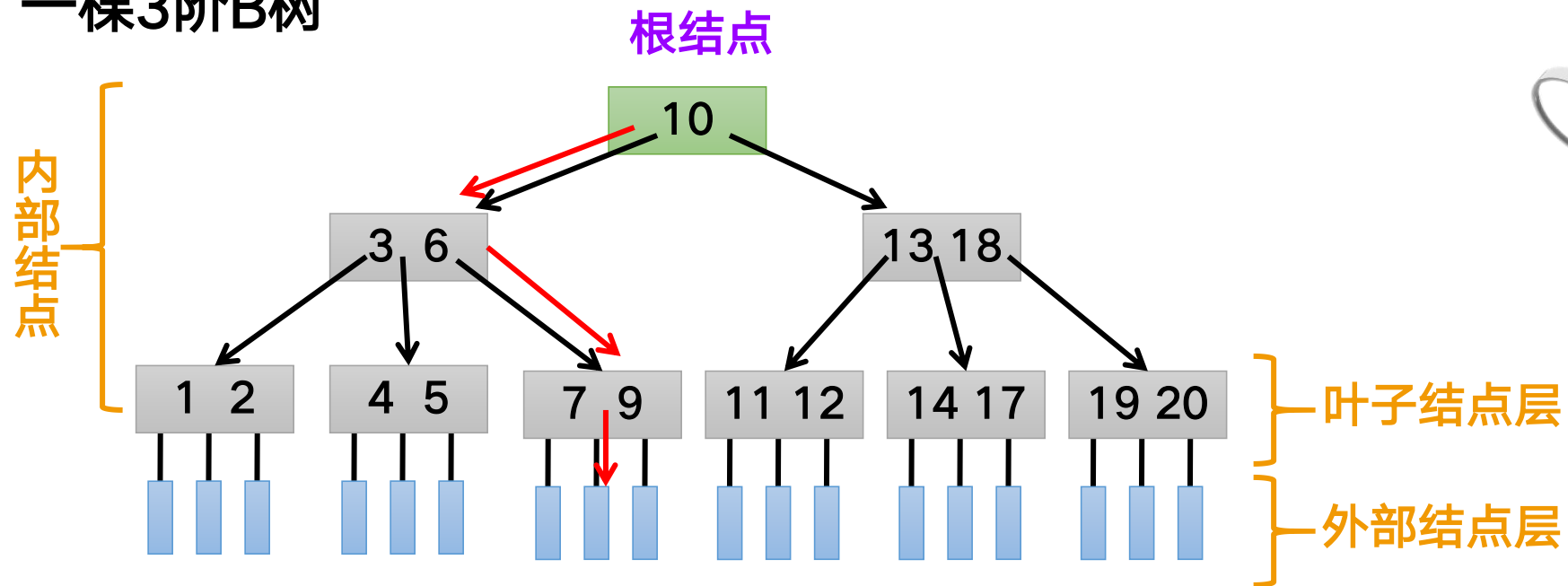
查找关键字11

查找成功



② 不成功的查找

一棵3阶B树



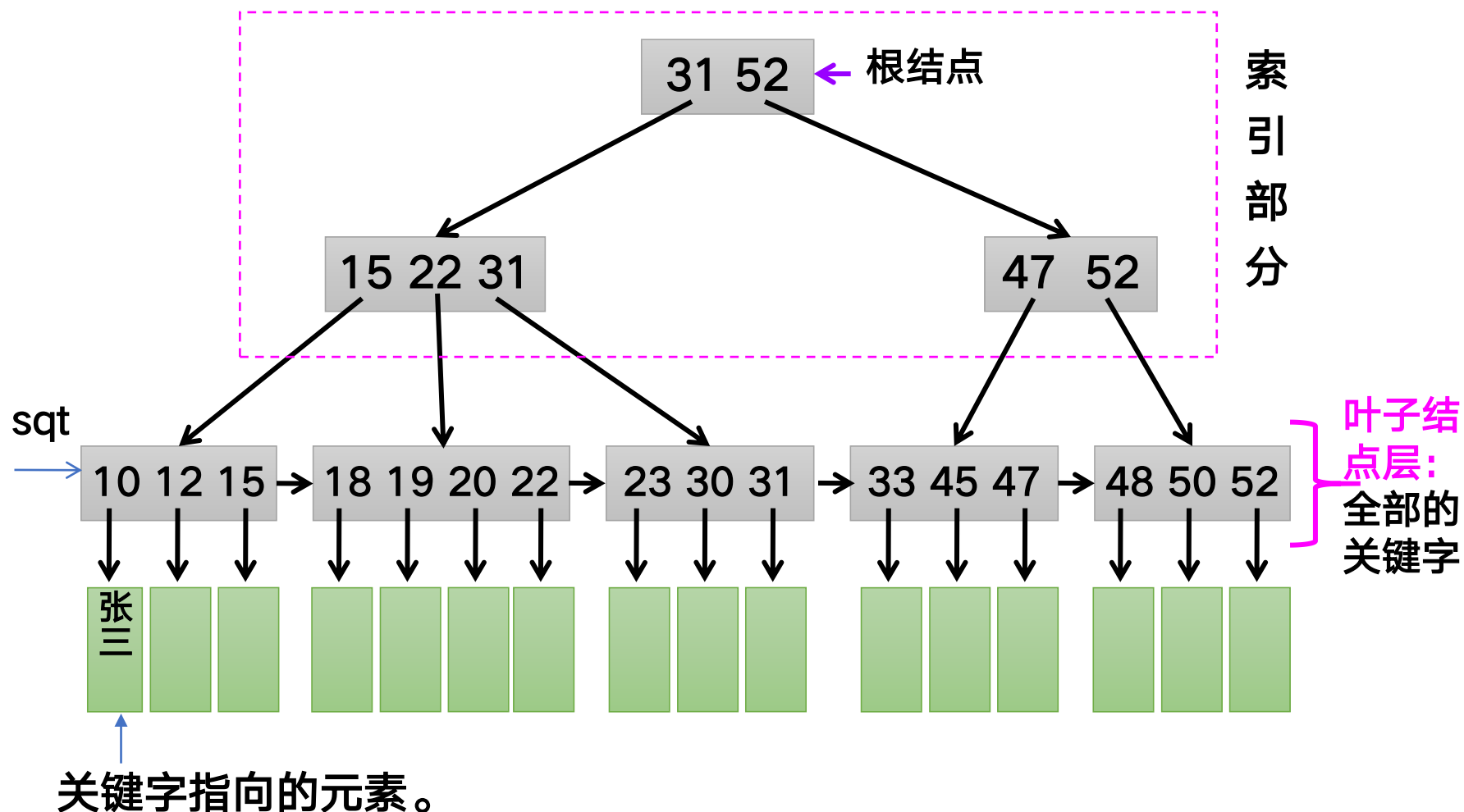
查找关键字8

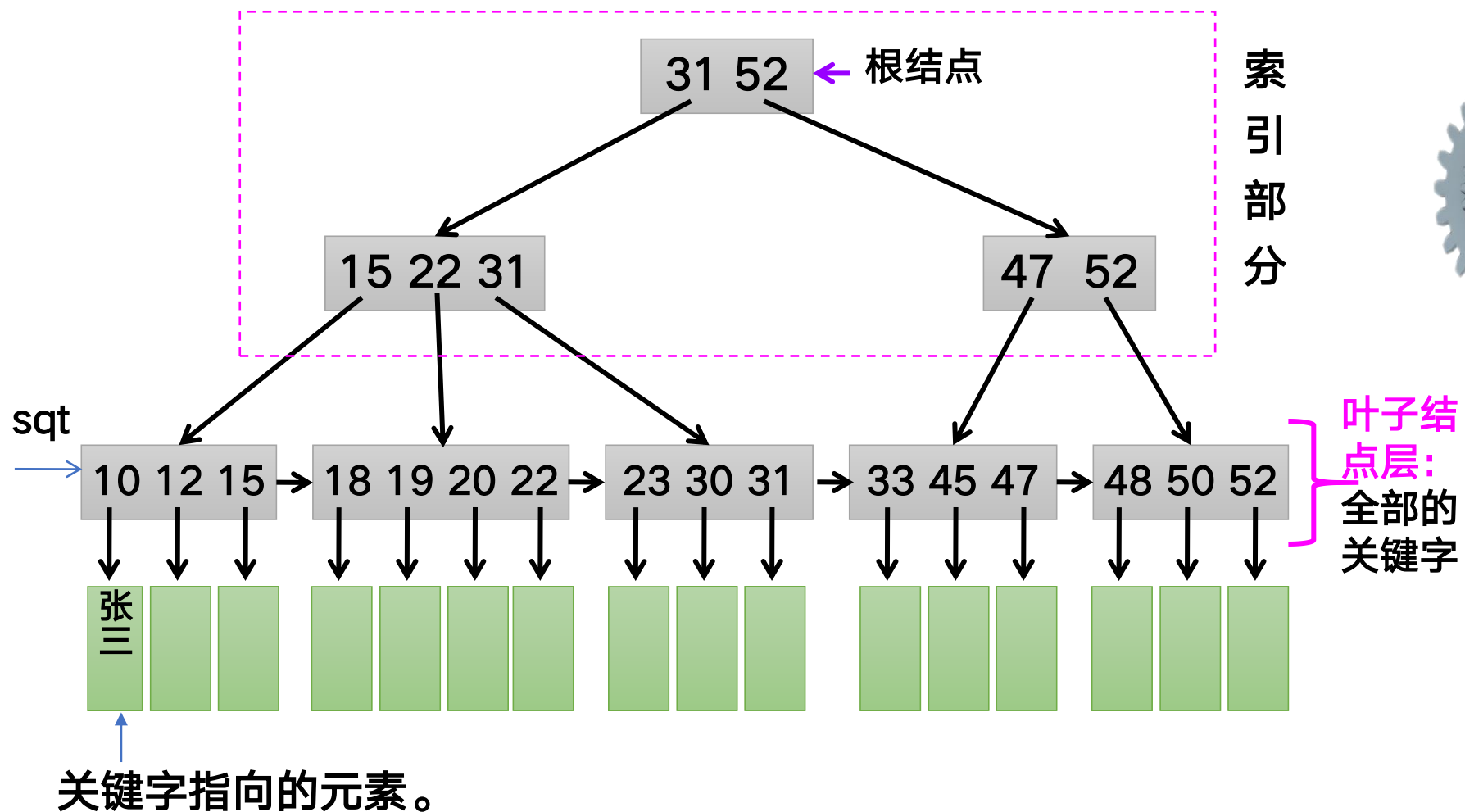
查找失败



9.3.5 B+树

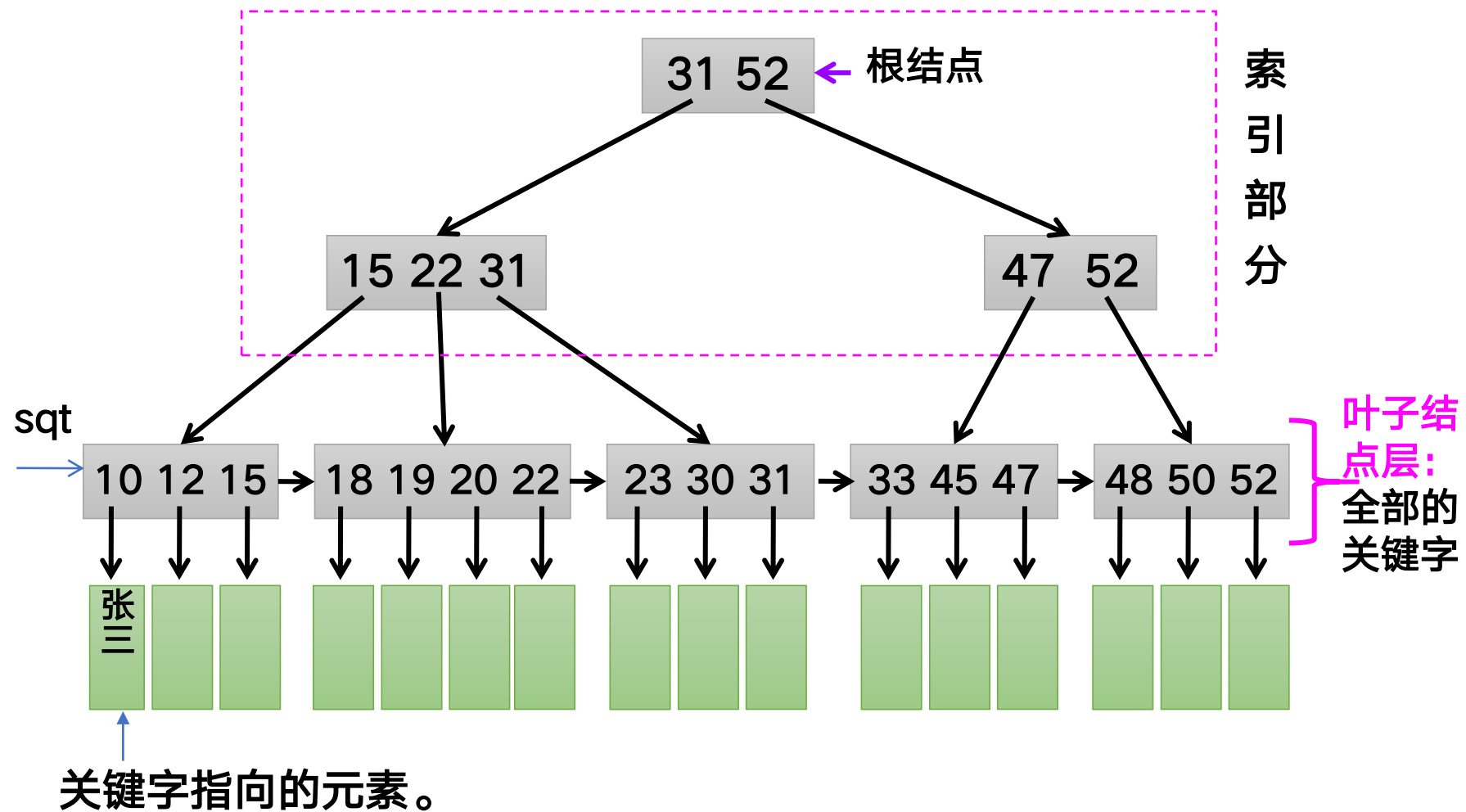
B+树是B树的一种变形。一棵4阶的B+树示例：



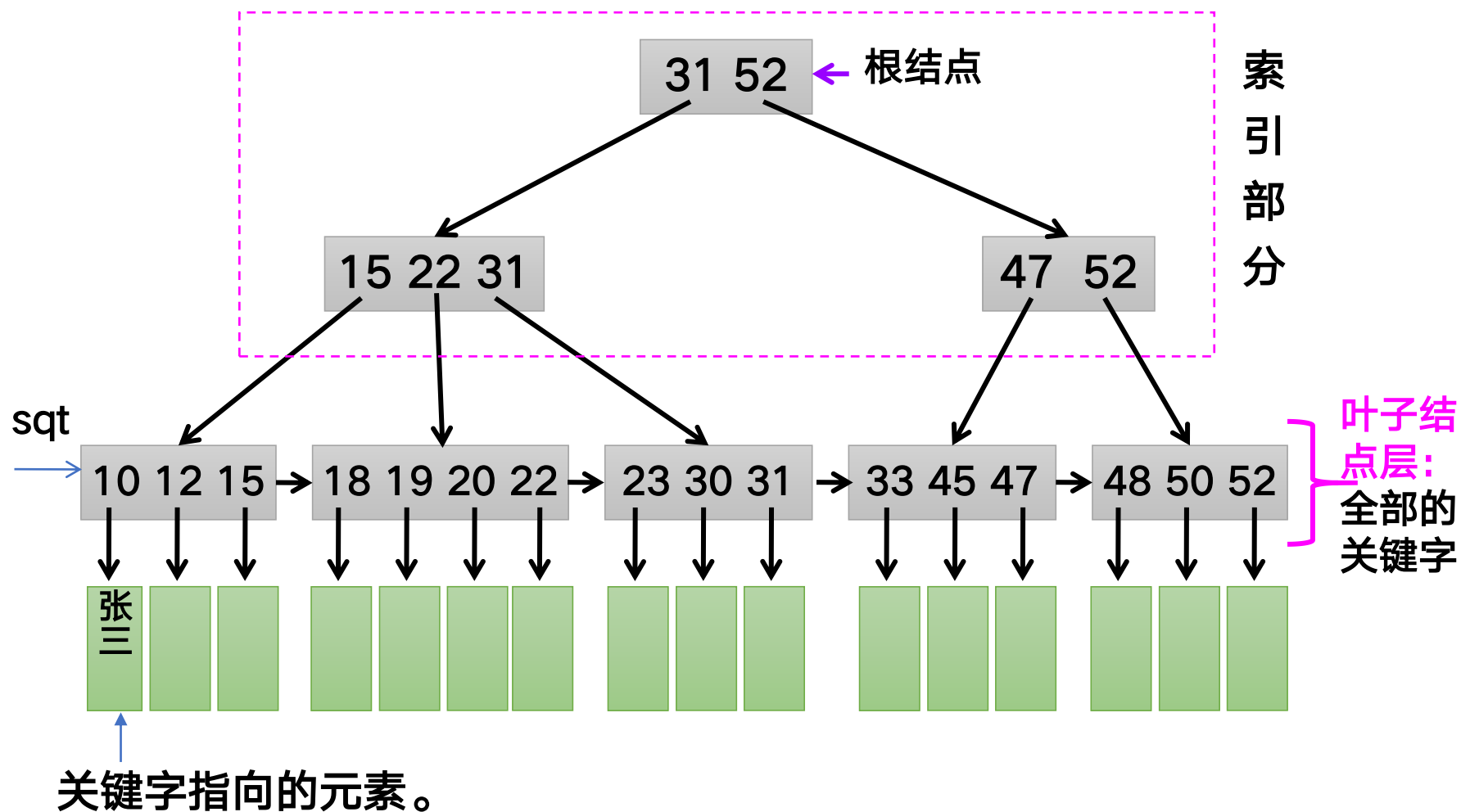


B+树的定义：一棵 m 阶B+树满足下列要求：

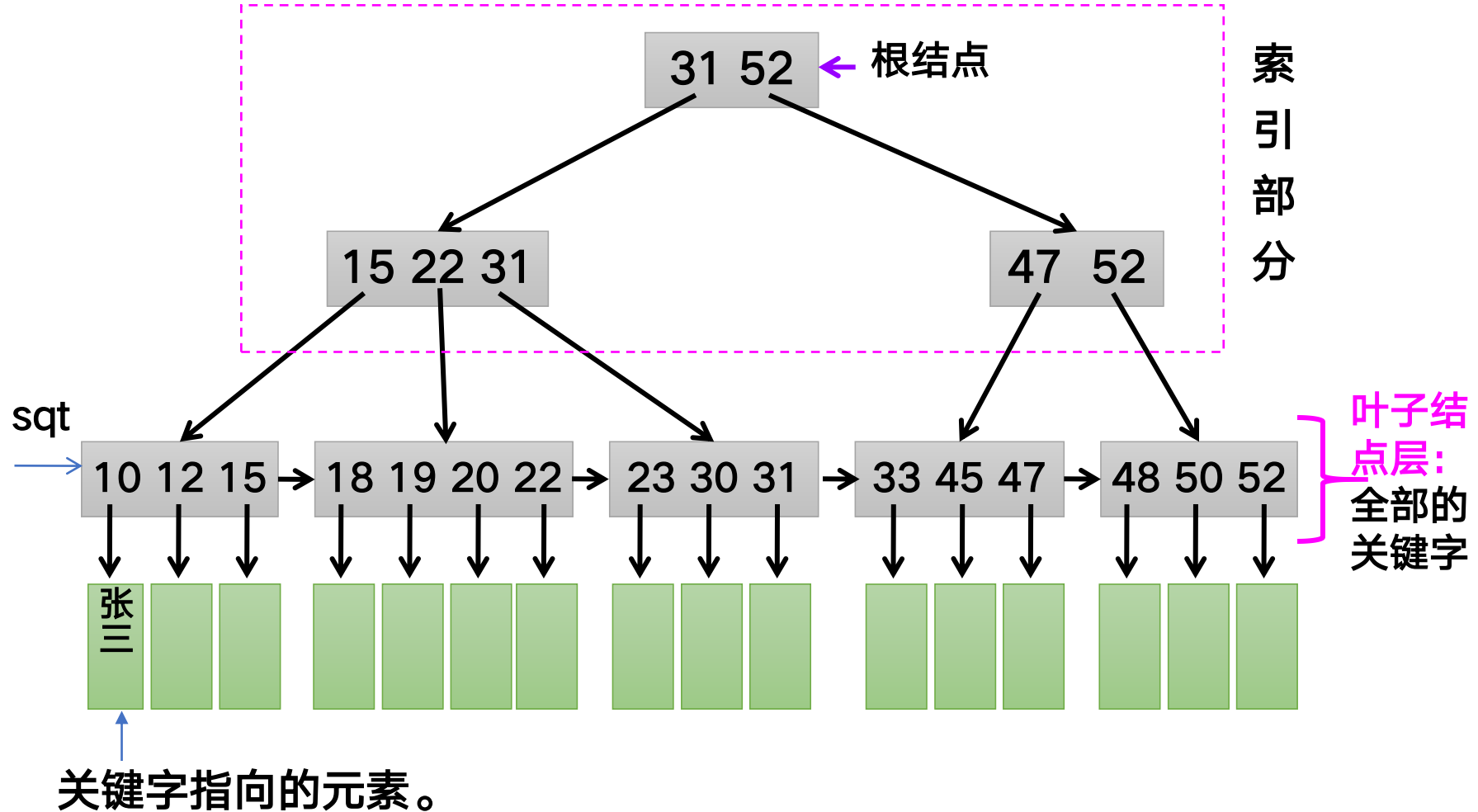
- ❶ 每个分支结点至多有 m 棵子树。这里 $m=4$ 。



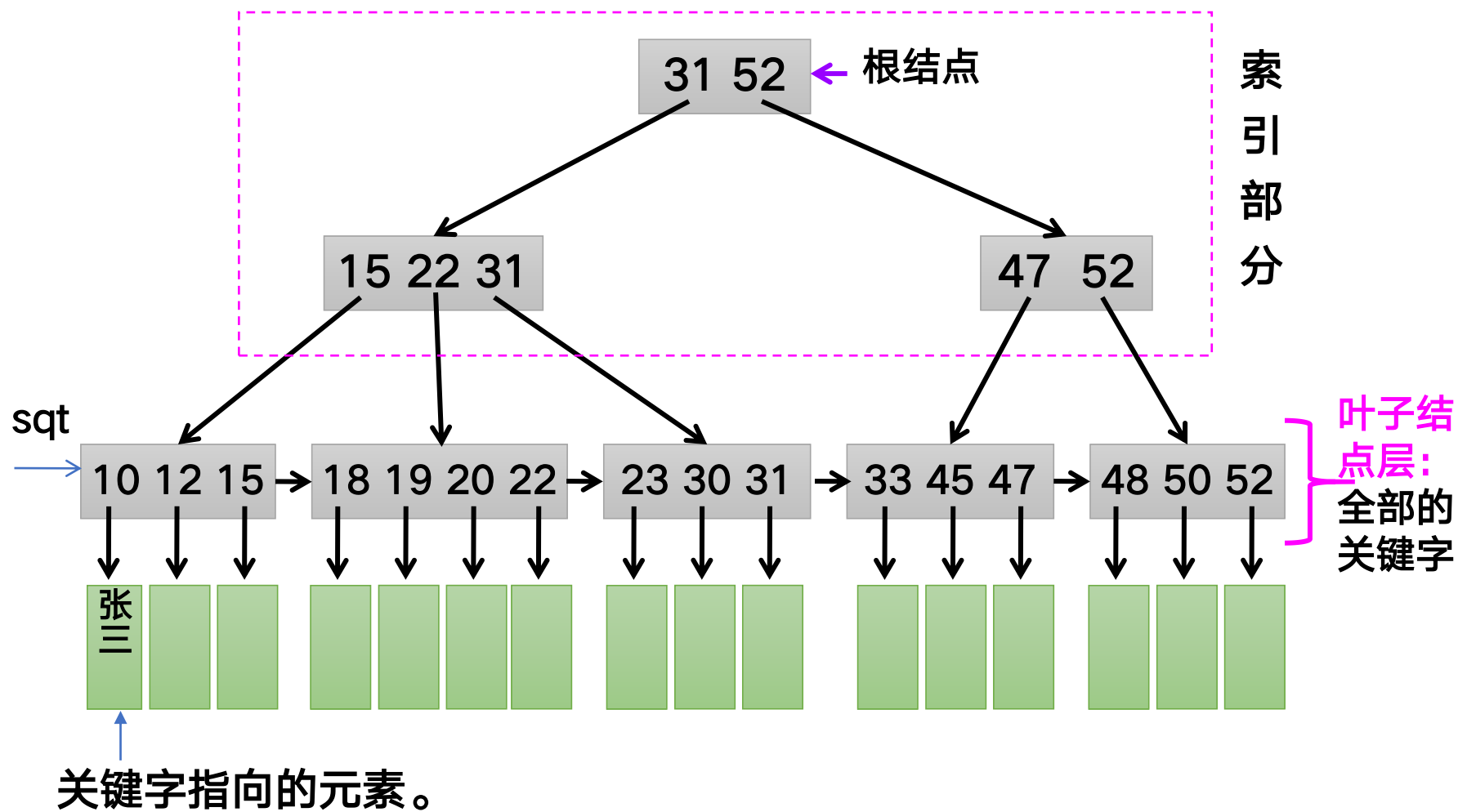
② 根结点或者没有子树，或者至少有两棵子树。



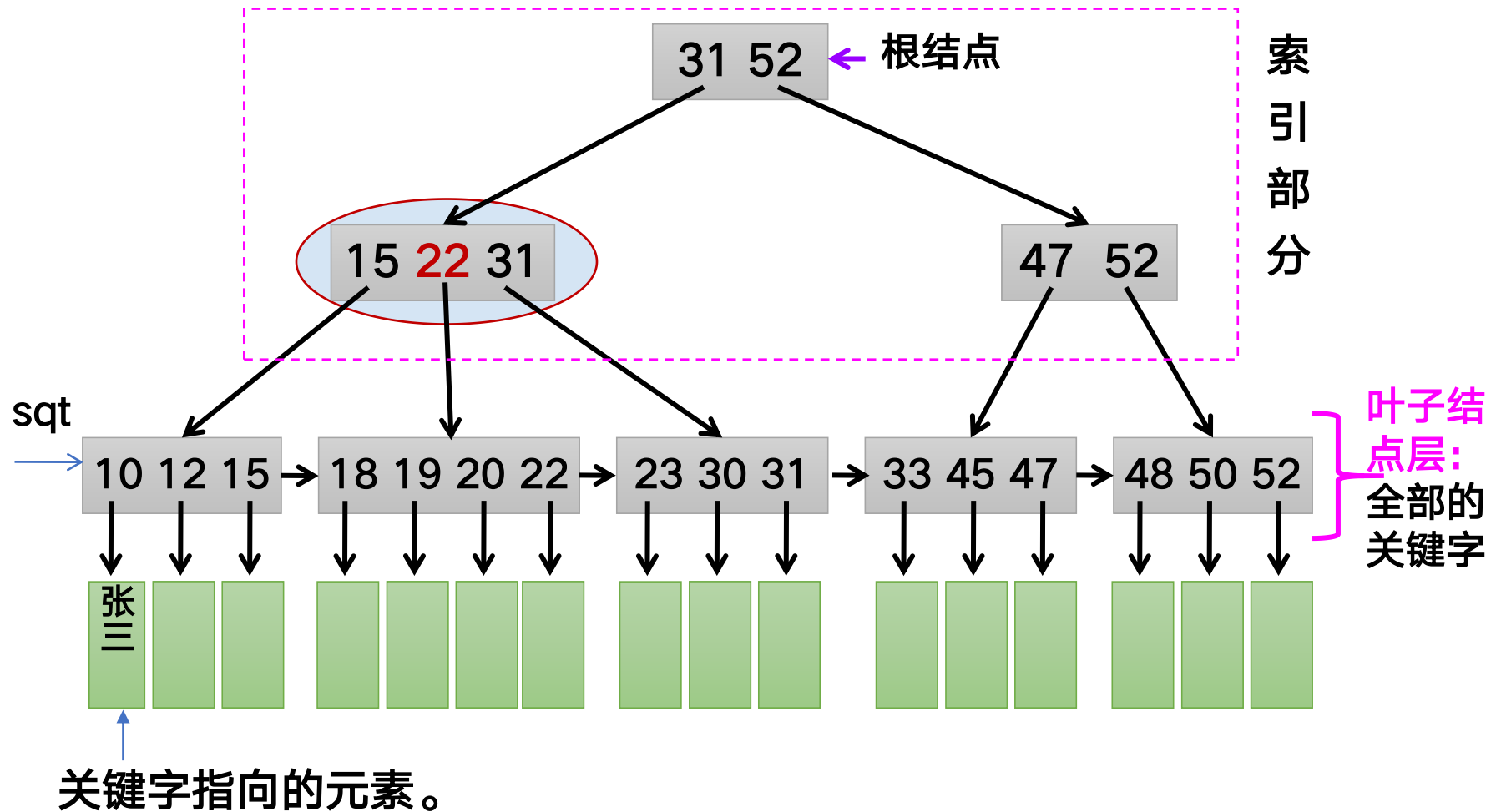
❸ 除根结点外，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树，即至少有2棵子树。



④ 有 n 棵子树的结点恰好有 n 个关键字。子树个数与该结点的关键字个数相同。

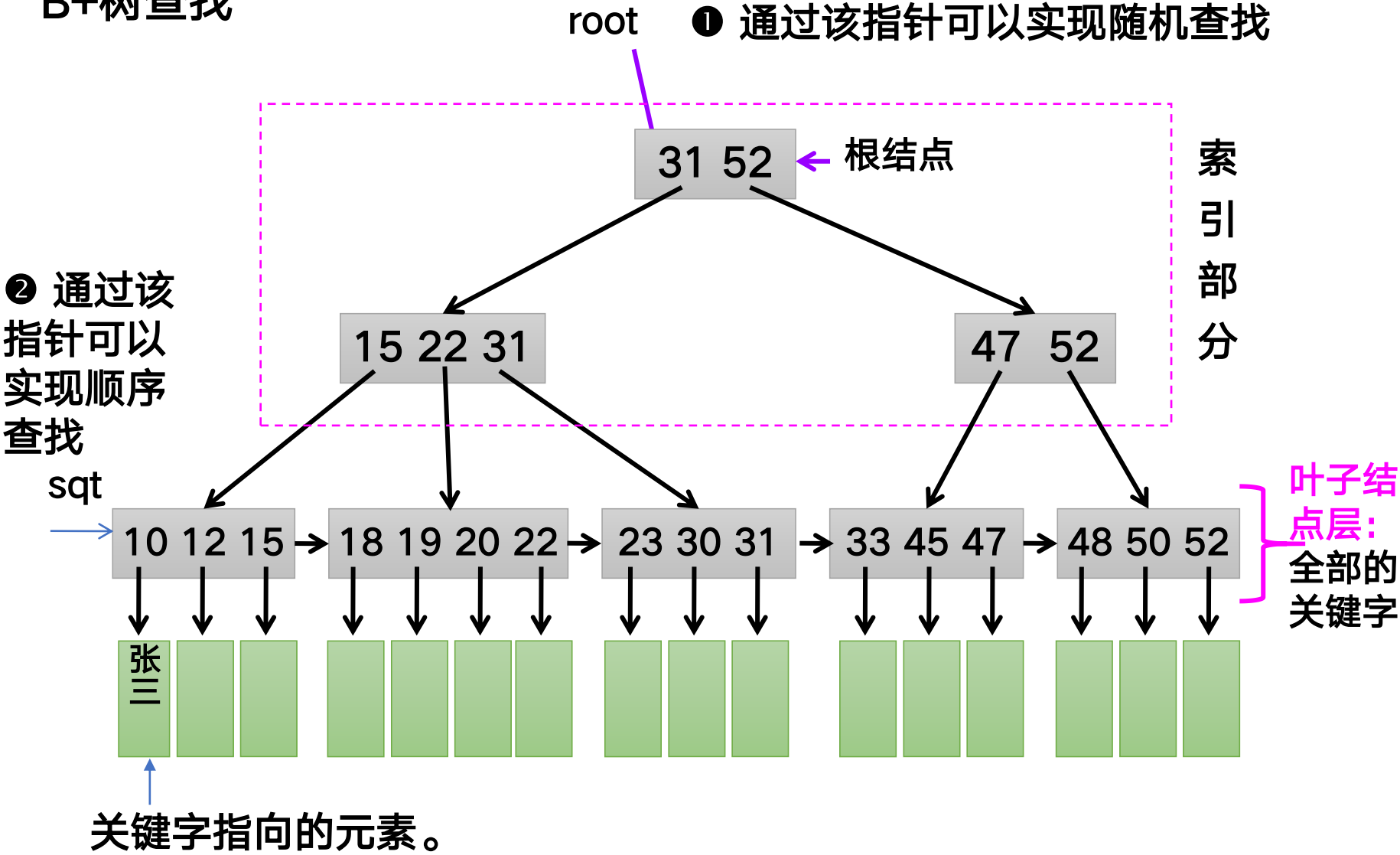


- ⑤ 所有叶子结点包含全部关键字及指向相应元素的指针，而且叶子结点按关键字大小顺序链接。并将所有叶子结点链接起来。

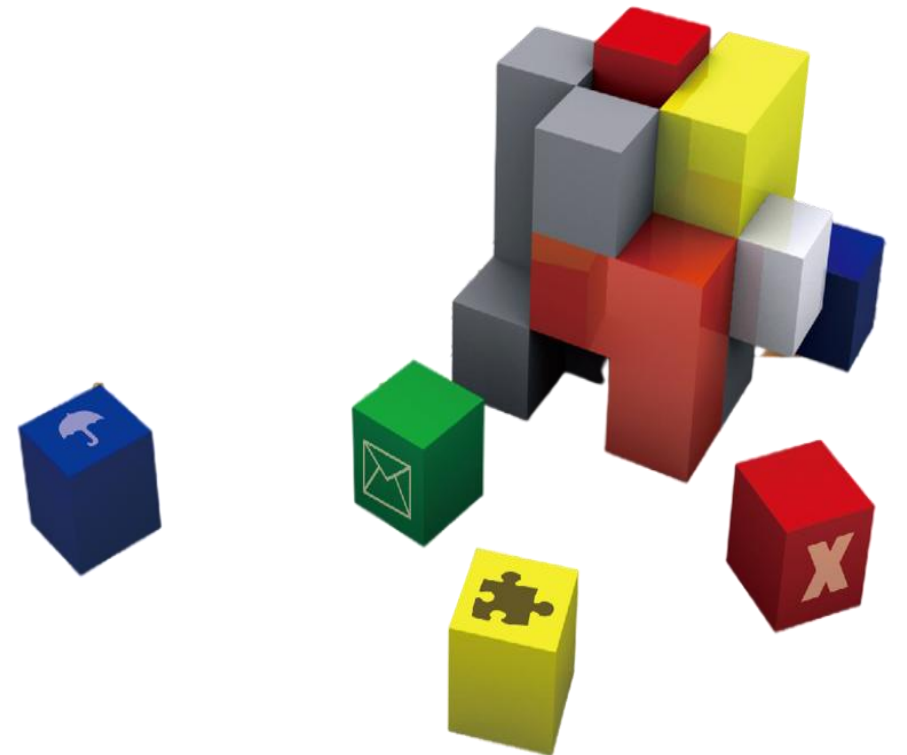


- ⑥ 所有分支结点（可看成是索引的索引）中仅包含它的各个子结点（即下级索引的索引块）中最大关键字及指向子结点的指针。

B+树查找



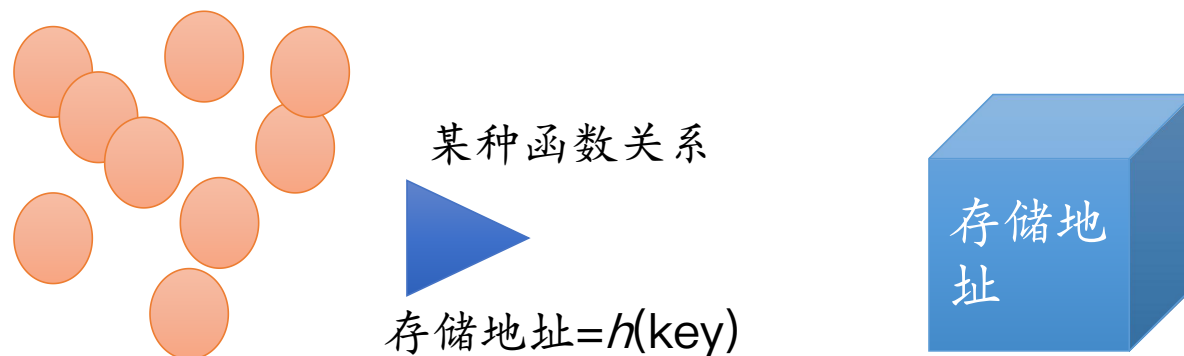
B+树插入和删除操作



9.4 哈希表的查找

9.4.1 哈希表的基本概念

1、哈希表适合情况



注意：哈希表是一种存储结构，它并非适合任何情况，主要适合记录的关键字与存储地址存在某种函数关系的数据。

示例

学号 姓名

201001001 张三
201001003 李四
...
201001025 王五

记录数 $n=20$ ，无序



传统存储方法:存放在一个数组中

0	1	...	19
201001001	201001003	...	201001025
张三	李四	...	王五

20个元素空间

查找学号为201001025的学生姓名:

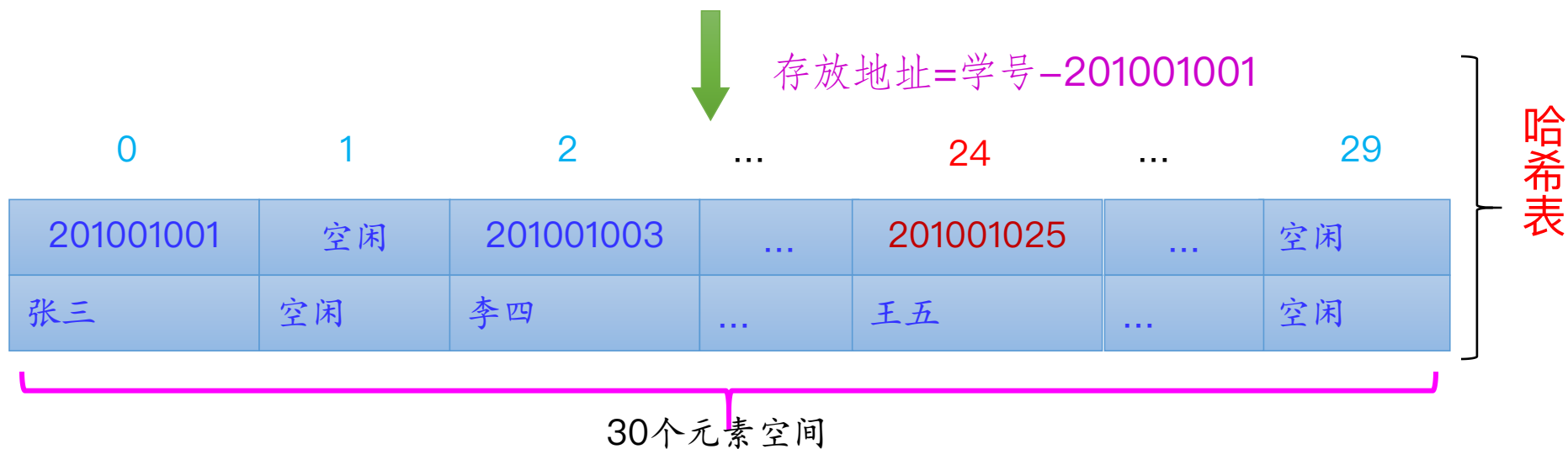
- 从头到尾顺序查找，时间复杂度为 $O(n)$ 。
- 若学号有序，二分查找，时间复杂度为 $O(\log_2 n)$ 。

学号 姓名

另一种存储结构:

201001001	张三
201001003	李四
...	
201001025	王五

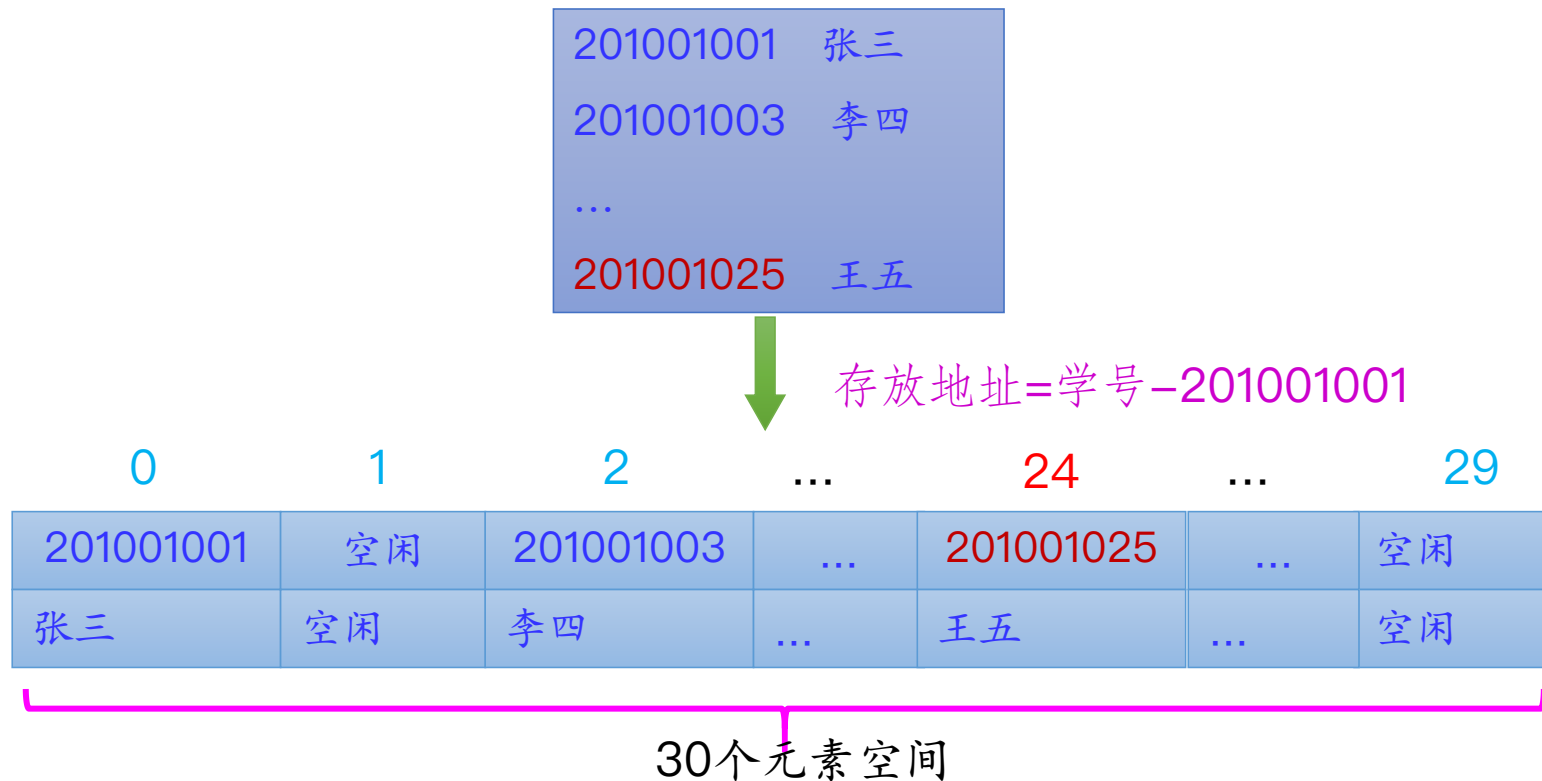
$n=20, m=30$



查找学号为201001025的学生姓名:

- ① 计算:地址 $d = 201001025 - 201001001 = 24$
- ② 和24处的学号比较, 相等, 返回姓名“王五”

时间复杂度 $O(1)$



问题:

(1) 如果学号不连续, 中间空缺很多学号, 会浪费很多空间?

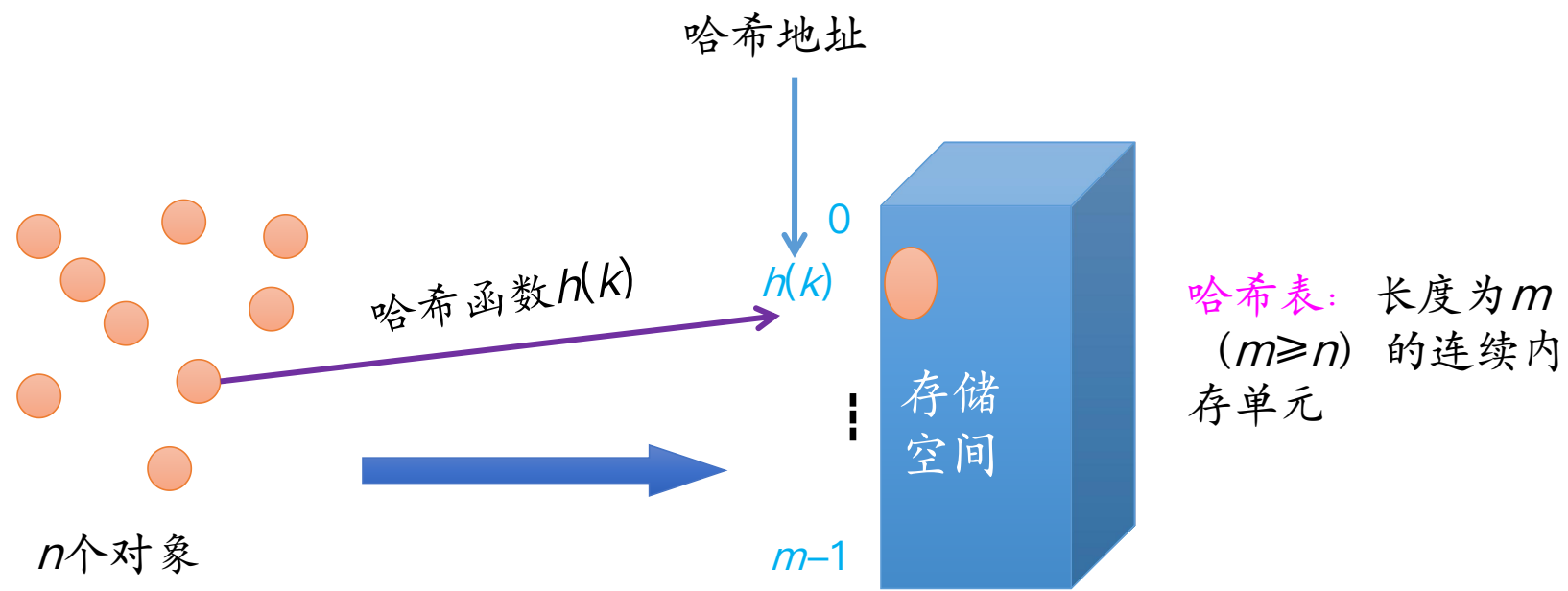
用求模方法解决, 如存放地址=(学号-201001001)% m 。保证存放地址在 $0 \sim m-1$ 中。

(2) 如果出现两个不同学号的元素存放地址相同?

称为冲突。如何解决?

2、几个概念

① 哈希函数和哈希地址



哈希函数: 把关键字为 k 的对象存放在相应的哈希地址中

② 哈希冲突

对于两个关键字分别为 k_i 和 k_j ($i \neq j$) 的记录, 有 $k_i \neq k_j$, 但 $h(k_i) = h(k_j)$ 。把这种现象叫做哈希冲突 (同义词冲突)。

在哈希表存储结构的存储中, 哈希冲突是很难避免的!!!



哈希表中出现同义词冲突是指（ ）。

- A. 两个元素具有相同的序号
- B. 两个元素的关键字不同，而其他属性相同
- C. 数据元素过多
- D. 两个元素的关键字不同，而对应的哈希函数值（存储地址）相同

答案：D。

3、哈希表设计

哈希表设计主要需要解决哈希冲突。实际中哈希冲突是难以避免的，主要与3个因素有关：

- 与装填因子有关。装填因子 α =存储的记录个数/哈希表的大小= n/m $\Rightarrow$ α 越小，冲突的可能性就越小； α 越大（最大可取1），冲突的可能性就越大。通常使最终的控制控制在0.6~0.9的范围内。
- 与所采用的哈希函数有关。好的哈希函数会减少冲突的发生；不好的哈希函数会增加冲突的发生。
- 与解决冲突方法有关。好的哈希冲突解决方法会减少冲突的发生。

所以哈希表设计的重点:

- 尽可能设计好的哈希函数
- 设计解决冲突的方法。

9.4.2 哈希函数构造方法

1、直接定址法

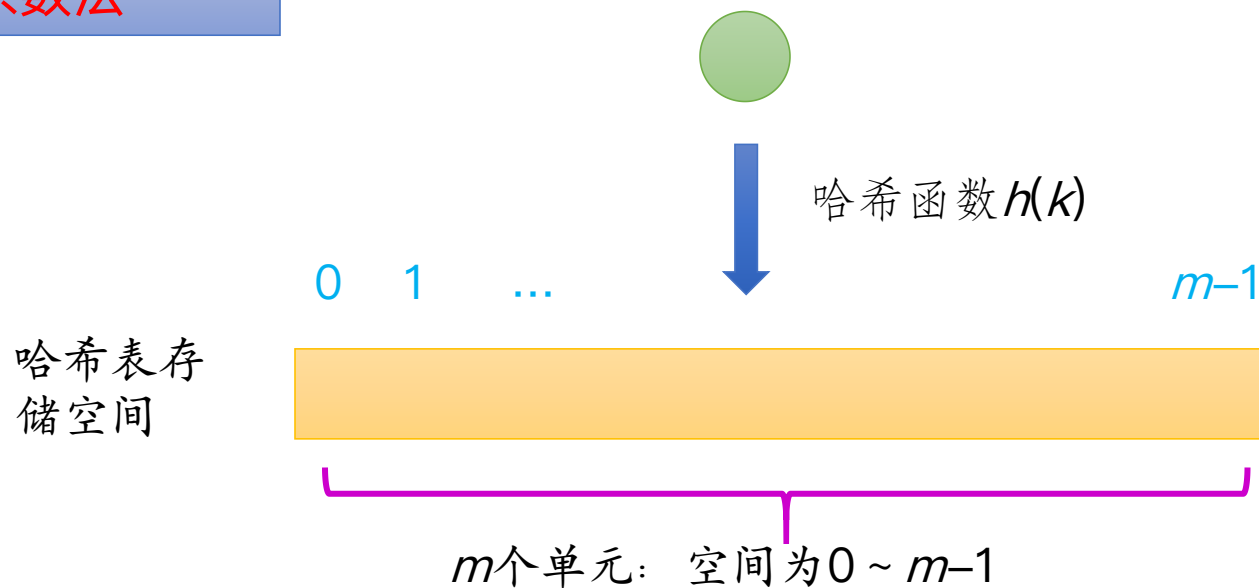
直接定址法是以关键字 k 本身或关键字加上某个数值常量 c 作为哈希地址的方法。
直接定址法的哈希函数 $h(k)$ 为：

$$h(k) = k + c$$

例如：

$$h(\text{学号}) = \text{学号} - 201001001$$

2、除留余数法



除留余数法的哈希函数 $h(k)$ 为:

$$h(k) = k \bmod p \quad (\text{mod为求余运算, } p \leq m)$$

p 最好是质数（素数）。

除留余数法就是把 n 个记录按关键字映射到 $0 \sim m-1$ 的哈希空间中。而模 p （素数）时出现冲突的可能性更小。

3、数字分析法

关键字

9	2	3	1	7	6	0	2
9	2	3	2	6	8	7	5
9	2	7	3	9	6	2	8
9	2	3	4	3	6	3	4
9	2	7	0	6	8	1	6
9	2	7	7	4	6	3	8
9	2	3	8	1	2	6	2
9	2	3	9	4	2	2	0

为取
哈希最后
地址两位作

0	2
7	5
2	8
3	4
1	6
3	8
6	2
2	0

哈希地址的集合为(2, 75, 28, 34, 16, 38, 62, 20)。

大数值范围

哈希函数 $h(k)$

小数值范围



示例

下列关于哈希函数的说法正确的是（ ）。

- A. 哈希函数越复杂越好
- B. 哈希函数越简单越好
- C. 用除余法构造的哈希函数是最好的
- D. 在冲突尽可能少的情况下，哈希函数越简单越好

答：D。

【例9-9】 假设哈希表长度 $m=13$ ，采用除留余数法哈希函数建立如下关键字集合的哈希表(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)，共11个关键字。

解： $n=11$, $m=13$ ，设计除留余数法的哈希函数为：

$$h(k)=k \bmod p$$

p 应为小于等于 m 的素数，设 $p=13$ 。

关键字: 16 74 60 43 54 90 46 31 29 88 77



$$h(29)=3$$

0	1	2	3	4	5	6	7	8	9	10	11	12
		54	16	43	31		46	60	74			90

注意: 存在哈希冲突。

9.4.3 哈希冲突解决方法

1、开放定址法

开放定址法：冲突时找一个新的空闲的哈希地址。



怎么找空闲单元？

(1) 线性探测法

线性探测法的数学递推描述公式为：

$$d_0 = h(k)$$

$$d_i = (d_{i-1} + 1) \bmod m \quad (1 \leq i \leq m-1)$$

非同义词冲突：哈希函数值不相同的两个记录争夺同一个后继哈希地址 $\Rightarrow$
堆积（或聚集）现象。

(2) 平方探测法

平方探测法的数学描述公式为：

$$d_0 = h(k)$$

$$d_i = (d_0 \pm i^2) \bmod m \quad (1 \leq i \leq m-1)$$

查找的位置依次为： d_0 、 d_0+1 、 d_0-1 、 d_0+4 、 d_0-4 、 ...

平方探测法是一种较好的处理冲突的方法，可以避免出现堆积现象。它的缺点是不能探测到哈希表上的所有单元，但至少能探测到一半单元。

【例9-10】 假设哈希表长度 $m=13$ ，采用除留余数法哈希函数建立如下关键字集合的哈希表：

(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)。

并采用线性探测法解决冲突。

关键字: 16 74 60 43 54 90 46 31 29 88 77

$h(29)=3$ $\Rightarrow$ 冲突

$d_0=3, d_1=(3+1) \% 13=4$ $\Rightarrow$ 仍冲突

$d_2=(4+1) \% 13=5$ $\Rightarrow$ 仍冲突

$d_3=(5+1) \% 13=6$ $\Rightarrow$ OK

0	1	2	3	4	5	6	7	8	9	10	11	12
77		54	16	43	31	29	46	60	74			90

共探测4次

关键字: 16 74 60 43 54 90 46 31 29 88 77

$h(88)=10$



0	1	2	3	4	5	6	7	8	9	10	11	12
77		54	16	43	31	29	46	60	74	88		90

共探测1次

关键字: 16 74 60 43 54 90 46 31 29 88 77

$h(77)=12$

⇒ 冲突



$d_0=12, d_1=(12+1) \% 13=0$

⇒ OK

0	1	2	3	4	5	6	7	8	9	10	11	12
77		54	16	43	31	29	46	60	74	88		90

共探测2次

哈希表创建完毕

最终的哈希表

哈希表ha[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1



哈希表的构成:

- 哈希函数: 本例为 $h(k)=k \bmod 13$
- 解决冲突方法: 本例为线性探测法

开放定址法哈希表查找 k 过程:

```
d=h(k);  
while (ha[d]!=空 && ha[d]!=k)  
    d=采用某种探测法求出下一地址;  
if (ha[d]==空)  
    return 失败标记;  
else  
    return ha[d];
```

成功查找的情况

查找关键字为29的记录:

$h(29)=29\%13=3$: $16 \neq 29$;

$d_0=3$, $d_1=(3+1)=4$: $43 \neq 29$;

$d_2=(4+1)=5$: $31 \neq 29$;

$d_3=(5+1)=6$: $29 = 29$ 。成功!

需要4次关键字比较

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

哈希表成功查找完毕

❶ 对于前面构建的哈希表：成功查找ASL_{成功}计算

哈希表ha[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
<i>k</i>	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

探测次数恰好等于查找到该记录所需要的关键字比较次数！



$$\text{ASL}_{\text{成功}} = \frac{2+1+1+1+1+4+1+1+1+1+1}{11} = 1.364$$

不成功查找的情况

查找关键字 $x=47$ 的记录

$h(47)=47\%13=8:$ $\Rightarrow 60 \neq 47$

$d_0=8, d_1=(8+1)=9:$ $\Rightarrow 74 \neq 47$

$d_2=(9+1)=10:$ $\Rightarrow 88 \neq 47$

$d_3=(10+1)=11:$ 此处为 空, 查找失败!

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
k	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

需要 4 次关键字比较

哈希表失败查找完毕

② 对于前面构建的哈希表：不成功查找ASL_{不成功}计算

哈希表ha[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
<i>k</i>	77		54	16	43	31	29	46	60	74	88		90
探测次数	2	1	10	9	8	7	6	5	4	3	2	1	3

↑

↑

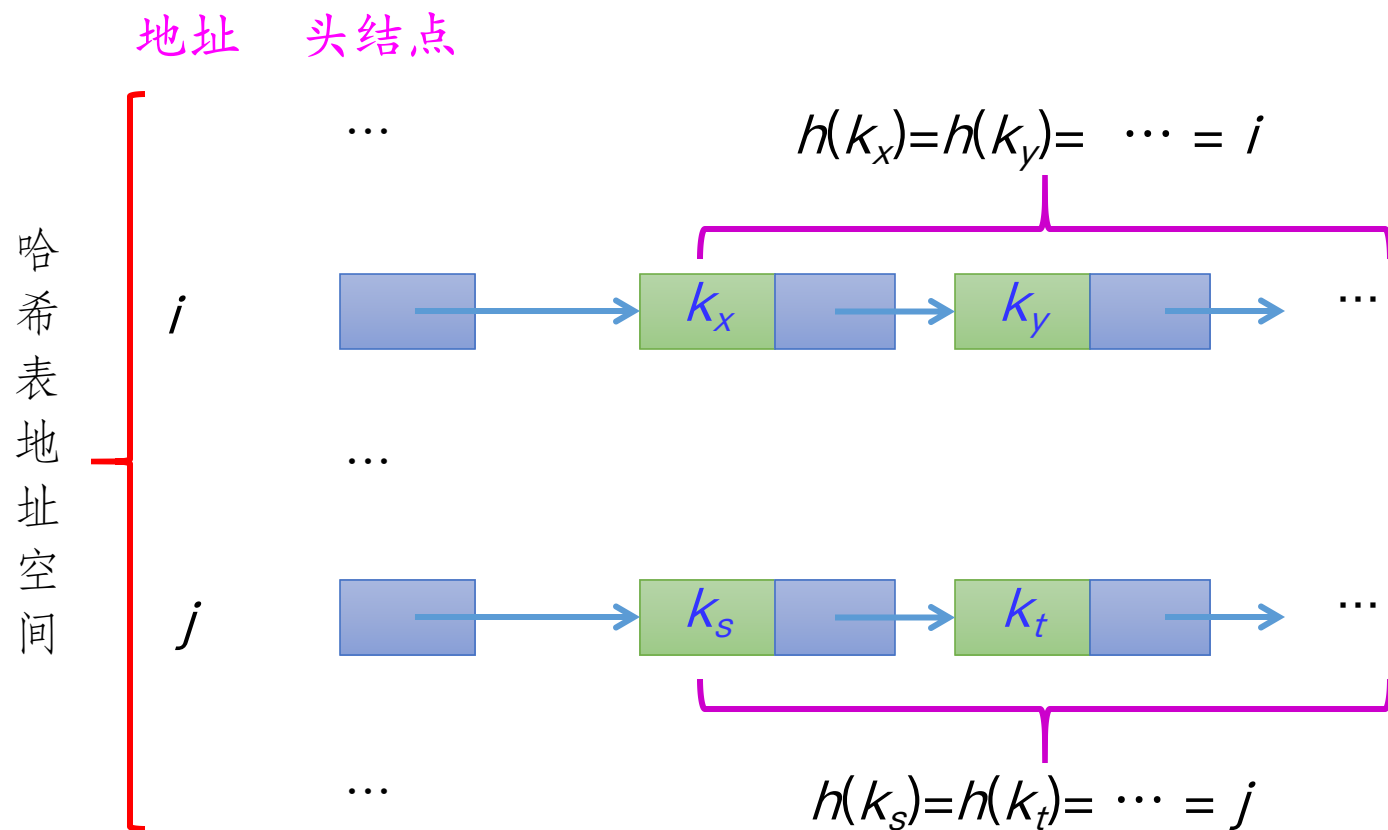
一个关键字不在哈希表中，探测次数为10，探测到空位置，需要10次关键字比较
 一个关键字不在哈希表中，探测次数为2，探测到空位置，需要2次关键字比较

↓

$$ASL_{\text{不成功}} = \frac{2+1+10+9+8+7+6+5+4+3+2+1+3}{13} = 4.692$$

2、拉链法

拉链法是把所有的同义词用单链表链接起来的方法。



【例9-10】对例9-9的关键字序列，构造采用拉链法解决冲突的哈希表。

关键字集合:

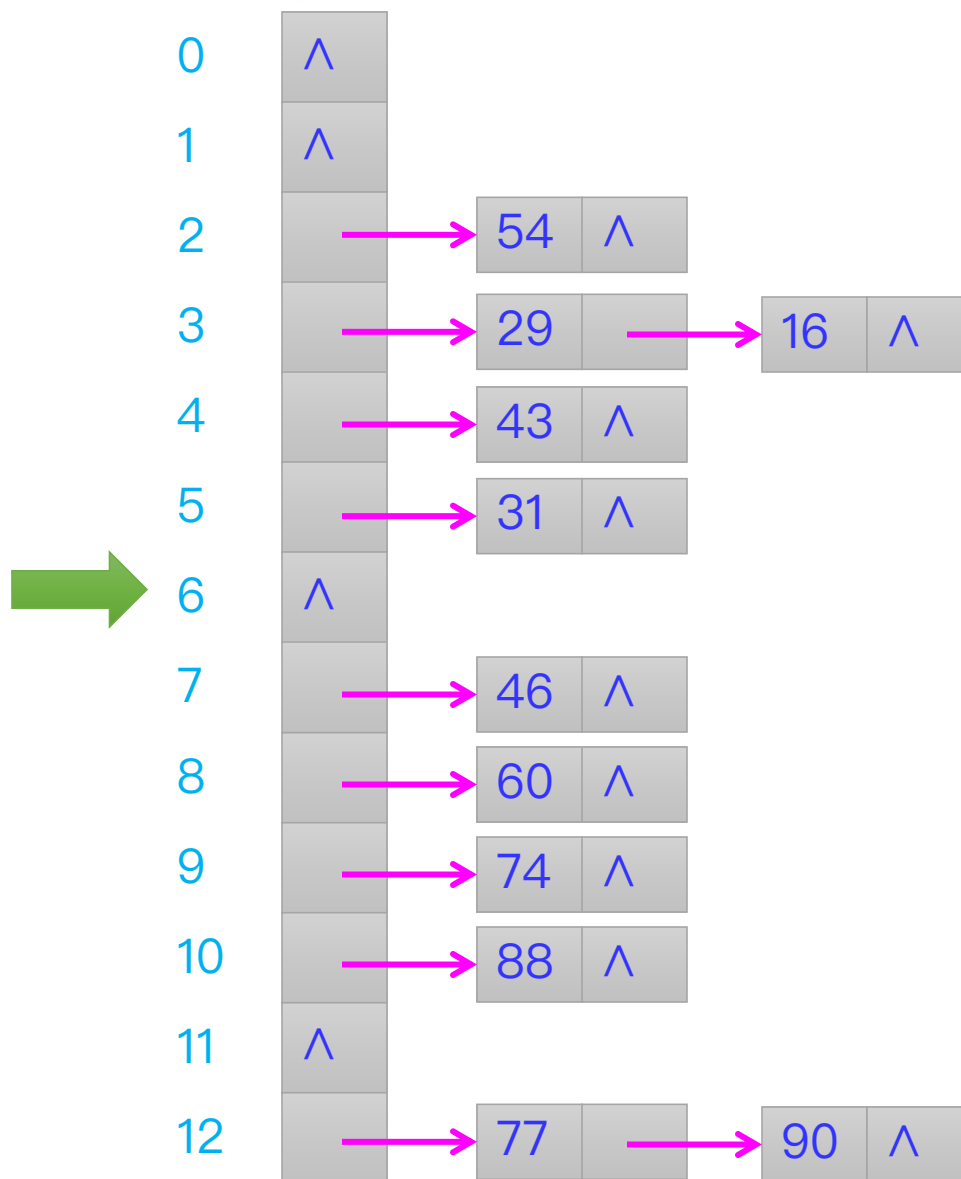
(16, 74, 60,

43, 54, 90,

46, 31, 29,

88, 77)

采用拉链法构造的哈希表 $ha[0..12]$



拉链法哈希表查找 k 过程:

```
d=h(k);  
p=ha[d];  
while (p!=NULL && p->key!=k)  
    p=p->next; //在ha[d]的单链表中查找  
if (p==NULL)  
    return 失败标记;  
else  
    return p所指结点;
```

成功查找的情况

查找关键字为90的记录:

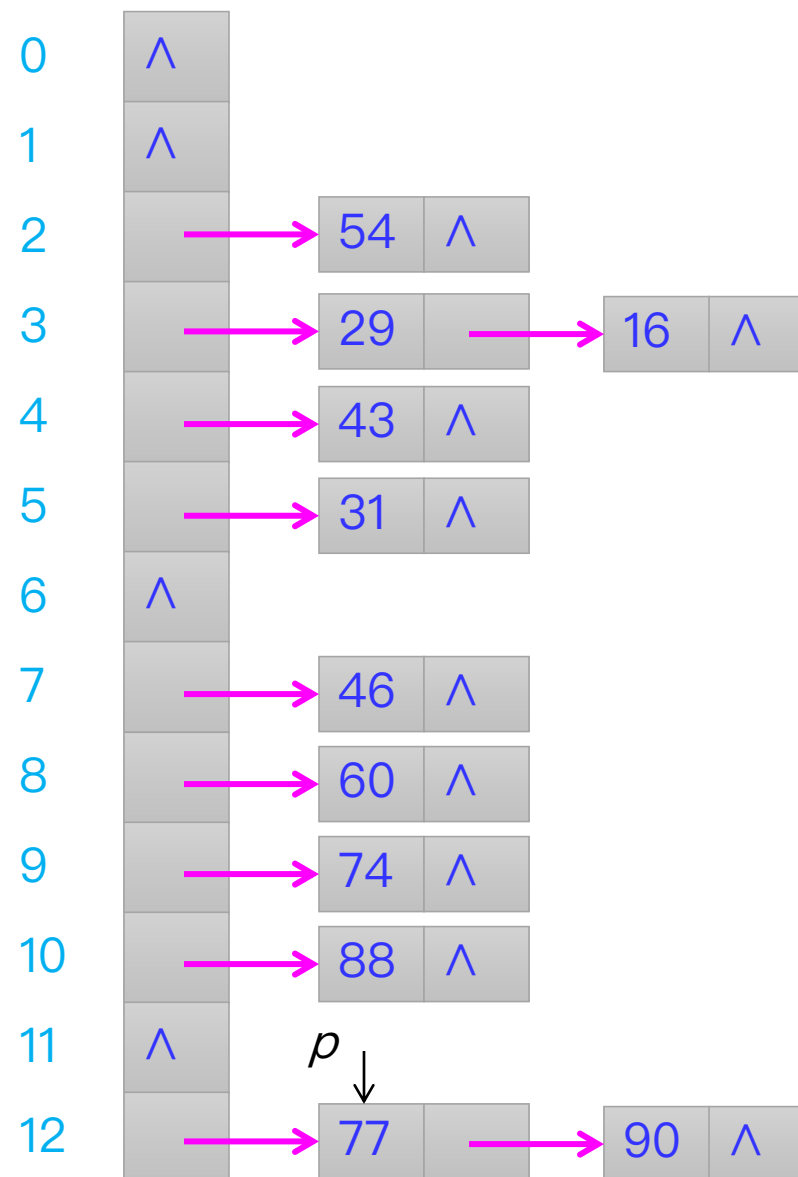
$$h(90)=90\%13=12$$

p 指向 $ha[12]$ 的第1个结点, $77 \neq 90$;

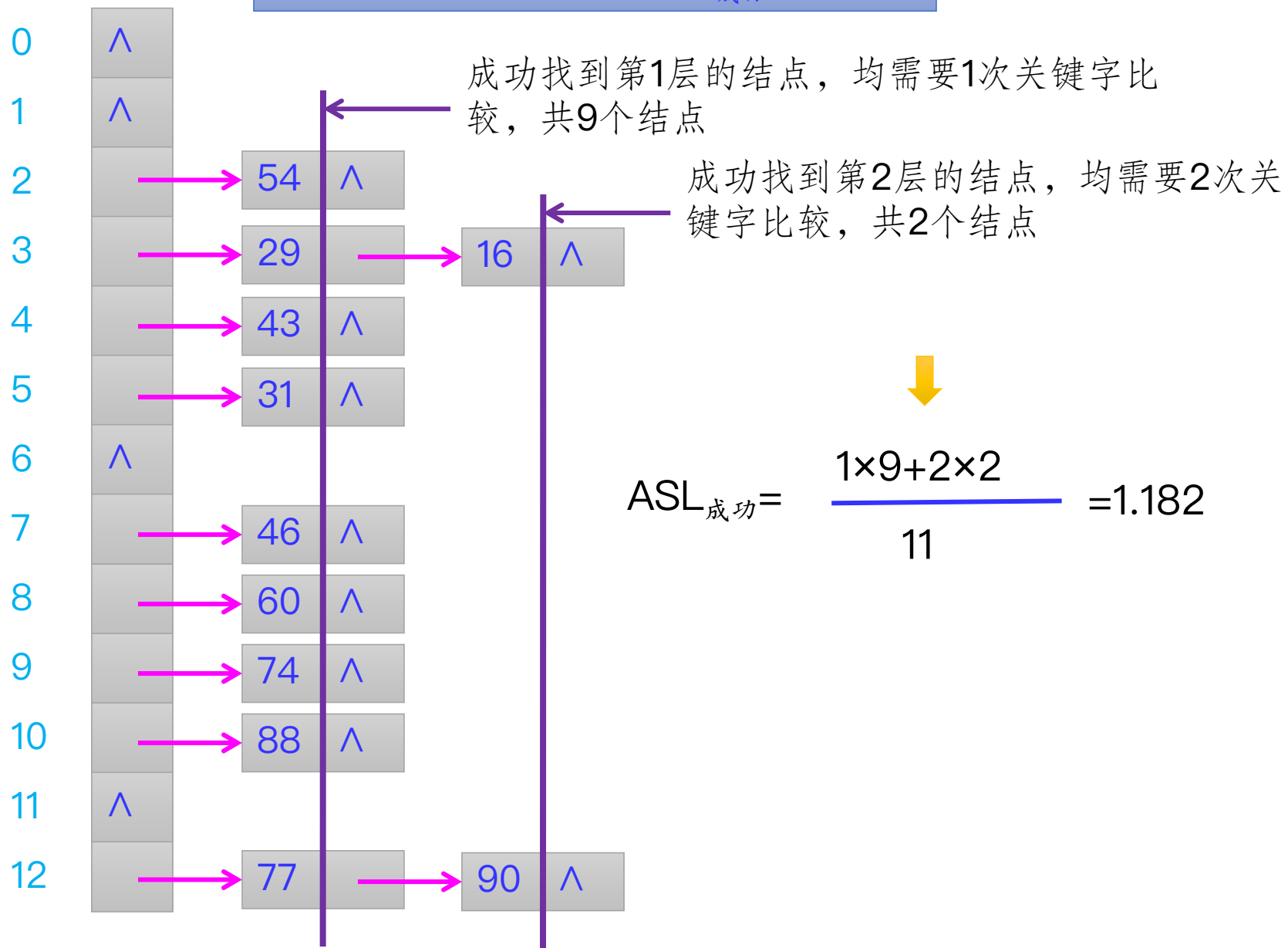
p 指向 $ha[12]$ 的第2个结点, $90=90$ 。成功!

2次关键字比较

哈希表成功查找完毕



① 拉链法中成功查找的ASL_{成功}计算



不成功查找的情况

查找关键字为59的记录:

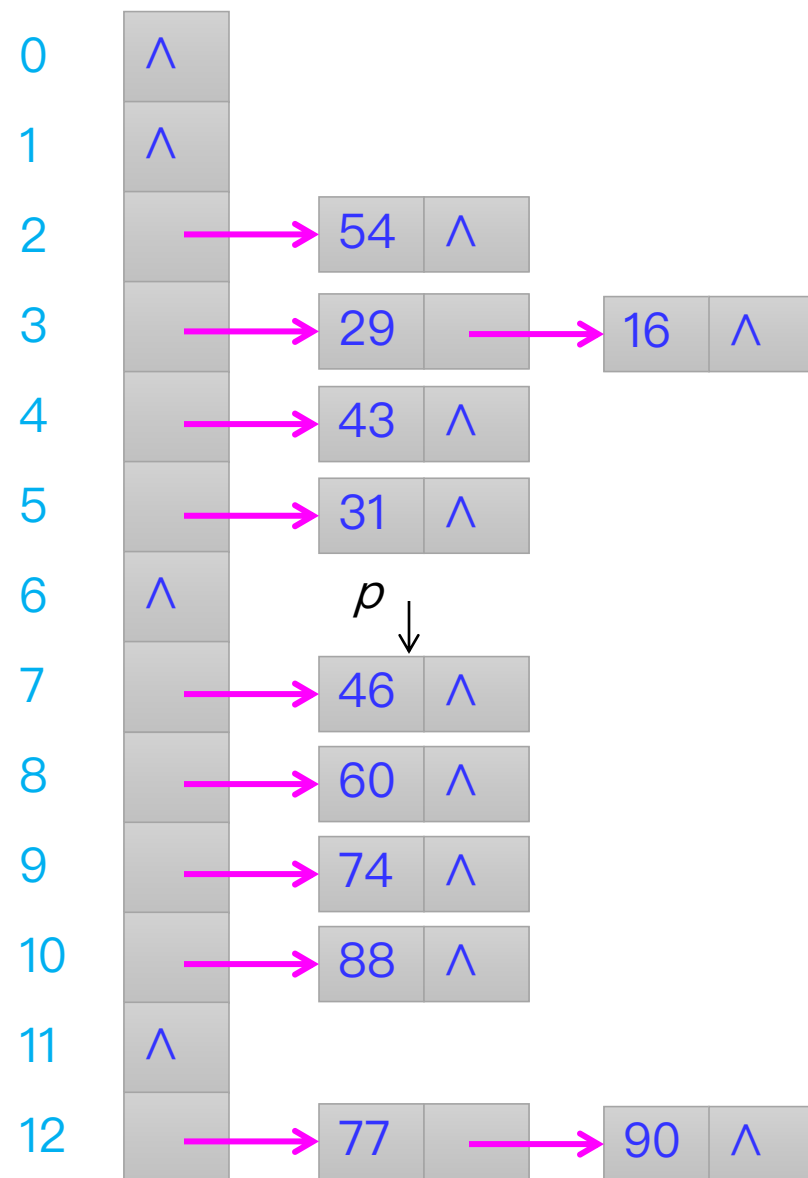
$$h(59)=59\%13=7$$

p 指向 $ha[7]$ 的第1个结点, $46 \neq 59$;

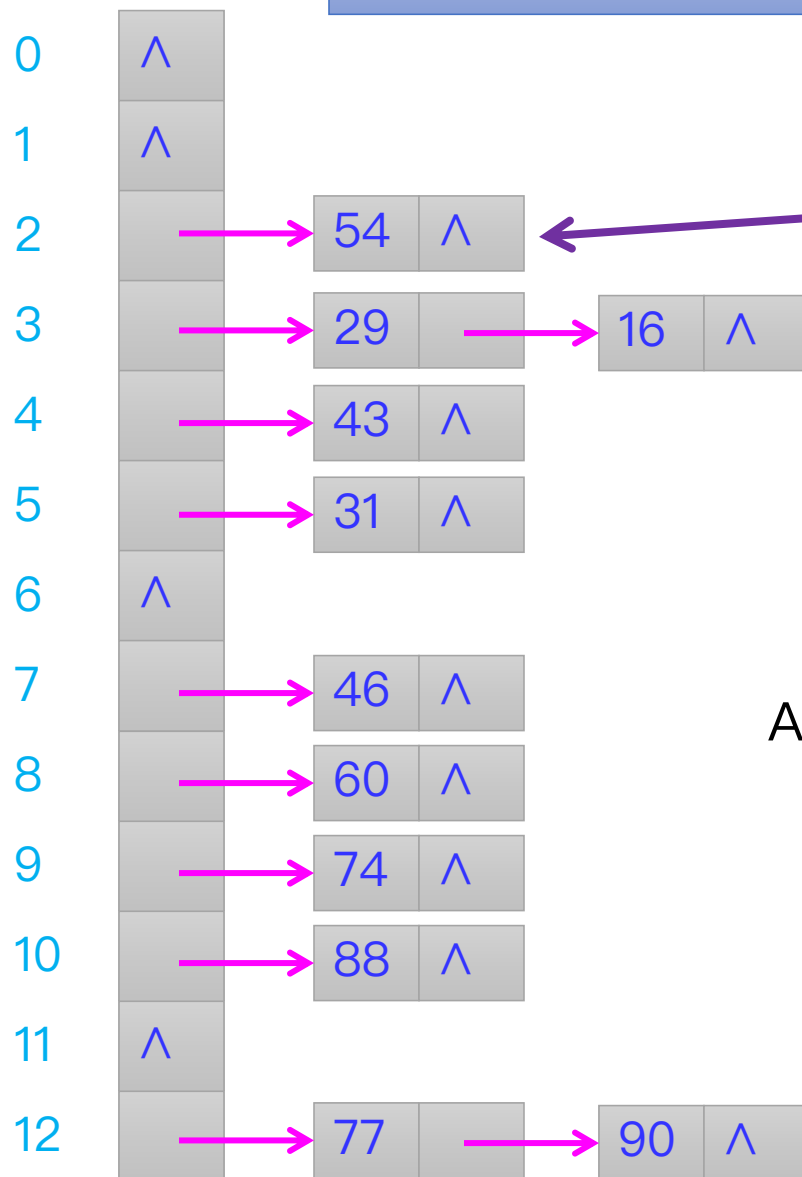
$p=NULL$ 。失败!

1次关键字比较

哈希表不成功查找完毕



② 拉链法不成功查找的ASL_{不成功}计算



有1个结点的单链表，不成功查找需要1次关键字比较，共有7个这样的单链表
有2个结点的单链表，不成功查找需要2次关键字比较，共有2个这样的单链表

$$ASL_{\text{不成功}} = \frac{1 \times 7 + 2 \times 2}{13} = 0.846$$

$\alpha = n/m$

【例9.11】 用关键字序列{7, 8, 30, 11, 18, 9, 14}构造一个哈希表，哈希表的存储空间是一个下标从0开始的一维数组，哈希函数为 $H(key)=(key \times 3) \bmod 7$ ，处理冲突采用线性探测法，要求装填（载）因子为0.7。

(1) 画出所构造的哈希表。

(2) 分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

解：（1）这里 $n=7$ ， $\alpha=0.7=n/m$ ，则 $m=n/0.7=10$ 。计算各关键字存储地址的过程如下：

$$H(7)=7 \times 3 \bmod 7=0$$

$$H(8)=8 \times 3 \bmod 7=3$$

$$H(30)=30 \times 3 \bmod 7=6$$

$$H(11)=11 \times 3 \bmod 7=5$$

$$H(18)=18 \times 3 \bmod 7=5 \quad \text{冲突}$$

$$d_1=(5+1) \bmod 10=6 \quad \text{仍冲突}$$

$$d_2=(6+1) \bmod 10=7$$

$$H(9)=9 \times 3 \bmod 7=6 \quad \text{冲突}$$

$$d_1=(6+1) \bmod 10=7 \quad \text{仍冲突}$$

$$d_2=(7+1) \bmod 10=8$$

$$H(14)=14 \times 3 \bmod 7=0 \quad \text{冲突}$$

$$d_1=(0+1) \bmod 10=1$$



下标	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	
探测次数	1	2		1		1	1	3	3	

查找成功的平均查找长度计算

下标	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	
探测次数	1	2		1		1	1	3	3	

(2) 在等概率情况下:

$$ASL_{\text{成功}} = \frac{1 + 2 + 1 + 1 + 1 + 3 + 3}{7} = 1.71$$

查找不成功的平均查找长度计算

下标	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	
探测次数	1	2		1		1	1	3	3	
不成功的探测次数	3	2	1	2	1	5	4	3	2	1

(2) 在等概率情况下:

$$H(key) = (key \times 3) \bmod 7$$



任何关键字不属于{7, 8, 30, 11, 18, 9, 14}的关键字 x , $H(x)$ 在0~6之间

$$ASL_{\text{不成功}} = \frac{3 + 2 + 1 + 2 + 1 + 5 + 4}{7} = 2.57$$



哈希表在查找成功时的平均查找长度（ ）。

- A. 与处理冲突方法有关而与装填因子 α 无关
- B. 与处理冲突方法无关而与装填因子 α 有关
- C. 与处理冲突方法有关且与装填因子 α 有关
- D. 与处理冲突方法无关且与装填因子 α 无关

答： C。